Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Воронежский государственный технический университет»
(ФГБОУ ВО «ВГТУ», ВГТУ)

Факультет информационных технологий и компьютерной безопасности
Кафедра систем информационной безопасности
Направление подготовки (специальность) 10.05.03 Информационная безопасность автоматизированных систем
(код и наименование направления подготовки)
Профиль/программа/направленность Обеспечение информационной безопасности распределённых информационных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Дипломная работа
(вид выпускной квалификационной работы – бакалаврская работа, дипломная работа (проект), магистерская диссертация)
Катюрин Дмитрий Александрович
(фамилия, имя, отчество обучающегося)

Тема: Нейросетевая реализация электронного курса освоения основ кибербезопасности: алгоритмическое и программное обеспечение

Состав выпускной квалификационной работы:

Расчетно-пояснительная записка на 164 страницах
Графическая часть на _____ листах

Расчетно-пояснительная записка к выпускной квалификационной работе:

Заведующий кафедрой _____ (подпись) (А.Г. Остапенко) (инициалы, фамилия)

Руководитель _____ (подпись) (Е.А. Москалева) (инициалы, фамилия)

Консультанты:

по _____ (наименование раздела, подпись) ( ) (инициалы, фамилия)

по _____ (наименование раздела, подпись) ( ) (инициалы, фамилия)

по _____ (наименование раздела, подпись) ( ) (инициалы, фамилия)

по _____ (наименование раздела, подпись) ( ) (инициалы, фамилия)

по _____ (наименование раздела, подпись) ( ) (инициалы, фамилия)

Обучающийся _____ (подпись) Д.А. Катюрин (инициалы, фамилия)

«02» 04 2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГТУ», ВГТУ)

Факультет информационных технологий и компьютерной безопасности
Кафедра систем информационной безопасности
Направление подготовки/специальность 10.05.03 Информационная безопасность автоматизированных систем
(код, наименование)
Профиль/программа/направленность Обеспечение информационной безопасности распределённых информационных систем

Утверждаю _____
(дата)
Зав.кафедрой _____
(подпись)

ЗАДАНИЕ
ПО ПОДГОТОВКЕ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Дипломная работа
дипломная работа (проект), бакалаврская работа, магистерская диссертация

Обучающемуся Катюрину Дмитрию Александровичу
фамилия, имя, отчество

1. Тема: Нейросетевая реализация электронного курса освоения основ кибербезопасности: алгоритмическое и программное обеспечение

ФГБОУ ВО «ВГТУ», ВГТУ
по заданию какой организации выполняется работа

приказ №01-1-10/3-2979 от 25.06.2025
утверждена приказом по университету

2. Срок сдачи обучающимся выпускной квалификационной работы 02.07.2025

3. Исходные данные

4. Краткое содержание выпускной квалификационной работы

Раздел 1 Определение особенностей реализации разрабатываемого нейросетевого курса видеолекций дисциплины «Введение в специальность»
перечень подлежащих разработке в выпускной квалификационной работе вопросов

Раздел 2 Алгоритмическая и программная реализация системы генерации нейросетевого «аватара»

Раздел 3 Алгоритмическая и программная реализация образовательной цифровой площадки

5. Перечень графического материала (с точным указанием обязательных чертежей по разделам

6. Консультанты (с указанием относящихся к ним разделов)

График выполнения выпускной квалификационной работы

№ п/п	Наименование разделов выпускной квалификационной работы	Срок выполнения этапов работы
1	Введение	22.03.2025 – 22.05.2025
2	Особенности реализации разрабатываемого нейросетевого курса видеолекций дисциплины «Введение в специальность»	22.03.2025 – 22.05.2025
3	Алгоритмическая и программная реализации системы генерации нейросетевого «аватара»	23.05.2025 – 31.05.2025
4	Алгоритмическая и программная реализация образовательной цифровой площадки	01.06.2025 – 24.06.2025
5	Заключение	25.06.2025 – 02.07.2025

Руководитель _____ (подпись)

Дата выдачи задания 22.03.2025

Задание принял к исполнению 22.03.2025 (дата)

Подпись обучающегося _____

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Воронежский государственный технический университет»

АННОТАЦИЯ
выпускной квалификационной работы

Обучающегося Катюрин Дмитрий Александрович

Тема ВКР: Нейросетевая реализация электронного курса освоения основ кибербезопасности: алгоритмическое и программное обеспечение

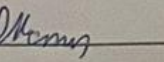
Характеристика выпускной квалификационной работы

Актуальность исследования: Актуальность темы обусловлена резким ростом количества киберугроз и несоответствием традиционных образовательных методов когнитивным особенностям молодежи, подверженной «клиповому мышлению», что снижает эффективность подготовки специалистов в области информационной безопасности. Решение проблемы требует переработки традиционных методов обучения, учитывая особенности восприятия информации в условиях «клипового мышления», при этом обеспечивая сохраняя системность знаний.

Задачи, решаемые в ВКР: исследование и разработка концептуального и технического подхода к реализации нейросетевого электронного курса видеолекций освоения основ кибербезопасности, включающего определение технических особенностей системы, проектирование её архитектуры, модели функционирования и взаимодействия компонентов; алгоритмизация и программная реализация модулей для создания нейросетевого «аватара», способного автоматически озвучивать лекционный материал с возможностью персонализации голоса диктора, согласования визуального образа с семантикой видеолекции в целях адаптации к когнитивным особенностям аудитории; разработка цифровой площадки, позволяющей провести апробацию сгенерированного нейросетевого курса, с возможностью проверки знаний студентов, а также обратной связью с целью дополнения и улучшения существующего.

Структура ВКР: особенности реализации разрабатываемого нейросетевого курса видеолекций дисциплины «Введение в специальность»; алгоритмическая и программная реализации системы генерации нейросетевого «аватара»; алгоритмическая и программная реализация образовательной цифровой площадки.

Краткая характеристика полученных результатов: изучены особенности разработки электронного нейросетевого курса, направленного на освоение основ кибербезопасности; создан нейросетевой инструментарий, способный генерировать видеолекции произвольной продолжительности с возможностью персонализации визуального облика и звучания виртуального «аватара»; разработана образовательная цифровая платформа, предназначенная для хранения и воспроизведения подготовленных материалов, а также для проведения апробации учебного курса с целью повышения его эффективности.

Автор ВКР  Катюрин Дмитрий Александрович

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 7
1 Особенности реализации нейросетевого разрабатываемого курса видеолекций дисциплины «Введение в специальность» 12
1.1 Когнитивные тренды поколений Z и Альфа в условиях тотальной цифровизации 12
1.2 Технические ограничения технологии создания нейросетевого «аватара» 22
1.3 Классификация методов генерации нейросетевых «аватаров» 24
1.4 Архитектура взаимодействия компонентов системы генерации нейросетевого «аватара» 34
1.4.1 Требования к системе генерации нейросетевого «аватара» 34
1.4.2 Компоненты системы нейросетевого «аватара» 35
1.4.3 Взаимодействие компонентов системы нейросетевого «аватара» 37
1.5 Архитектуры взаимодействия компонентов образовательной цифровой площадки 39
1.5.1 Требования к компонентам веб-приложения 39
1.5.2 Компоненты клиентской части веб-приложения 40
1.5.3 Компоненты серверной части веб-приложения 42
1.5.4 Взаимодействие компонентов и обеспечение безопасности веб-приложения 44
2 Алгоритмизация и программная реализации системы генерации нейросетевого «аватара» 48
2.1 Модуль генерации голоса 48
2.1.1 Структура модуля 48
2.1.2 Алгоритм генерации голоса 49
2.2 Модуль определения фонем 51
2.2.1 Структура модуля 51
2.2.2 Алгоритм анализа и сегментации голоса 52

2.2.3 Алгоритм определения фонем ... 60
2.2.4 Алгоритм разметки фонем ... 66
2.2.5 Алгоритм преобразования фонем в покадровый формат ... 74
2.3 Модуль захвата особенностей лица ... 80
2.3.1 Структура модуля ... 80
2.3.2 Алгоритм получения ключевых точек лица ... 81
2.3.3 Алгоритм захвата особенностей лица ... 87
2.4 Модуль захвата движений головы ... 94
2.4.1 Структура модуля ... 94
2.4.2 Алгоритм вычисления параметров движения головы ... 96
2.5 Модуль генерации «аватара» ... 104
2.5.1 Структура модуля ... 104
2.5.2 Алгоритм создания «аватара» ... 105
3 Алгоритмизация и программная реализация образовательной цифровой площадки ... 115
3.1 Базы данных информационного обеспечения образовательной цифровой площадки ... 115
3.1.1 Архитектура базы данных ... 115
3.1.2 Таблицы общего назначения ... 115
3.1.3 Таблицы системы тестирования ... 118
3.1.4 Таблицы системы лабораторных работ ... 124
3.2 Образовательная цифровая площадка ... 131
3.2.1 Подсистема просмотра видеолекций ... 131
3.2.2 Подсистема аутентификации ... 134
3.2.3 Подсистема тестирования ... 140
3.2.4 Подсистема лабораторных работ ... 143
3.3 Обеспечение безопасности веб-приложения образовательной цифровой площадки ... 145
ЗАКЛЮЧЕНИЕ ... 150
СПИСОК ЛИТЕРАТУРЫ ... 152

ВВЕДЕНИЕ

В эпоху цифровой трансформации кибербезопасность становится ключевым элементом защиты информационных систем, что подразумевает актуальность подготовки специалистов информационной безопасности, способных противостоять киберугрозам. Однако традиционные образовательные форматы сталкиваются с вызовом клипового мышления – доминирующего среди молодежи стиля восприятия информации через короткие, визуально насыщенные фрагменты [1]. Это явление, характеризующееся рассеянным вниманием, поверхностным анализом данных и сложностью усвоения сложных логических конструкций, ставит под угрозу эффективность обучения дисциплинам, где критически важны системность и глубина знаний [2]. Привычные формы подачи материала – лекции, текстовые материалы и стандартные тесты теряют эффективность, так как не соответствуют когнитивным паттернам молодого поколения, выросшего в условиях информационного перегруза [3-4]. В этой связи возникает необходимость пересмотреть подходы к созданию обучающего материала, опираясь на исследования, раскрывающие специфику клипового мышления и его влияние на образовательные результаты.

В своей статье «Клиповое мышление в контексте образовательного процесса» авторы указывают на фрагментарность мышления нынешних студентов и доминирование эмоционально заряженных визуальных образов [5]. После традиционных лекционных занятий и прохождения тестирования на изученную тематику у студентов наблюдается краткосрочное запоминание терминов без понимания и функциональной взаимосвязи, что приводит к отсутствию остаточных знаний и ставит под угрозу формирование компетенций в области информационной безопасности. Ключом к решению данной проблемы является интеграция в образовательный процесс методов, адаптированных под клиповое мышления: визуализация данных, интерактивность и геймификация, в соответствии с теорией когнитивного обучения Р. Майера [6].

Однако реализация таких адаптированных методик сталкивается с принципиальными сложностями в условиях динамично развивающейся сферы информационной безопасности. Создания интерактивных лекций, которые включали бы в себя

методики клипового мышления, требуют значительных временных ресурсов на разработку и постоянную актуализации контента, в ответ на появление новых киберугроз. Разработка контента включает в себя не только подготовку актуальных данных и визуализацию ключевых тезисов, но и запись лектора с последующим сведением в видеоредакторе. Решением может стать интеграция нейросетевых технологий, уже доказавших свою эффективность в смежных сферах. В сфере информационной безопасности, нейроинструментарий используется для анализа вредоносного ПО, а также выявления аномального трафика [7]. В сфере образования нейросетевые технологии много лет используются для обработки результатов экзаменов ЕГЭ. Применение нейросетевых технологий позволяет обрабатывать большое количество информации и уменьшать количество рутинных действий человека.

Одним из перспективных направлений в области нейросетевых технологий является генерация нейросетевого диктора. Исследования в этой области активно развиваются с 2023 года, благодаря росту вычислительных мощностей и интереса к искусственному интеллекту [8-10]. Современные вычислительные системы позволяют клонировать тональность и стиль речи человека, а также генерировать реалистичного «аватара». Такие технологии активно внедряются в различных сферах: от профессиональной озвучки новостных лент и автоматизированного производства рекламных роликов до интеграции в «умные» устройства и разработки интерактивных голосовых помощников. Данную технологию можно использовать для создания автоматизированного рассказчика видеолекций. Это не только упростит процесс создания и обновления контента, но и повысит качество лекций за счёт их адаптации под особенности клипового мышления аудитории.

Анализ существующих методов создания обучающего контента и развития нейросетевых систем генерации видеодиктора позволяет сделать вывод об актуальности темы исследования с учетом наличия следующих противоречий между:

– объективной необходимостью адаптации образовательных программ высших учебных заведений к современным тенденциям нейросетевых технологий и отсутствием готовых методологий реализации электронного курса видеолекций,

сочетающих научную глубину с эмоциональной вовлечённостью через яркие визуальные образы, геймификацию и интерактивность;

– потребностью в генерации продолжительных, реалистичных видеолекций с цифровым аватаром, как средства преодоления ограничений клипового мышления и повышения вовлеченности, и отсутствием готовых нейросетевых решений, способных обеспечить такой уровень качества и длительности видеогенерации;

– необходимостью в оперативной объективной оценке эффективности и постоянном улучшении нейросетевого обучающего контента и отсутствием специализированных цифровых площадок, предоставляющих инструментарий для комплексной апробации, тестирования и анализа обратной связи;

Объектом исследования выступают студенты университетов, а также абитуриенты, следующие современным тенденциям клипового мышления.

Предмет исследования – нейросетевой инструментарий реализации электронного курса основ кибербезопасности.

Целью исследования является алгоритмическое и программное обеспечение нейросетевой реализации электронного курса освоения основ кибербезопасности.

Задачи, решаемые в ходе исследования:

1) Исследование и разработка концептуального и технического подхода к реализации нейросетевого электронного курса видеолекций освоения основ кибербезопасности, включающего определение технических особенностей системы, проектирование её архитектуры, модели функционирования и взаимодействия компонентов.

2) Алгоритмизация и программная реализация модулей для создания нейросетевого «аватара», способного автоматически озвучивать лекционный материал с возможностью персонализации голоса диктора, согласования визуального образа с семантикой видеолекции в целях адаптации к когнитивным особенностям аудитории.

3) Разработка цифровой площадки, позволяющей провести апробацию сгенерированного нейросетевого курса, с возможностью проверки знаний студентов, а также обратной связью с целью дополнения и улучшения существующего.

На защиту выносятся следующие результаты:

1) Результаты анализа и проектирования модулей, включающие технические требования, архитектуру нейросетевого модуля, сценарии взаимодействия компонентов и подходы к расширению функциональности для образовательной платформы.

2) Нейросетевой инструментарий для автоматического создания реалистичных аватаров, позволяющий создавать диктора для видеолекций электронного курса освоения основ кибербезопасности.

3) Интерактивная образовательная платформа с библиотекой видеолекций, системой автоматизированной проверки знаний, позволяющая получать обратную связь от студентов.

Результаты, выносимые на защиту, имеют следующую новизну:

1) Предложены оригинальные технические решения для нейросетевого инструментария генерации «аватара», включающие уникальную архитектуру системы, модульную организацию компонентов и методы их взаимодействия.

2) Впервые разработан нейросетевой инструментарий для генерации видеолекций с неограниченной по продолжительности, в отличие от существующих аналогов, имеющих временные ограничения на создаваемые видеофрагменты.

3) Разработанная интерактивная образовательная платформа в отличие от существующих аналогов позволяет не только просматривать видеолекции, но и получать обратную связь от студентов, а также проводить проверку знаний.

Теоретическая значимость результатов дипломного проектирования состоит в следующем:

1) Предложенные методы синтеза «аватаров» вносят значимый теоретический вклад в развитие генеративного искусственного интеллекта, существенно расширяя возможности автоматизации создания образовательного видеоконтента, а также обеспечивают адаптацию учебных материалов к специфике клипового мышления.

2) Разработанные методики создания «аватаров» могут служить основой для разработки аналогичных моделей, ориентированных на решение узкоспециализированных прикладных задач.

3) Сформулированные технические принципы, охватывающие архитектуру системы, реализацию её компонентов и механизмы их взаимодействия, способствуют стандартизации процессов разработки нейросетевых решений. Это позволяет повысить уровень формализации и стандартизации процессов разработки систем искусственного интеллекта, обеспечивая прозрачность алгоритмических решений.

Практическая ценность результатов дипломного проектирования состоит в следующем:

1) Подробное техническое описание, охватывающее функционирование и взаимодействие компонентов разработанного нейросетевого инструментария, послужит опорой для разработчиков. Оно позволяет расширять и адаптировать систему под новые образовательные задачи.

2) Разработанный нейросетевой инструментарий предоставляет практические средства для автоматизации создания видеолекций курса освоения основ кибербезопасности, что позволяет адаптировать обучающие программы к специфике клипового мышления.

3) Разработанная интерактивная образовательная платформа позволяет провести апробацию созданного электронного курса посредством сбора обратной связи от студентов, а также проведением проверки остаточных знаний путем автоматизированного тестирования.

1 Особенности реализации нейросетевого разрабатываемого курса видеолекций дисциплины «Введение в специальность»

1.1 Когнитивные тренды поколений Z и Альфа в условиях тотальной цифровизации

В эпоху цифровой трансформации технологии стали неотъемлемой частью повседневности, пронизывая все аспекты жизни. Они используются как для профессиональной деятельности, так и для личного общения. Компьютеры, смартфоны, умные часы и другие гаджеты, постоянно подключены к глобальной сети. Эти устройства формируют не просто инструментарий, а целую экосистему, где коммуникации, развлечения и обучение сливаются в единый цифровой поток. Поколение Z — молодые люди, рождённые между 2000 и 2010 годами, взрослеют в уникальных условиях информационного изобилия [11]. Для них поиск ответа на любой вопрос превратился в рутинный процесс, занимающий доли секунды. Определение научного термина, рецепт блюда и получение новостной сводки за день стали мгновенными операциями. Это поколение не просто использует технологии, оно формируется и живет внутри этой цифровой экосистемы с самого сознательного возраста. Их опыт кажется переходным по сравнению с абсолютной погруженностью в цифровой мир следующего поколения. Их младшие братья и сестры, представители альфа-поколения, родившиеся примерно с 2010 года погружены в цифровую среду буквально с рождения. Эти дети не просто взрослеют с технологиями. Они не знают мира без мгновенного доступа в интернет, сенсорных экранов и голосовых помощников. Их первые шаги в обучении часто связаны с интерактивными приложениями, построенные с элементами геймификации и игрофикации. Эти цифровые инструменты активно дополняют, а иногда и заменяют традиционные игрушки и книги, формируя принципиально новые основы познания и взаимодействия с миром.

Достаточно ввести запрос в строку поисковика браузера и алгоритмы, основанные на машинном обучении, мгновенно предложат сотни вариантов контента, ранжированных по релевантности. Среди них могут быть экспертные статьи из

рецензируемых журналов, блоги авторитетных специалистов, видеолекции на различных образовательных платформах, подкасты с дискуссиями и интерактивные инфографики. Например, поиск по запросу «информационная безопасность» пользователь сразу получит базовые понятия из данной области, а также безграничное количество обучающего материала по различным аспектам информационной безопасности: технические подробности новых средств защиты информации, правовые нормы и принципы криптографии.

Особое место в этом потоке занимают нейросетевые ассистенты, такие как ChatGPT, DeepSeek и Qwen которые появились несколько лет назад. Эти системы, обученные на сотнях терабайтах текстовых данных, выходят за рамки простой выдачи ссылок [12]. Чат боты способны не только сжать сложную научную работу до краткого резюме, но и адаптировать объяснение под уровень знаний пользователя: студент получит развёрнутый анализ с примерами, а школьник – упрощённую аналогию. Новейшие нейросетевые ассистенты фокусируется на точности фактов, сверяя ответы с актуальными базами данных.

Ключевой особенностью таких ассистентов является имитация естественной беседы. Пользователь может уточнять детали, менять контекст или просить привести альтернативные точки зрения, что превращает поиск информации в динамичный диалог. Это контрастирует с ранними этапами цифровой эры, когда взаимодействие с технологиями напоминало работу со статичной энциклопедией.

Экспоненциальный рост объема создаваемой человечеством информации оказывает глубокое влияние на повседневную жизнь. Увеличение масштабов создаваемых данных сопровождается ростом их потребления. Молодое поколение в свободное время активно взаимодействует с соцсетями и видеохостингами, дополнительно насыщаясь информацией. Они привыкли к постоянному мощному информационному потоку. При этом сама информация должна быть легкоусвояемой, не требующей глубокого осмысления. Из-за ее огромного количества потребляемая информация неизбежно упрощается. Поэтому в социальных сетях доминируют изображения с краткими провокационными заголовками и короткие развлекательные ролики, в то время как объёмные текстовые анализы теряют аудиторию из-за низкого времени

просмотра и склонности пользователей к поверхностному ознакомлению. Этот сдвиг в сторону краткости и визуальности – не просто предпочтение, а реакция на новые условия. Постоянное потребление больших объёмов данных порождает информационную перегрузку. Такое состояние возникает, когда избыток информации провоцирует снижение концентрации, усталость, повышенную раздражительность и рост уровня стресса. В ответ на этот вызов мозг вырабатывает защитные механизмы. Фрагментарное, клиповое восприятие становится ключевой стратегией, позволяющей отбросить избыточную информацию и избежать когнитивного истощения в условиях непрерывного «информационного шума» [13].

Легкодоступность информации радикально изменила паттерны мышления. Раньше освоение знаний требовало систематического труда. Требовались походы в библиотеки, вдумчивый разбор книг с выделением основных тезисов, ручное конспектирование. Теперь же этот процесс зачастую сводится к нескольким запросам в поисковике или чат-боте. Получаемые краткие ответы, хотя и дают нужную информацию, зачастую лишены глубины контекста и нюансов. Молодое альфа поколение и зумеры, привыкшие совмещать домашние задания с перепиской в мессенджерах и общением в голосовых чатах, демонстрируют фрагментарность мышления. Она выражается в частом переключении между задачами, зачастую без глубокой проработки ни одной из них. Этот подход к поиску информации формирует когнитивные привычки, при которых длительная концентрация на одной сложной задаче становится все труднее.

Парадокс в том, что знания остаются сложными для усвоения, несмотря на кажущееся изобилие данных. Огромное количество источников информации создаёт иллюзию её доступности, но на практике приводит к «информационному шуму». Пользователи редко сравнивают и анализируют разные точки зрения, ограничиваясь выборочным усвоением фактов. Разнородность и распространенность информации снижают мотивацию к самостоятельному анализу и формированию собственных выводов, которые и запоминаются лучше всего. Вместо чтения учебников и глубокого разбора тем ученики и студенты всё чаще выбирают краткие текстовые и видеообзоры. В них материал сокращается до минимума, игнорируя детали, необходимые для

полноценного понимания. Это порождает поверхностное знание концепций, а не освоение теоретической базы. Подобный когнитивный стиль становится основой фрагментарного мышления, формированию которого способствуют несколько ключевых факторов [14]:

– информационная перегрузка. Ежедневное погружение в поток новостей из соцсетей, развлекательных роликов, непрерывный обмен сообщениями в мессенджерах и агрессивная реклама, провоцирующие когнитивную усталость и уменьшение фокуса внимания;

– информационное разнообразие. Поток данных из множества источников, которые могут противоречить друг другу, создавая иллюзию выбора, но усложняя попытки выстроить целостную картину;

– культура многозадачности. Параллельное потребление контента: чтение конспекта лекции совмещая с просмотром развлекательных видео формируют «рваный» ритм внимания;

– клиповый формат восприятия. Тенденция предпочитать информацию, подающуюся короткими, динамичными и эмоционально насыщенными порциями (клипами, постами, мемами), вместо длинных текстов или сложных нарративов. Этот формат стал доминирующим в цифровую эпоху, радикально меняя способы взаимодействия с контентом;

– снижение критического мышления. Проявляется в нежелании глубоко анализировать информацию, проверять её достоверность и устанавливать причинно-следственные связи из-за большой скорости информационного потока.

Клиповое мышление формирует привычку к усвоению информации через краткие тезисы, вытесняя системное восприятие. Человек концентрируется на отдельных фрагментах, но теряет способность к длительной фокусировке на сложных материалах. При этом многозадачность, часто ассоциируемая с цифровой эпохой, является некой иллюзией. Когнитивная нагрузка распределяется между задачами. Человеческий мозг в каждый конкретный момент может выполнять только одну интеллектуальную задачу. Рассеивание внимания, характерное для зумеров и альфа поколения,

снижает наблюдательность и устойчивость к внешним отвлекающим факторам. Этот аспект особенно заметен в образовательной среде.

Современное образование сталкивается с серьёзным вызовом. Традиционные методы преподавания, такие как классические лекции и учебники, становятся всё менее эффективными для поколения, сформированного эпохой коротких видеороликов и мгновенного доступа к информации. Молодёжь быстро теряет интерес к продолжительным монологам преподавателей и объёмным текстовым материалам. Способность к концентрации внимания у таких студентов ослабевает уже через несколько минут. Когнитивное восприятие молодого поколения адаптировано к ярким визуальным образам, частой смене сцен и интерактивной обратной связи, характерной для соцсетей и игр [15].

Для повышения вовлечённости учащихся необходим пересмотр подходов к подаче учебного материала. Первоочередным шагом является переход от объёмных текстовых материалов к формату наглядных презентаций. В презентациях важно минимизировать количество текста, оставляя только базовые тезисы. Следует использовать цветовые акценты для выделения важных моментов. Презентации следует наполнять контрастными схемами, анимацией и краткими ключевыми тезисами, обеспечивающими мгновенное визуальное восприятие. Материал следует разбивать на компактные фрагменты по 7–10 минут, сопровождая их краткими выводами между блоками. Такой подход помогает сохранить концентрацию и построить логическую связь между частями материала. Это делает процесс обучения более динамичным и схожим с привычным для альфа-поколения контентом. Он фокусирует внимание студентов на основных тезисах, подкрепляя устное изложение визуальной составляющей.

Помимо визуализации информации, мощным инструментом повышения вовлечённости является геймификация учебного процесса – применение игровых механик в неигровом контексте [16]. Этот подход создаёт комфортную среду на занятиях, повышая мотивацию, эффективность и качество обучения. Такой подход позволяет моделировать профессиональные ситуации, в которых решение задач способствует повторению и усвоению материала. Существует множество платформ, реализующих интерактивное обучение: Stepik, Coursera, Kahoot, Wooclap и другие. Они включают

задания, которые необходимо выполнять онлайн, проверяя корректность действий в реальном времени. Также существуют и специализированные платформы для конкретных областей, таких как программирование (JavaRush, ITVDN, Ката Академия). Они возникли как ответ на неэффективность традиционных методов обучения. Например, необходимость перепечатывать код из книги самоучителя для решения задач отнимает время и демотивирует, приводя к пропуску заданий. На платформах код пишется и проверяется онлайн, что устраняет рутину. Курсы содержат обязательные и дополнительные задачи. Обязательные задачи блокируют продвижение дальше без их решения, обеспечивая усвоение ключевых концепций. Необязательные задания повышенной сложности предоставляют возможность для дополнительного развития навыков и получения виртуальных бонусов. Этот структурированный подход, основанный на немедленной обратной связи и преодолении постепенно усложняющихся вызовов, напрямую отвечает потребностям поколения, выросшего в цифровой среде. Он превращает обучение в увлекательный и результативный процесс.

Эффективность геймификации в цифровом обучении демонстрирует, что особенности клипового мышления могут быть конструктивно использованы в образовательной среде. Переход в цифровую образовательную среду меняет восприятие клипового мышления. Потребность в быстром доступе к информации становится преимуществом и главным критерием дизайна онлайн школы. Образовательные платформы используют склонность к частой смене задач, структурируя материал на небольшие законченные блоки. Доступ к ним открывается последовательно. Учащемуся предоставляется выбор из нескольких одновременно активных тем (обычно 3-5), что формирует мотивацию к их освоению. Такой подход создает ощущение понятной и достижимой цели. Малый объём доступных для решения задач не подавляет мотивацию. При этом необходимость завершить часть из них для доступа к новым разделам поддерживает учебную динамику. Просмотр далеких от текущего прогресса разделов ограничен, чтобы предотвратить демотивацию из-за кажущегося непосильного объема материала и ощущения «необъятной книги». Частое переключение внимания, которое мешает глубокому погружению, компенсируется за счёт разнообразия форматов подачи материала. Текст дополняется иллюстрациями, видео и интерактивными

заданиями, что поддерживает устойчивый интерес и предотвращает снижение вовлеченности. Распространенным решением является использование формата ленты из социальных сетей, где учебный материал структурирован в лаконичные смысловые блоки. Каждый модуль завершается кратким обязательным тестом или заданием. Практическое задание используется для подтверждения усвоение информации. Таким образом, цифровые платформы трансформируют изначально проблемные аспекты клипового мышления в инструменты поддержки мотивации и переработанного структурированного познания.

Помимо адаптации к когнитивным особенностям учащихся, цифровые платформы позволяют эффективно масштабировать образовательный процесс. Традиционное требование к небольшим группам для обеспечения индивидуального внимания преподавателя сталкивается с ограничениями при массовом обучении. Образовательные платформы преодолевают это ограничение за счёт автоматизации ключевых процессов. Для каждого учащегося предусмотрено персонализированное тестирование, а проверка заданий выполняется автоматически, обеспечивая мгновенную обратную связь. Возможна реализация проверки рукописных работ с применением технологий и алгоритмов компьютерного зрения. Это позволяет сохранять высокий уровень объективности и регулярности контроля независимо от количества обучающихся. Широкое распространенность образовательных платформ, направленных на подготовку к экзаменам, и их сопоставимая с малыми группами результативность подтверждают эффективность такого подхода. Возможность проведения онлайн-занятий для значительной аудитории демонстрирует, что цифровые инструменты не только адаптируют обучение под современные реалии восприятия, но и расширяют доступность образовательного контента и охват без ущерба для качества.

Формат видеолекций стал ответом на требования современного восприятия учебного материала. Такой подход позволяет сочетать визуальные и аудиоэлементы. На экране демонстрируются яркие слайды с ключевыми тезисами, инфографикой и анимацией, а диктор рассказывает лекцию. Такой формат удобнен для молодого альфа-поколения и зумеров, которым привычно одновременно обрабатывать информацию через разные каналы восприятия. Они успевают визуально считывать

материал с презентации и слушать диктора. Для эффективного усвоения и записи материала в видеолекции необходимо создавать короткие паузы между блоками информации. Эти преднамеренные остановки дают учащимся время осмыслить информацию, сделать заметки и переключить внимание.

Материал разрабатывается преподавателем с опорой на актуальные данные. Это особенно актуально в сфере информационной безопасности, где ежедневно возникают новые угрозы и методы защиты. Также подготавливается наглядная презентация, которая визуально иллюстрирует этот озвучиваемый материал. Важным аспектом является включение в обучение свежих данных и реальных практических примеров, подготовленных действующими сотрудниками и экспертами в области ИБ. Обучение должно сочетать теоретическое описание с практическими примерами для всестороннего раскрытия темы. Формат презентации подходит для иллюстрации материала с помощью графиков, таблиц и схем. Также презентации позволяют встраивать видеофрагменты с практической демонстрацией рассматриваемых тем. Практические примеры особенно ценны, так как показывают применение теоретических знаний в смоделированных ситуациях. Такой подход значительно углубляет понимание новых и сложных концепций.

Чтобы усилить эффект, видеолекции необходимо дополнить цифровым «аватаром» – нейросетевым персонажем, который заменяет традиционного диктора. Такой «аватар» становится «лицом» курса. Его образ выбирается под целевую аудиторию. Важно, чтобы «аватар» был запоминающимся и вызывал эмоциональный отклик. При этом он не должен быть слишком ярким или контрастным. Избыточная контрастность приведет к потере внимания, отвлекая от основного содержания лекций. Главная задача «аватара» – усилить подачу материала, не конкурируя с ним. Нейросетевые технологии обеспечивают возможность гибко настраивать внешность и образ цифрового персонажа в зависимости от тематики лекции. Например, при раскрытии вопросов правовой охраны данных «аватар» может демонстрировать деловой стиль одежды, напоминающий внешность юриста или специалиста в области информационной безопасности. Для тем, связанных с анализом кибератак, уместно использовать образ хакера, для ассоциации с киберпреступностью, что способствует более глубокому

погружению в содержание за счёт соответствия образа изучаемому материалу. Узнаваемый облик «аватара», сбалансированный дизайн, умеренная визуальная активность и нейтральная цветовая гамма помогут сохранить фокус на образовательной составляющей, добавив курсу современности и динамики.

Использование нейросетевого «аватара» позволяет ускорить и удешевить создание видеоконтента. Технология исключает необходимость организации съемочного процесса: не требуется подготовка помещения, настройка освещения, наличие оборудования для записи звука и видео. Кроме того, нейросеть избавляет от множества рутинных задач, таких как запись повторных дублей из-за ошибок диктора. Для генерации контента достаточно подготовить текст для озвучивания и презентацию, которая будет синхронизирована с повествованием. Это сокращает время производства контента в разы и минимизирует расходы на техническую инфраструктуру.

Еще одно преимущество нейросетевого «аватара» заключается в снижении затрат на актуализацию учебных материалов. Это особенно критично для сферы информационной безопасности, где ежегодно появляются новые киберугрозы, методы атак и стандарты защиты, что требует частого обновления контента. «Аватар» позволяет вносить изменения точечно, без необходимости полной перезаписи видео диктора. Достаточно обновить материал и сгенерировать обновлённый фрагмент. При этом сохраняется единый стиль повествования, дикция, внешний вид и окружение. В результате процесс сводится к замене устаревшего фрагмента лекции на актуальный, что экономит время и ресурсы.

Нейросети стали неотъемлемой частью повседневной жизни студентов. Они активно используют их для поиска ответов, решения задач и анализа данных. Видеолекции с нейросетевым «аватаром» органично встраиваются в эту цифровую экосистему, так как формат соответствует привычным для альфа-поколения видеороликам и интерактивному контенту соцсетей.

Однако сама по себе технология «аватара» – лишь часть образовательного решения. Максимальный эффект достигается при интеграции с специализированной платформой, которая объединяет видеолекции, адаптивные тесты, практические задания с элементами геймификации, а также доступ к актуальным новостным

ресурсам по темам курса. Такой подход не только структурирует обучение, но и создает среду для погружения в предмет. Крайне важно, чтобы видеолекции занимали центральное место в интерфейсе платформы. Поколение альфа привыкло получать контент за 2-3 клика, и если материал «спрятан» в меню или требует поиска, то он будет восприниматься аудиторией как устаревший или неважный. При этом каждая видеолекция должна быть доступна как со стационарного устройства, так и с телефона, что позволит повторять материал в транспорте или во время перерывов.

Формат образовательной платформы позволяет реализовать систему апробации знаний через тестирования, которые становятся обязательным этапом освоения материала. Тестирование решает проблему поверхностного подхода, когда студенты не внимательно изучают материал лекции. Автоматическая проверка тестов обеспечивает объективность оценки, а также снижает нагрузку на преподавателей, освобождая от рутинной работы. Система мгновенно анализирует ответы и предоставляет результат. Дополнительно, система позволяет легко создавать уникальные варианты для каждого студента. Преподавателю достаточно разработать общий банк вопросов по теме, а платформа автоматически сгенерирует индивидуальный тест. Интеграция элементов геймификации усиливает вовлеченность, превращая обучение в активный процесс. Такой формат позволяет легко интегрировать сторонние демонстрации и практические примеры, которые наглядно демонстрируют практическое применение теоретических знаний.

В эпоху цифровой трансформации технологии прочно интегрировались в повседневную жизнь, особенно для альфа-поколения и зумеров, которые с детства погружены в цифровую экосистему. Постоянный доступ к информации через гаджеты сформировал новые когнитивные паттерны. Получение знаний становится мгновенно, но из-за колоссального роста количества создаваемой и потребляемой информации, знания становятся более поверхностными. Это привело к снижению концентрации, упрощению формата восприятия информации и доминированию клипового мышления, где предпочтение отдается кратким, визуально насыщенным сообщениям. Образование столкнулось с необходимостью адаптации. традиционные методы уступают место интерактивным платформам, геймификации, визуализации материала и

коротким видеолекциям, которые соответствуют ритму цифрового поколения. Нейросетевые ассистенты и цифровые «аватары» не только упрощают доступ к знаниям, но и делают обучение более персонализированным и эффективным. Современные образовательные подходы, опираются на особенности цифрового восприятия. Они формируют новую парадигму, где динамика, доступность и вовлеченность становятся аспектами к эффективному усвоению знаний в условиях быстро меняющегося мира.

1.2 Технические ограничения технологии создания нейросетевого «аватара»

Любая технология, включая создание нейросетевых «аватаров», имеет ряд фундаментальных ограничений, которые проявляются как на этапе разработки, так и в процессе её практического применения. Эти ограничения влияют на возможность расширения проекта, вектор развития технологии в будущем, доступность для конечных пользователей и восприятие контента аудиторией. При проектировании и разработке такого рода систем необходимо учитывать существующие ограничения сразу, чтобы не столкнуться с непреодолимыми барьерами [17]. Разработчикам всегда приходится балансировать между техническими возможностями, экономической и временной целесообразностью, чтобы достичь жизнеспособного результата. Выделим ключевые ограничения технологии нейросетевых «аватаров», которые необходимо учитывать при разработке и обучении.

Первым и наиболее значимым ограничением является зависимость времени генерации от разрешения изображения. Процесс создания каждого кадра видео требует большого количества вычислений от видеоускорителя, при этом увеличение размера создаваемого кадра в пикселях приводит к нелинейному росту затрачиваемого времени [18]. Для создания 1 минуты видео в стандартном и общепринятом разрешении 256x256 пикселей требуется 2-3 минуты аппаратного времени, что является достойным и эффективным результатом. При увеличении изображения в два раза до 512x512 пикселей время на создание одного кадра возрастёт минимум в четыре раза из-за

квадратичного увеличения количества пикселей. На практике замедленнее будет более существенным, из-за сложности используемых алгоритмов при генерации видео. Таким образом разработчикам приходится искать компромиссы между скоростью генерации видео и качеством.

Вторым критическим фактором становится высокие требования к аппаратным, составляющим системы, на которой происходит генерация. Основной упор приходится на количество видеопамяти, которое предоставляет видеоускоритель [19]. Видеопамять используется для вычисления параметров изображения, а также хранения уже созданных кадров. Инструменты генерации обрабатывают полностью всё видео сразу, что требует хранение тысячи параметров и готовых кадров. Это приводит к огромным требованиям на объём видеопамяти. Для создания 10 минутного видео, которое будет создаваться целиком, может потребоваться до 100 ГБ видеопамяти. Данный объём недоступен для потребительских графических ускорителей, и может быть достигнут только серверными решениями. Необходимо учитывать данное ограничение при проектировании и создании нейросетевого инструментария, на котором необходимо создавать видеолекции продолжительностью более одного часа.

Третье ограничение связано с феноменом «зловещей долины». Этот феномен может вызывать подсознательное отторжение и чувство дискомфорта у зрителя из-за почти реалистичного «аватара» [20]. Дискомфорт возникает из-за незначительных дифектов текстуры кожи лица, освещения, и ошибок анимации таких как неестественное движение глаз, задержки движения губ или странных покачиваний головой. Для минимизации этого эффекта в мультфильмах, анимационные студии на стадии постобработки добавляют незначительные шумы в звуковую дорожку персонажей, придают небольшую ассиметрию мимики и движений, а также изменяют идеальную текстуру лица персонажей.

Четвертое ограничение связано с высокой ресурсоёмкость процесса обучения модели нейросетевого «аватара» с нуля. Такая модель представляет собой комплекс взаимосвязанных нейросетевых компонентов, каждая из которых выполняет свою определенную функцию [21]. Создание такой модели требует слаженной работы каждого нейросетевого модуля. Кроме того, для обучения больших моделей требуются

огромные датасеты с тысячами часов размеченного видео. Сложность проектируемой модели требует огромного количества тестовых данных. Сбор, сортировка и разметка такого количества видео занимает сотни часов. Для хранения такого объёмного датасета может потребоваться террабайты свободного пространства хранилища. Также потребуются тысячи аппаратных часов для обучения собственной модели. Таким образом создание с нуля модели «аватара» потребует огромного количества потраченного времени на поиск и создание датасета, а также обучения модели.

Представим ограничения технологии создания нейросетевого «аватара» в виде схемы на рисунке 1.

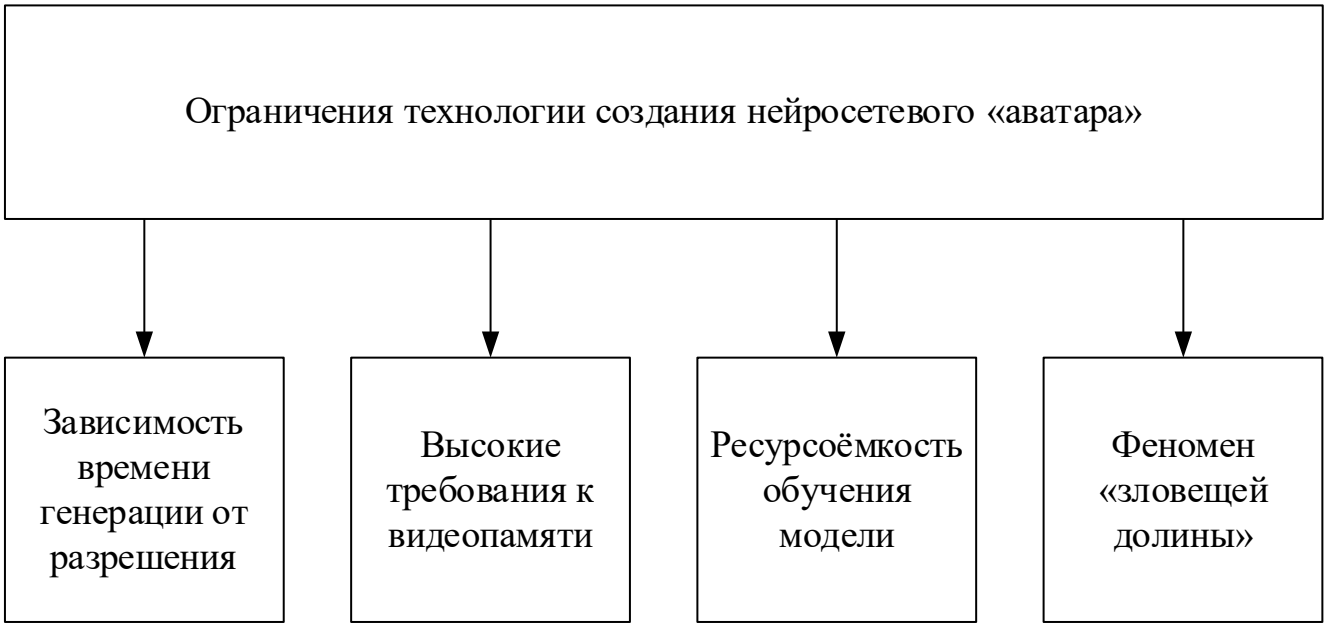


Рисунок 1 – Ограничения технологии создания нейросетевого «аватара»

Все ограничения необходимо учитывать на стадии планирования и проектирования, чтобы в дальнейшем не столкнуться с непреодолимыми трудностями на стадии разработки и эксплуатации.

1.3 Классификация методов генерации нейросетевых «аватаров»

Современные подходы к разработке нейросетевых «аватаров» основаны на принципиально различных подходах обработки данных, что определяет их

уникальные возможности и ограничения. Основное различие между моделями заключается в типе информации, используемой для генерации визуальной составляющей. Каждый формат требует специфических алгоритмов обработки, а выбор между ними напрямую зависит от доступности данных и вычислительных ресурсов. С развитием и масштабированием моделей большое значение приобретает способность модели адаптироваться к зашумленным данных, что повышает стабильность получаемого видео. При наличии небольшого дефекта в предоставляемых данных возможно появление артефактов анимации, из-за чего потребуется заново генерировать фрагмент. Визуальная составляющая «аватара» может формироваться на основе [22]:

- аудиозаписи;
- видеозаписи;
- текстовых описаний;
- 3D-моделей;
- параметрических моделей.

Сложность сбора данных напрямую влияет на выбор архитектуры модели. Например, датасеты с изображениями цифр или лиц легко доступны благодаря их распространенности в открытых источниках, что упрощает проведение экспериментов с базовыми моделями. Обучение моделей требующих спецефичные видео данные, возникает необходимость в самостоятельном сборе и разметки информации. Этот процесс увеличивает временные затраты и создаёт технические сложности обеспечения качества записей и обработки мультимодальных сигналов. Модели нейросетевых «аватаров» основанные на параметрических моделях предлагают компромиссное решение. В отличии от систем, опирающихся на прямую обработку видеопотоков, они используют компактные математические представления, что снижает требования к объему данных и вычислительным ресурсам. Однако сбор таких данных дополнительно усложняется. Необходимо не только провести сбор видеоматериалов, но разработать специальные алгоритмы анализа собранных данных. Также может потребоваться дополнительная обработка видеоматериала, включающая в себя извлечение ключевых признаков, нормализация данных и устранение шумов.

Тип данных, используемых для обучения, напрямую влияет на скорость разработки и эффективность модели. Так, анимация статичного изображения требует в десятки раз меньше времени благодаря минимальному набору изменяемых параметров. В отличие от этого, работа с 3D-моделями подразумевает управление сотнями параметров: от мимики лица и поворотов головы до динамики тела и жестикуляции. Это позволяет достичь высокой реалистичности, но значительно усложняет процесс обучения из-за необходимости синхронизировать множество анатомически связанных элементов. Ключевой компромисс заключается в балансе между сложностью модели и доступностью вычислительных ресурсов.

Самый первый метод создания нейросетевого «аватара» основан на анимировании статичного изображения с использованием эталонного видео, содержащего движения головы. В качестве входных данных для генерации визуальной составляющей требуется изображение персонажа и видеозапись, с которой будет скопирована динамика движений. Этот подход позволяет перенести повороты и наклоны головы из исходного видео на статичный портрет, создавая иллюзию анимации. Аудиодорожка в таком методе не используется. Это делает невозможным синхронизацию мимики губ с произносимой речью. Такие системы послужили отправной точкой в развитии технологии нейросетевых «аватаров», демонстрируя потенциальные возможности нейросетей в создании контента. Генерация происходит быстро, так как процесс сводится к трансформации пикселей исходного изображения без учета произносимых слов и синтеза новых элементов. В современных условиях его применение считается нецелесообразным из-за низкого качества результатов.

Основным недостатком данного метода является отсутствие связи между визуальными и аудио данными. Модели такого вида сфокусированы на копировании движений, не учитывая мимику персонажа. Из-за этого создаваемые кадры часто содержат большое количество искажений при поворотах головы. Эти проблемы возникают из-за недостаточного объёма исходной информации. Нейросеть использует только статичное изображение и ограниченный набор опорных положений головы, что не позволяет создать качественное изображение при критических изменениях

положения головы. Применение моделей FOMM-abs [23], FOMM-rel и X2Face [24] основанных на анимировании статичной иллюстрации изображено на рисунке 2.



Рисунок 2 – Использование моделей анимирования изображения

На изображениях наблюдаются заметные искажения, возникающие из-за несоответствия между исходным положением головы персонажа на статичном изображении и динамикой эталонного видео. Нейросетевая модель, пытаясь адаптировать статичный портрет к движениям из видео, выполняет растяжение и деформацию текстур, что приводит к нарушению пропорций лица. Проблема усугубляется отсутствием обратной связи между визуальными и пространственными данными. Нейросеть не имеет информации о глубине сцены и освещении, что приводит к нарушению перспективы и теневых переходов. Тем не менее, этот метод остается важным этапом в разработке технологий анимации, так как демонстрирует базовые принципы работы с межкадровыми преобразованиями. Имеющиеся недостатки были исправлены в моделях, которые требуют для генерации большего количества информации от пользователя.

Более совершенным методом создания «аватара» является подход, основанный на синтезе движений с использованием опорного аудиосигнала. В этом методе визуальная составляющая генерируется на основе трёх компонентов: статичного изображения персонажа, эталонного видео с динамикой головы и аудиодорожки, содержащей речь. Наличие дополнительной информации позволяет значительно повысить реалистичность анимации по сравнению с ранними методами, где аудио не учитывалось вовсе. С помощью обработки аудиофайла можно получить информацию о том, что слово произносится и о тишине между словами. Ключевой особенность является возможность анализа звукового файла для определения моментов тишины и изменения громкости речи. Паузы между словами синхронизируются с нейтральной мимикой, а изменяющаяся амплитуда указывает на произношение слов. Однако анимация артикуляции слов остаётся ограниченной. Модель реагирует лишь на амплитуду звука, не распознавая семантику речи и отдельные фонемы. Такие модели произвели большой скачек в реалистичности, по сравнению с моделями без учета голоса «аватара». Применение модели CVPR 2021, которая использует опорное аудио для создания «аватара» изображено на рисунке 3.



Рисунок 3 – Использование модели на основе опорного аудио

Генерируемое видео получается реалистичнее по сравнению с моделями, не учитывающими аудио-составляющую, поскольку дополнительно анимируются движения рта, глаз и бровей, синхронизированные с речью. Однако метод не решает проблему искажения лица диктора при значительном различии положения головы на исходном изображении и управляющем видео. Если в исходном кадре голова повёрнута в профиль, а в управляющем видео используется фронтальный ракурс, это может привести к неестественному смещению черт лица. Кроме того, резкие движения головы в управляющем видео вызывают размытия контуров и потерю пропорций.

Этот метод подходит для создания простых анимации. С помощью моделей на основе опорного аудио создаются системы, позволяющие в реальном времени анимировать персонажа. Системы анимации в реальном времени в качестве движения головы использую заранее заготовленные параметры, которые циклически повторяются. Такие системы анализируют звуковой фрагмент и сразу применяют анимацию к персонажу. Данная технология активно используется в индустрии видеоигр для автоматизации движений рта персонажей во время диалогов. Благодаря этому художники избавлены от необходимости вручную прорисовывать каждое движение губ, что значительно сокращает производственные сроки. Эмоциональная составляющая достигается за счет применения перенастроенных шаблонов, которые не только ускоряют процесс, но и усиливают реалистичность персонажей, обеспечивая плавную интеграцию мимики с интонациями речи.

Следующим методом создания нейросетевого «аватара» является подход, основанный на синтезе изображений через вычисление многофакторных параметров. В качестве входных данных система использует три ключевых компонента: статичное изображение человека, аудиодорожку с речью диктора и динамические параметры лица, извлечённые из опорного видео. Этот метод не только наследует преимущества предыдущих решений, но и значительно усиливает визуальную составляющую за счёт глубокого анализа данных.

Основой для генерации служат алгоритмы компьютерного зрения, такие как OpenCV, которые обрабатывают опорное видео, извлекая из него сотни параметров. Эти данные включают в себя различные характеристики рассказчика:

– положение и ориентация головы в пространстве;

– мимика;

– эмоциональные паттерны;

– движение глаз и направление взгляда;

– текстура и детали кожи (морщины покраснения).

Большое внимание уделяется окружению. Изменения в освещении или фоновых объектах динамически влияют на рендеринг «аватара», обеспечивая естественное взаимодействие с виртуальной средой. Так, тени на лице адаптируются под условный источник света, а блики на коже корректируются в зависимости от яркости фона. Всё это позволяет избежать «пластикового» эффекта, характерного для ранних методов анимации. С помощью методов компьютерного зрения можно получить множество параметров окружение, в котором находится рассказчик:

– уровень освещенности;

– угол падения света;

– распределение теней;

– контраст между фоном и объектом.

В новейших моделях для вычисления параметров лица используются гибридные алгоритмы, сочетающие классические методы компьютерного зрения с глубинным обучением. Нейросетевые модели, такие как 3D Morphable Models, воссоздают трёхмерную структуру лица, учитывая даже незаметные глазу изменения: микродрожание губ, асимметрию бровей или динамику морщин при улыбке. Это позволяет преобразовать плоское изображение в адаптивную 3D-модель, которая служит основой для анимации. Точность таких систем достигает субпиксельного уровня, что критически важно для избежание эффекта «зловещей долины». Применение модели AAAI2023 StyleTalk [25], которая использует параметры 3DMM для создания «аватара» изображено на рисунке 4.



Рисунок 4 – Использование параметризованной модели

Генерируемое видео получается реалистичнее за счёт использования многоуровневых параметров, извлечённых из исходных данных. Из управляющего видео точно копируется мимика и выделяется эмоция за счёт анализа опорного материала и создания параметризованной 3D-маски лица (3DMM). Эта маска математически воспроизводит геометрию и динамику черт, что обеспечивает естественную синхронизацию эмоций с движениями лицевых мышц.

Главное преимущество данного метода – его универсальность. За счёт анализа и расчёта параметров достигается реалистичная картинка с минимальным количеством артефактов. Такого эффекта получилось добиться путём насыщения нейросети дополнительной информацией об окружении и самом рассказчике. При этом время генерации возросло незначительно. Современные методы компьютерного зрения, такие как оптимизированные нейросетевые трекеры, обрабатывают десятки кадров в секунду, практически не замедляя генерацию. Однако этот метод не лишен

недостатков. Основная сложность состоит в изначальном обучении модели. Нейронная сеть, обрабатывающая сотни взаимосвязанных параметров, требует обширных размеченных датасетов. Кроме того, данные должны быть очищены от шумов и размытий, в целях предотвращения появления артефактов.

Последний метод генерации нейросетевого «аватара» основан на использовании предварительно подготовленной 3D-модели персонажа. В качестве входных данных система принимает:

– 3D-модель с реализованной скелетной анимацией, где каждая часть тела (конечности, суставы, мимические зоны) имеет управляемые параметры;

– аудиодорожку для синхронизации речи с движением губ и эмоциональной окраской;

– вектор команд, задающий действия персонажа.

Главное преимущество метода заключается в полном контроле над анимацией. В отличие от подходов, ограниченных мимикой, здесь реализуется движение всего тела: перемещение по сцене, взаимодействие с объектами, сложная жестикуляция. Например, «аватар» может одновременно улыбаться, манипулировать предметами и менять позу, сохраняя физическую правдоподобность суставов. Это достигается за счёт предварительной настройки ригов – цифровых связок и шарниров, которые определяют диапазон движений 3D-модели [26].

Данный метод также имеет свои недостатки. Необходимо создание детализированной 3D-модели, которая будет включать в себя все необходимые контрольные точки для алгоритмов генерации. В создаваемых данным методом видео присутствует большое количество дефектов анимации. Появляются неестественные движения, искривления позвоночника при повороте, дрожание конечностей и нарушение пропорций. Это связано с дефицитом обучающих данных – для тренировки нужны тысячи часов видео с метками движений, захваченных в разных ракурсах и условиях. Кроме того, метод крайне ресурсоёмок. Для рендеринга коротких фрагментов на 10 секунд видео требуется до 10 ГБ видеопамяти, что исключает его применение на большинстве персональных компьютеров. Такие требования обусловлены необходимостью

параллельной обработки высокополигональных 3D-моделей, нейросетевых слоёв и физических симуляций.

Таким образом, были рассмотрены ключевые методы генерации нейросетевых «аватаров». Каждый из них обладает как уникальными преимуществами, так и фундаментальными ограничениями. Выбор метода генерации зависит требований к создаваемому видео, а также от аппаратных мощностей имеющихся систем. Представим существующие виды нейросетевых «аватаров» в виде схемы на рисунке 5.

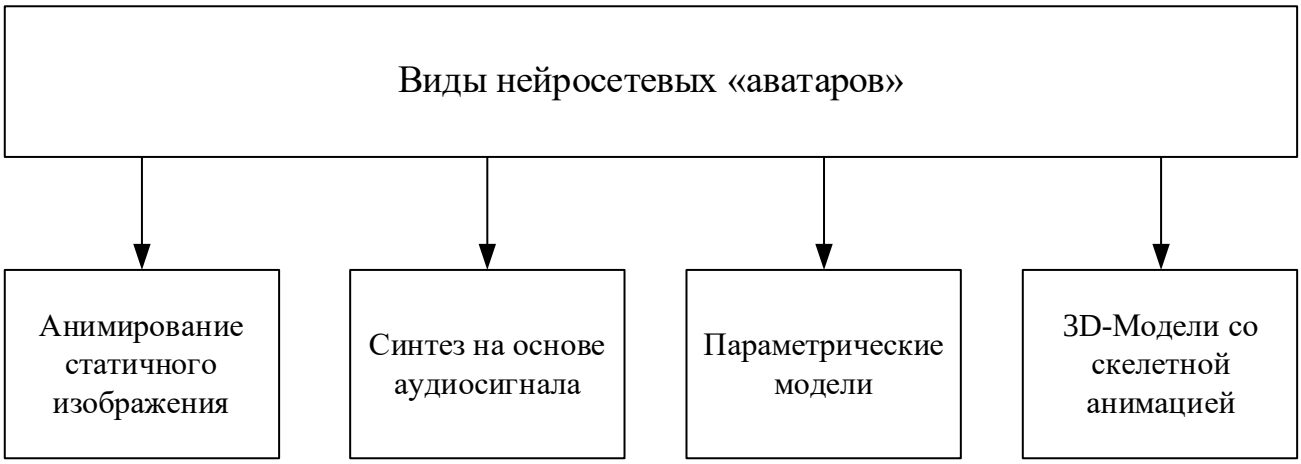


Рисунок 5 – Виды нейросетевых «аватаров»

В качестве основы для разработки нейросетевого курса был выбран метод, основанный на получении множества параметров сцены. Этот подход обеспечивает высокий уровень реалистичности и минимальное количество дефектов анимации, что достигается за счёт анализа сцены опорного видео методами компьютерного зрения. Кроме того, метод позволяет использовать его на персональном компьютере без необходимости подключения к облачным сервисам или использования серверных решений.

1.4 Архитектура взаимодействия компонентов системы генерации нейросетевого «аватара»

1.4.1 Требования к системе генерации нейросетевого «аватара»

Развитие искусственного интеллекта приводит к тому, что нейронные сети перестают быть узкоспециализированным инструментом. На производствах они автоматически выявляют дефекты продукции, повышая качество и снижая затраты на брак [27-28]. В маркетинге нейросети анализируют поведение потребителей, создавая таргетированную рекламу. Это позволяет повысить продажи и предложить человеку тот товар, который ему может понравиться [29]. Современные нейронные сети используются в сфере дизайна. Они способны создавать и дополнять изображения используя только текстовый запрос [30]. Системы умных голосовых помощников используются повсеместно в смартфонах. Эта впечатляющая функциональность результат длительного обучения сложных моделей на гигантских массивах обучающих данных. Их способность находить закономерности в этих данных открывает новые возможности для автоматизации и творчества [31].

Процесс обучения нейросетевой модели является самым ресурсоемким и длительным этапом ее создания. Он требует масштабной подготовительной работы. Требуется поиск, создание и тщательная категоризация миллионов обучающих данных. В истории технологий встречаются креативные решения, совмещающие одновременно несколько задач. Например, система reCAPTCHA, изначально разработанная для проверки «не является ли пользователь человеком», одновременно помогала в цифровизации книг, распознавая фрагменты текста с плохих сканов и классифицировала объекты на изображениях через пользовательские клики [32]. Процесс обучения напрямую зависит от качества данных. Зашумленные данные неизбежно приводят к низкому качеству итоговой модели, который проявляется в артефактах и шуме на выходе. При этом нельзя использовать сгенерированные другими нейронными сетями данные, поскольку они усреднены и лишены уникальных деталей. В результате модель, обученная на таких данных, теряет способность к воспроизведению нюансов. Она начинает выдавать как результат усредненные, наиболее распространенные

варианты. Явление деградации разнообразия, известное как «режим коллапса», особенно опасно для творческих задач, где важна уникальность. Поиск и подготовка достаточно большого объема чистых, разнообразных данных остается одной из главных инженерных сложностей в разработке современных нейросетей. Именно поэтому разрабатываемая система нейросетевого «аватара» построена на использовании заранее обученных моделей генерации. Их применение требует значительно меньших вычислительных ресурсов и занимает меньше времени.

Система нейросетевого «аватара» представляет собой автономную экосистему, которая позволяет использовать собственные данные пользователя для создания и генерации компонентов персонажа. Этот подход устраняет зависимость от внешних источников данных. Минимальное участие человека требуется только на этапе подготовки исходных элементов. После этого специализированные модули автоматически преобразуют данные в формат, совместимый с моделью генерации. Такая архитектура позволяет адаптировать систему под индивидуальные задачи. Благодаря модульности и предобученным моделям, процесс генерации остаётся быстрым и энергоэффективным, что делает технологию доступной даже на устройствах со средней производительностью.

1.4.2 Компоненты системы нейросетевого «аватара»

Для создания аватара требуется 5 независимых компонент. Основным компонентом для формирования уникального «аватара» является изображение персонажа в формате 256x256 пикселей. Выбор такого разрешения обусловлен оптимизацией под среднебюджетные устройства. Увеличение размера изображения приводит к кратному росту времени генерации и нагрузки на видеопамять. Использование такого формата позволяет добиться хорошего качества изображения и быструю генерацию персонализированного «аватара».

Для реализации голосового компонента «аватара» был разработан модуль синтеза речи. В основе решения лежит адаптированный инструментарий Coqui TTS [33]. Это открытый проект по преобразованию текстовых данных в аудиформат. Модуль

способен создавать высококачественную выразительную речь с естественной интонацией. Ключевой особенностью системы является технология клонирования голоса [34]. Для создания персонализированного голосового фрагмента требуется предоставить образец речи длительностью от 10 секунд. Обработанный системой, этот образец позволяет синтезировать речь, максимально приближенную по тембру и характеру к голосу человека в предоставленной записи [35].

Реализация естественной речевой анимации аватара требует не только аудиодорожки, но и точных данных о временной последовательности произносимых фонем. Этот компонент служит ключевым элементом для синхронизации движения губ с аудиозаписью и обеспечения визуальной согласованности. Для формирования списка фонем используется предобученная модель распознавания фонем, которая позволяет определить фонему и её продолжительность. Особое внимание уделяется точности временных меток. Небольшие отклонения на длинных видео более 10 минут приводят к накоплению ошибок и полной рассинхронизации аудио и видео составляющей. Для минимизации таких эффектов внедрена система адаптивной коррекции, которая динамически перераспределяет временные сдвиги в течение всего видео, сохраняя стабильную синхронизацию даже при сложных сценариях произношения. Чтобы минимизировать эффект накопления ошибок в длительных сегментах применяется метод фрагментированной обработки. Он осуществляет принудительную синхронизацию аудио и видео компонентов каждую минуту. Чтобы минимизировать эффект рассинхронизации используется метод фрагментированной обработки, который позволяет каждую минуту синхронизировать речь с видео составляющей. Архитектура обработки небольшими сегментами гарантирует, что локальные погрешности не успевают суммироваться до критического уровня. Это обеспечивает стабильно высокое качество синхронизации на протяжении всего видео независимо от его продолжительности.

Для создания «аватара» также требуются специальные коэффициенты, которые получаются путем анализа опорного видео. Видео анализируется с помощью методов компьютерного зрения на базе алгоритмов OpenCV. Алгоритмы этой библиотеки используются для обнаружения человека в кадре, преобразование видео в

последовательность кадров изображения и взаимодействия с изображениями [36-37]. На следующем этапе применяются алгоритмы обработки нейросетевого инструментария Deep3DFaceRecon. Методы обработки этой библиотеки позволяют выделить ключевые точки лица с опорного видео и с считывать множество параметров с лица человека и окружающей его сцены. Эти параметры формируют основу для создания параметрической маски, которая служит базой при генерации аватара. Для повышения детализации дополнительно анализируются микродвижения лица, что позволяет передать эмоциональную выразительность персонажа.

1.4.3 Взаимодействие компонентов системы нейросетевого «аватара»

Созданные модули формируют единую экосистему, позволяющую генерировать «аватара» на основе пользовательских данных. Возможность генерации видео произвольной продолжительности является основным преимущество над аналогами. Для обработки используется подход, при котором данные разделяются на фрагменты по 1 минуте, а затем объединяются в целостное видео. Алгоритмы разделения адаптируются к содержанию, что обеспечивает плавные переходы между фрагментами. Такая архитектура минимизирует нагрузку на видеопамять, что позволяет использовать устройства со средними характеристиками. Для удобства повторного использования была добавлена возможность создания профилей «аватаров». Пользователь, один раз создав персонажа может создать профиль и использовать его повторно. Для формирования профиля необходимо изображение персонажа, сгенерированный компонент с данными о положении головы в формате. mat и образец голоса. Для генерации видео применяются параметрические модели FaceGenerator и Styletalk. Они используют матрицы параметров для создания реалистичных изображений. Реалистичность усиливается за счёт микродвижений головы, извлечённых из опорного видео. Система анализирует поведение реального рассказчика и переносит естественные наклоны и повороты на «аватара». Все требуемые компоненты для генерации аватара представлены на рисунке 6.

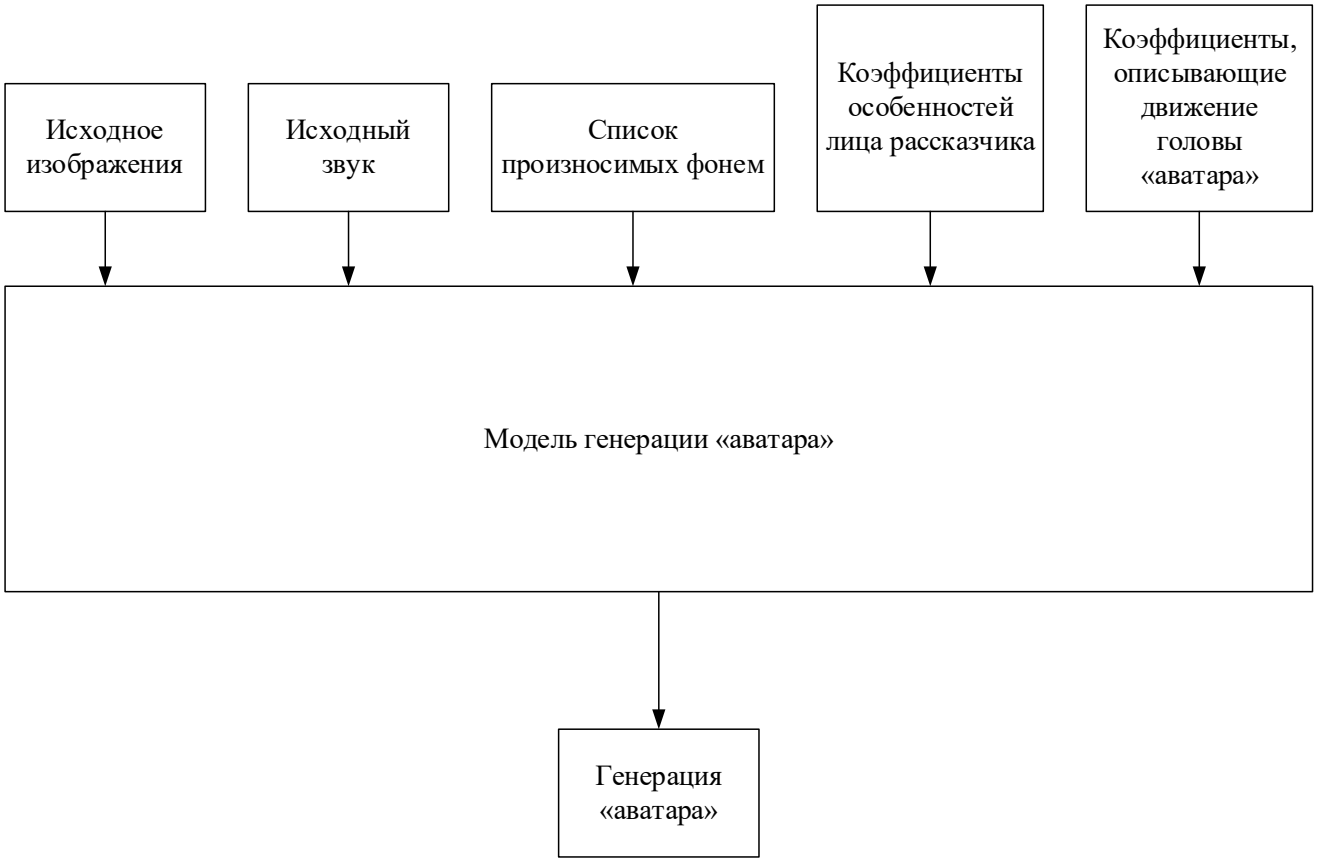


Рисунок 6 – Требуемые компоненты для генерации

Разработанная система генерации нейросетевого «аватара» представляет собой комплексное решение, объединяющее передовые технологии машинного обучения и компьютерного зрения. На основе заранее обученных моделей FaceGenerator, Styletalk, Deep3DFaceRecon и адаптированного инструментария Coqui TTS и OpenCV система обеспечивает высокую реалистичность создаваемого персонажа. Модульная архитектура позволяет гибко настраивать систему под конкретные задачи. Каждый компонент функционирует независимо, что упрощает модификацию и расширение функционала без глубоких изменений в общей кодовой базе.

1.5 Архитектуры взаимодействия компонентов образовательной цифровой площадки

1.5.1 Требования к компонентам веб-приложения

Перед началом разработки любого веб-приложения необходимо тщательно спланировать его архитектуру. На этом этапе требуется учесть множество взаимосвязанных факторов: функциональные требования к системе, производительность, безопасность, масштабируемость и удобство разработки. Обоснованный выбор используемых технологий для каждого компонента системы напрямую влияет на эффективность реализации, стабильность работы и долгосрочную поддержку проекта. Традиционная архитектура веб-приложения состоит из трех основных компонентов, взаимодействующих между собой [38]:

– клиентская часть. Создаёт удобный пользовательский интерфейс, через который пользователь взаимодействует с приложением. Она может отправлять запросы на серверную часть для получения данных или выполнения операций, а также отображать результаты обработки;

– серверная часть. Обрабатывает запросы пользователя от клиентской части, взаимодействует с базой данных для извлечения и изменения информации. На этом уровне происходят вычисления, проверка данных и управление логикой приложения. Является важным промежуточным компонентом между пользователем и базой данных;

– база данных. Структурированно хранит множество информации посредством взаимосвязанных между собой таблиц. Работа с ней осуществляется через SQL-запросы, сформированные серверной частью. База данных не взаимодействует напрямую с клиентом, обеспечивая безопасность и централизованное управление данными.

При проектировании архитектуры веб-приложения важно учитывать не только технические инструменты, но и функциональные требования. Для разработки клиентской части основными критериями являются:

– компонентный подход. Позволяет разделить интерфейс на независимые компоненты, что обеспечивает повторное использование кода. Это упрощает масштабирование и поддержку для сложных интерфейсов, где одни и те же элементы используются в разных частях приложения;

– возможность создания одностраничного приложения. Такой подход обеспечивает высокую производительность, отзывчивость и плавность взаимодействия, так как страница не перезагружается при переходе между разделами;

– простота настройки. Минимальные требования к предварительной подготовке ускоряют старт разработки и позволяют сосредоточиться на логике приложения, а не на сложных настройках инфраструктуры. Сложные процессы конфигурирования также увеличивают вероятность ошибок при настройке, что может привести к появлению уязвимостей в безопасности системы.

1.5.2 Компоненты клиентской части веб-приложения

При проектировании веб-приложения необходимо правильно выбрать инструменты для разработки. Для разработки клиентской части приложения существует большое количество альтернатив. Рассмотрим три основные альтернативы для реализации клиентской части: традиционный инструмент разработки веб-приложений HTML/JavaScript/CSS, а также современные фреймворки React и Vue.

HTML является самой базовым системой для создания веб-приложений. Этот язык разметки основан на системе тегов, комбинируя которые разработчик создаёт базовое содержимое веб-страницы. Процесс работы с HTML интуитивно понятен, что упрощает начальное освоение. Сам по себе язык не позволяет создавать динамичные и интерактивные страницы. Для этого используется интеграция JavaScript, который позволяет реализовывать дополнительную логику. С помощью него можно создавать динамически изменяемые элементы интерфейса, выполнять вычисления и осуществлять асинхронные запросы к серверу. Для оформления стили страницы используется CSS. Он даёт возможность управления макетом, изменения цветов, добавления

анимации и типографики. Эта триада технологий составляет основу веб разработки, обладает большой экосистемой и универсальностью [39].

Несмотря на высокую универсальность, подход, основанный на этих технологиях, имеет существенные недостатки для разработки современных приложений. HTML страницы не поддерживают архитектуру одностраничных приложений. В классической модели каждая пользовательская навигация вызывает полную перезагрузку страницы с сервера [40]. Это приводит к заметным задержкам, мерцанию интерфейса и ухудшению пользовательского опыта, так как браузеру приходится заново загружать и обрабатывать HTML, CSS и JS. Одностраничные приложения динамически обновляют наполнение загруженной страницы, обеспечивая мгновенные переходы между страницами. Это становится стандартом для современных веб-приложений. Вторым ключевым недостатком является невозможность переиспользования кода. HTML не поддерживает возможность внедрения готовых шаблонов кода на страницу, из-за чего количество повторяющегося кода значительно вырастает. Эти ограничения делают связку HTML/JavaScript/CSS универсальными, но неэффективными для интерактивных быстрых приложений.

React предлагает принципиально иной подход, основанный на компонентной архитектуре. Весь интерфейс пользователя разбивается на независимые, повторно используемые блоки, которые можно комбинировать для создания сложного интерфейса. Это решает проблему дублирования кода. Один раз созданный компонент можно использовать в разных частях приложения. React также поддерживает одностраничную архитектуру, позволяя обновлять содержимое страницы динамически без перезагрузки. Однако разработка на React сопряжена и с определенными сложностями. Значимой трудностью является необходимость настройки окружения и выбора дополнительных инструментов. Сам React обеспечивает только отображение элементов, оставляя вопросы маршрутизации, взаимодействие с сервером и другие аспекты на усмотрение разработчика. Это требует тщательного подбора совместимых библиотек, что добавляет этап сложной конфигурации. Также React требует знание специфических концепций, которые необходимо применять для создания веб-приложений. Для создания базового, правильно настроенного проекта требуется большое

количество опыта. Развитая экосистема React и поддержка сообщества делает его привлекательным выбором для крупных и динамичных проектов с большим количеством разработчиков [41]. Для разрабатываемого приложения его использование является избыточным.

Vue, в свою очередь, представляет собой фреймворк с более низким порогом входа. Он сочетает гибкость компонентного подхода с интуитивно понятной структурой. Vue позволяет легко создавать одностраничные приложения, благодаря встроенной поддержке маршрутизации и управления состоянием. В отличие от React, Vue предоставляет более строгую структуру проекта. Это упрощает стандартные задачи, но при этом сохраняет возможность кастомизации. В Vue каждая страница описывается в одном файле, который объединяет разметку, стиль и логику. Для больших проектов Vue является менее вариативным по сравнению с React, так как он ориентирован на стандартные сценарии использования [42].

Выбор между этими технологиями зависит от масштаба проекта. При необходимости создать простой статичный сайт с минимальной интерактивностью, HTML/CSS/JavaScript является лучшим выбором. Для приложений, где важна скорость работы выбор идет между React и Vue. React подходит для больших проектов с многочисленной командой разработчиков. Vue идеален для проектов средней сложности, где важна простота внедрения и быстрая разработка. В качестве инструмента для клиентской части приложения будет использоваться Vue.

1.5.3 Компоненты серверной части веб-приложения

Для разработки серверной части необходимо выбрать инструментарий, который позволит обеспечить высокую скорость взаимодействия между пользовательским интерфейсом и базой данных. Среди популярных вариантов выделяются PHP, Flask и Node.js. Каждый инструмент имеет свои особенности и ограничения.

PHP является одним из самых популярных языков программирования для веб-разработки. Он позволяет создавать надежные серверные обработчики. Однако его архитектура имеет существенные недостатки для современных приложений. Он не

поддерживает асинхронное взаимодействие, что делает его неэффективным для применения в одностраничных приложениях, где требуется быстрая обработка множества параллельных запросов. Также PHP использует блокирующую модель ввода-вывода, что приводит к узким местам в производительности. [43] Имеются проблемы с обеспечением безопасности, что требует тщательной настройки и применения дополнительных мер защиты.

Flask предлагает большую вариативность и поддержку асинхронного ввода-вывода, что позволяет его использовать в одностраничных приложениях. Базовое наполнение этого инструмента очень ограничено. Из-за этого большинство функционала реализуется через внешние библиотеки и расширения [44]. Это значительно увеличивает риск появления уязвимостей и усложняет поддержку кода. Нестабильность интерфейсов и возникающие конфликты между расширениями создают дополнительные риски для безопасности и стабильности проекта.

Node.js выделяется как универсальное решение, оптимизированный под современные требования. Он разработан с учетом максимальной оптимизации обработки множества асинхронных запросов одновременно. Это обеспечивает высокую скорость работы как с пользовательским интерфейсом, так и с базой данных. Node.js отличается простотой настройки и обширными встроенными возможностями, что позволяет минимизировать количество пользовательских библиотек [45]. Высокая скорость и гибкость позволяет легко настроить взаимодействие с React или Vue фреймворками. Эти преимущества, а также устойчивость к проблемам, характерным для PHP и Flask, делают Node.js предпочтительным выбором для серверной части приложения.

Также необходимо выбрать базу данных для взаимодействия с серверной частью приложения. Существует разные типы баз данных: реляционные, графовые, сетевые, объектные и другие. Для структурированного хранения данных образовательной платформу лучше всего подходят реляционные базы данных. Современные реляционные базы данных имеют схожий уровень производительности. Это связано с тем, что все методы оптимизации, которые дают существенный прирост производительности за многие годы уже известны. Поэтому популярные СУБД предлагают

сопоставимый уровень производительности. В качестве системы управления базами данных для проекта была выбрана MariaDB [46]. Этот выбор обусловлен ее открытостью исходного кода и высокой производительностью, которая не уступает альтернативам в этой категории [47].

После анализа доступных технологий с учетом требований к безопасности, функциональности и производительности были выбраны оптимальные инструменты для реализации проекта. Vue был выбран для клиентской части благодаря стандартизированному инструментарию и встроенному функционалу. Встроенный Vue Router для навигации и Vuex для управления состоянием снижает зависимость от сторонних библиотек, что повышает безопасность проекта и с упрощает настройку. Node.js стал выбором для серверной части из-за поддержки асинхронности, которая критична для работы с одностраничными приложениями на Vue. Его популярность, гибкая экосистема и единообразие JavaScript на клиенте и сервере обеспечивают высокую производительность. MariaDB выбрана как открытая и долго развивающаяся СУБД, сочетающая высокую скорость работы с реляционной моделью.

1.5.4 Взаимодействие компонентов и обеспечение безопасности веб-приложения

Так как компоненты (клиент, сервер, БД) работают независимо друг от друга, необходимо организовать взаимодействие между ними. Связь между серверной часть и базой данных реализуется через выполнение стандартных SQL-запросов. Они позволяют добавлять, изменять и извлекать данные. Этот механизм взаимодействия является проверенным, надежным и не представляет концептуальных сложностей. Для связи между клиентским приложением и сервером создаётся API интерфейс для взаимодействия. Этот интерфейс представляет собой набор строго определенных эндпоинтов (готовых вопросов), на которые сервер отвечает структурированными данными в формате JSON [48-49]. Все запросы к API передаются исключительно по защищенному протоколу HTTPS, обеспечивающему шифрование и безопасную передачу

данных между клиентом и сервером. Схема взаимодействия компонентов веб-приложения представлена на рисунке 7.

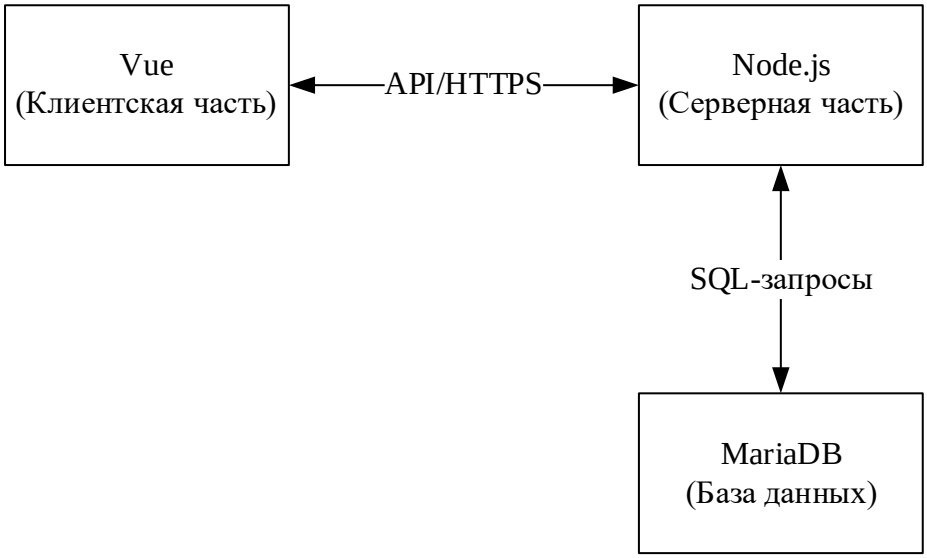


Рисунок 7 – Взаимодействие компонентов веб-приложения

Структура с тремя независимыми компонентами, которая взаимодействует между собой является традиционной для разработки веб-приложений. Такой подход позволяет интегрировать сторонние инструменты, добавлять промежуточное ПО для повышения отказоустойчивости, ускорения обработки данных и усиления безопасности. В рамках текущего проекта расширение архитектуры за счёт дополнительных модулей окажется избыточным. Выбранная технологическая связка уже полностью покрывает потребности по функциональности, производительности и надёжности.

После выбора инструментария следует этап проектирования архитектуры контента веб-приложения. Необходимо определить структуры страниц и функциональных разделов, из которых будет состоять приложение.

Веб-приложение должно реализовывать комплексный набор функций. Неотъемлемым компонентом является система аутентификации и управления пользователем. Должна присутствовать возможность регистрации и создания личного профиля. Профили разделяются на 2 типа: студент и преподаватель. В профиле студента указывается номер учебной группы. Студенты получают доступ к библиотеке видеолекций, а также к прохождению связанных тестов и лабораторных работ. Преподаватели обладают правами на загрузку новых видеолекций, формирование тестовых и

лабораторных заданий. Обязательным элементом системы является возможность самостоятельного восстановления пароля, что повышает удобство пользователей и снижения нагрузку на техническую поддержку, исключая необходимость ручного восстановления.

Библиотека видеолекций представляет собой каталог с возможностью онлайн-просмотра. Каждое видео сопровождает названием, кратким описанием и превью-изображением, что позволяет быстро ознакомиться с содержанием. Материалы структурированы для удобного поиска, а поддержка адаптивного качества видео обеспечивает комфортный просмотр на любых устройствах. После изучения лекции пользователь переходит к тестированию или лабораторной работе, закрепляя тему. Система тестирования основана на динамической генерации вопросов из заранее подготовленного банка. При запуске теста система случайным образом выбирает задания, формируя индивидуальный вариант для каждого студента. Интерактивные вопросы имеют несколько вариантов ответов. Тестирование ограничивается по времени, что помогает предотвратить списывание. Результаты проверяются автоматически, с выводом оценки и анализа ошибок. Лабораторные работы реализованы с элементами геймификации. Студент взаимодействует с интерактивными блоками, собирая решения в правильной последовательности. Задания имеют прогрессию сложности. На начальных вопросах требуется расположить блоки в правильном порядке. Для решения последних заданий, необходимо выбрать правильные из общего банка и выстроить их в нужном порядке.

Архитектура платформы предусматривает гибкость и масштабируемость. Реализован ролевой доступ, автоматизация задач и интерактивный интерфейс делают обучение более эффективным. Схема архитектуры образовательной платформы представлена на рисунке 8.

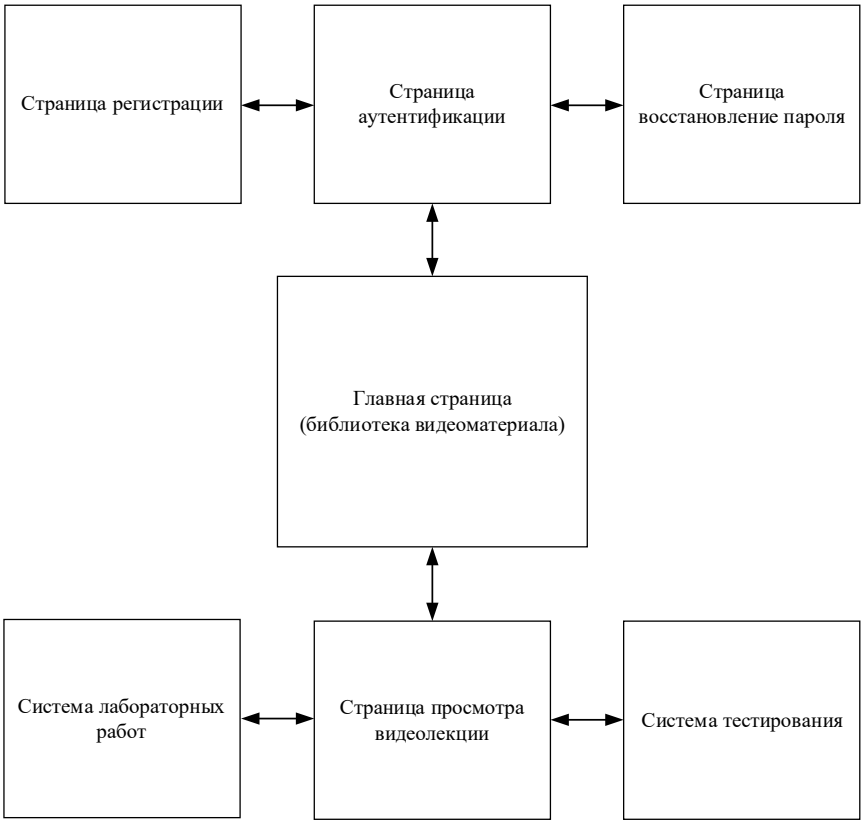


Рисунок 8 – Архитектура образовательной платформы

Разработанная архитектура веб-приложения для образовательной платформы основана на традиционной трёхуровневой структуре, что обеспечивает чёткое разделение функциональных зон и упрощает масштабирование. В качестве клиентской части выбран фреймворк Vue благодаря его балансу между гибкостью и простотой внедрения. Для серверной части оптимальным решением стал Node.js. Он обеспечивает асинхронную обработку запросов, высокую производительность и единообразие кода на клиенте и сервере за счёт использования JavaScript. В качестве реляционной СУБД выбрана MariaDB, сочетающая открытость, стабильность и производительность. Взаимодействие между компонентами реализовано через REST API [50]. Данные передаются в формате JSON по защищённому протоколу HTTPS, что гарантирует безопасность и гибкость интеграции. База данных связана с сервером через стандартные SQL-запросы, что обеспечивает надёжное хранение и обработку информации. Выбранные технологии обеспечивают высокую производительность, безопасности, простоту поддержки и удобство для пользователей.

2 Алгоритмизация и программная реализации системы генерации нейросетевого «аватара»

2.1 Модуль генерации голоса

2.1.1 Структура модуля

Модуль генерации голоса является комплексной системой, основанной на современных моделях синтеза речи и отвечающей широкому спектру функциональных и технических требований [51]. Он преобразует текстовые материалы лекции в естественную человеческую речь, обеспечивая эмоциональную окраску и интонацию [52]. Создаваемые аудиофайлы сохраняются в формате wav, что обеспечивает совместимость с нейросетевой моделью и исключает необходимость дополнительных преобразований, которые могли бы повлиять на качество звука. Выбор этого формата позволяет избежать потерь, связанных с сжатием, и гарантирует точность воспроизведения [53].

Ключевой особенностью модуля является возможность клонирования голоса на основе аудиозаписи, загружаемой пользователем. На основе эталонного аудио, длительностью до 1 минуты, система анализирует уникальные параметры голоса. Модель выделяет тембр голоса, ритм, особенности интонации и диапазон частот. Это позволяет создавать голос, визуально и эмоционально согласованный с «аватаром», что повышает реалистичность создаваемого «аватара». Скорость речи регулируется автоматически в зависимости от длины предложений и сложности контекста [54]. Многоязычная модель XTTS_v2 поддерживает 17 языков, что позволяет произносить специальные термины, которые написаны на английском языке в тексте, написанном на русском языке, без дополнительных преобразований.

Для обеспечения высокой производительности модуль использует аппаратное ускорение CUDA совместимых GPU. Данная технология позволяет ускорить вычисления в 5-7 раз по сравнению с использованием только центрального процессора. Это особенно важно при работе с объёмными проектами, такими как создание серии лекций, где требуется обработать десятки текстовых фрагментов [55]. Модуль также поддерживает конвейерную обработку, позволяя одновременно озвучивать несколько

текстов без повторного запуска программы. Например, пользователь может загрузить сразу 20 лекций, и система автоматически распределит задачи, сохранив результаты в отдельных файлах с понятной нумерацией. Это экономит время и упрощает подготовку лекционного материала. Гибкость системы проявляется и в возможности работать с голосами. Один текст можно озвучить разными голосами, чтобы выбрать наиболее подходящий вариант. И наоборот, несколько текстов можно обработать единым голосом, сохранив стилистическое единство материала. Функция подбора голосов удобна для создания персонализированных курсов, где важно соответствие «аватару».

2.1.2 Алгоритм генерации голоса

Модуль выполняет последовательную обработку текстовых данных для преобразования их в аудиозаписи с сохранением характеристик эталонного голоса. Процесс состоит из следующих шагов:

– загрузка входных данных. Программа считывает все текстовые файлы в формате .txt из директории input_text. Параллельно загружается аудиофайл с образцом голоса, параметры которого будут использоваться для синтеза речи;

– проверка корректности данных. Текстовые файлы должны содержать данные и иметь кодировку UTF-8, поддерживаемую моделью. Голосовой образец должен иметь длительность не менее 6 секунд;

– инициализация модели синтеза речи. Загружается многоязыковая модель XTTS_V2, которая настраивается на основе эталонного голоса. Модель анализирует тембр, интонацию и другие характеристики, чтобы воспроизвести их в сгенерированной речи;

– генерация аудиозаписей. Для каждого текстового файла из input_text выполняется синтез речи с сохранением стиля эталонного голоса [56];

– сохранение результатов. Сгенерированные аудиофайлы сохраняются в директорию output_audio. Для удобства, каждый файл получает имя, идентичное исходному текстовому документу.

Блок-схема алгоритма генерации голоса представлена на рисунке 9.



Рисунок 9 – Блок-схема алгоритма генерации голоса «аватара»

Таким образом, модуль генерации голоса представляет собой автономную компоненту системы, обеспечивающую надежное преобразование текстовых данных в аудио. При появлении более совершенных моделей синтеза речи их интеграция будет сводиться к замене нейросетевой модели без изменения общей логики работы модуля. Это делает систему масштабируемой и адаптивной к технологическим изменениям [57].

2.2 Модуль определения фонем

2.2.1 Структура модуля

Следующий этап разработки системы – создание модуля определения фонем, который играет ключевую роль в обеспечении точной синхронизации аудио и видео составляющих «аватара». Фонема, как минимальная звуковая единица, несущая смыслоразличительную функцию, становится основой для построения анимации губ и мимики персонажа [58]. Основная задача модуля заключается в анализе заранее сгенерированного голосового файла: извлечении фонем, определении их длительности и преобразовании данных в формат, совместимый с моделью генерации «аватара» [59-60]. Модуль последовательно обрабатывает звуковой файл, выделяя в нем участки произносимых слов и устанавливая временные метки, которые фиксируют моменты начала и окончания их произнесения. Затем слова передаются в модель определения фонем, где для каждого звука уточняются тайминги, учитывающие индивидуальные особенности интонации и скорости речи. На финальном этапе обработанные данные конвертируются в структуру, поддерживаемую моделью «аватара», обеспечивая синхронизацию движений губ с звучащей речью. Точность определения фонем и их временных характеристик напрямую влияет на реалистичность взаимодействия между визуальным образом персонажа и его голосом. Работа модуля организована через четыре последовательно выполняемых алгоритма, каждый из которых решает специфическую задачу подготовки данных для дальнейшей визуализации:

– алгоритм анализа и сегментации аудио;

– алгоритм определения фонем в произносимых словах;

– алгоритм разметки по времени произносимых фонем;

– алгоритм преобразования размеченных фонем в формат, предназначенный на считывание моделью генерации «аватара».

Модуль также адаптирован для работы в конвейерном режиме, что позволяет обрабатывать сразу несколько голосовых файлов без необходимости повторного запуска программы [61]. Эта функция особенно полезна при массовой подготовке лекционных материалов.

2.2.2 Алгоритм анализа и сегментации голоса

Алгоритм анализа и сегментации аудио реализован с использованием библиотеки pydub, которая предоставляет удобный интерфейс для работы с аудиоданными и является ключевым инструментом в этапах предварительной обработки звуковых файлов [62-63]. Эта библиотека поддерживает широкий спектр функций, таких как считывание аудиофайлов из форматов WAV, MP3, FLAC и других, преобразование различных параметров аудио (частоты дискретизации, битрейт), нарезку и объединение аудиосегментов, изменение громкости, измерение длительности, а также применение фильтров. Однако её основная ценность для данного модуля заключается в встроенных методах detect_silence и detect_nonsilence, которые позволяют точно определять границы речевых фрагментов и пауз [64].

Работа алгоритма основывается на анализе аудиофайла, где с помощью функций detect_silence и detect_nonsilence выделяются участки речи. Для работы этих функций необходимо задать два аргумента:

– silence_threshold – порог громкости в децибелах, ниже которого участок считается тишиной. По умолчанию используется значение -50 dB, которое часто используется для фильтрации фонового шума;

– min_silence_len – минимальная длительность тишины в миллисекундах, которая необходима для разделения сегментов речи. Значение по умолчанию 90 мс, что соответствует паузам между словами.

Эти параметры настраиваются эмпирически, чтобы обеспечить баланс между чувствительностью к коротким словам и устойчивостью к ложным срабатываниям, вызванным шумами или переходными процессами. Для этого выделяемые фрагменты сохраняются, чтобы пользователь мог удостовериться в качестве и точности выделяемых слов, и при необходимости изменить настройки. После первоначальной сегментации система проходит этап постобработки, направленный на устранение ошибок и улучшение качества выделенных фрагментов.

Первым шагом постобработки является фильтрация ложноположительных срабатываний. Это короткие фрагменты, ошибочно определенные как слова из-за присутствующих в записи шумов. Для этого вводится ограничение минимальной длительности сегмента в 120 миллисекунд, который может быть классифицирован как слово. Это значение выбрано на основе анализа длительности фонем в русском языке, где даже самые короткие звуки редко бывают короче указанного порога. Сегменты, не соответствующие этому критерию, объединяются с соседними участками тишины.

Вторым этапом уточнения данных выполняется объединение смежных фрагментов тишины. Это позволяет избежать разрыва между паузами, которые на самом деле являются единым интервалом, но были разделены из-за небольших колебаний громкости. В результате система формирует последовательность, состоящую из строго чередующихся блоков слов и тишины. Такая структура позволяет упростить дальнейшую обработку, поскольку позволяет исключить неоднозначность в интерпретации временных меток.

Еще одной особенностью данного алгоритма является гибкое разделение аудиозаписи на фрагменты длительностью около 1 минуты, что обусловлено техническими ограничениями вычислительных ресурсов. Основная цель – обеспечить совместимость с оборудованием, обладающим 6 ГБ видеопамяти, которая требуется для рендеринга «аватара». Одна минута аудио с сопутствующей визуализацией занимает около 5.3–5.8 ГБ памяти, что соответствует возможностям среднебюджетных GPU.

Алгоритм избегает жёсткого разделения по фиксированным временным интервалам (например, ровно 60 секунд). Вместо этого он ищет моменты тишины длительностью не менее 500 мс, которые естественным образом разделяют смысловые блоки речи. Это позволяет сохранить целостность предложений и избежать разрывов в середине слов. Главным достоинством подхода является возможность генерации анимации «аватара» даже на системах с ограниченными вычислительными ресурсами. За счёт разделения аудиозаписи на фрагменты длительностью около 1 минуты, алгоритм учитывает требования к видеопамяти при дальнейших вычислениях. Алгоритм оптимизирует использование аппаратных ресурсов, максимально загружая видеокарту без превышения лимитов памяти. Обработка часовой лекции разбивается на 60 независимых задач, каждая из которых выполняется последовательно, что исключает перегрев и обеспечивает стабильную работу системы. Подход адаптивного разделения позволяет адаптировать процесс генерации. При изменении конфигурации оборудования параметры разделения просто настраиваются. Если устройство располагает 12 ГБ видеопамяти, длительность фрагментов можно увеличить до 2 минут, сохранив принцип разделения по паузам. Это позволяет масштабировать систему.

Процесс анализа аудиозаписи алгоритмом состоит из следующих шагов:

– очистка выходных директорий fragments, output, json_fragments для хранения промежуточных и конечных результатов. Перед началом обработки алгоритм удаляет все файлы из указанных директорий, чтобы избежать конфликтов с предыдущими сессиями. Это гарантирует, что новые данные будут записаны корректно, а также предотвращает накопление устаревших файлов, которые могут вызвать ошибки;

– проверка наличия входной директории Sound, содержащей исходные аудиофайлы. Алгоритм проверяет, существует ли папка Sound, куда пользователь должен поместить аудиофайлы для обработки. Если директория отсутствует, процесс завершается с ошибкой «Директория Sound не существует», чтобы избежать некорректной работы модуля;

– поиск аудиофайлов форматов .wav и .mp3 в директории Sound. После подтверждения наличия директории программа сканирует её содержимое, выделяя

файлы с поддерживаемыми расширениями. Если подходящие аудиозаписи не найдены, выводится сообщение «Аудиофайлы отсутствуют в папке Sound», и алгоритм останавливается. Это предотвращает попытку обработки без данных;

– загрузка аудиофайла через библиотеку pydub для дальнейшей обработки. Каждый найденный аудиофайл поочередно загружается с помощью библиотеки pydub, которая обеспечивает универсальный интерфейс для работы с различными форматами. В процессе загрузки проверяется целостность файла, и при обнаружении повреждений выводится соответствующее уведомление;

– определение интервалов тишины и слов с помощью функций detect_silence и detect_nonsilence с заданными параметрами silence_threshold и min_silence_len. Для разделения аудио на речевые сегменты и паузы используются встроенные методы pydub. Порог silence_threshold в -50 dB определяет уровень громкости, ниже которого участок считается тишиной, а min_silence_len длиной в 100 мс задаёт минимальную длительность паузы для разделения сегментов. Эти параметры настраиваются для баланса между чувствительностью к коротким словам и устойчивостью к шумам;

– фильтрация ложноположительных срабатываний. Короткие речевые фрагменты менее 120 мс объединяются с соседними паузами. После первоначальной сегментации система анализирует длину выделенных речевых участков. Сегменты, короче 120 мс, считаются ложными срабатываниями (например, шумами или артефактами) и объединяются с ближайшими паузами. Это улучшает точность сегментации и снижает количество ошибок в дальнейшей обработке;

– объединение смежных интервалов тишины для упрощения структуры временных меток. Смежные паузы, разделенные небольшими колебаниями громкости, объединяются в один интервал. Это устраняет разрывы между паузами, создавая чистую структуру чередующихся блоков слов и тишины, что упрощает работу с временными метками на следующих этапах;

– формирование массива fragmentArray с началом и концом каждого речевого сегмента. Все обработанные данные собираются в массив fragmentArray, где каждый

элемент содержит временные метки начала и окончания речевого фрагмента. Эта структура используется для последующего анализа аудио;

– проверка совпадения общей длительности сегментов с длительностью исходного аудиофайла. Алгоритм сравнивает суммарную продолжительность всех сегментов с длительностью исходного файла. Если данные не совпадают, это может указывать на ошибки в сегментации, и выводится сообщение «Продолжительность полученных фрагментов не совпадает с исходным файлом»;

– разделение аудиофайла на фрагменты методом гибкого разделения согласно данным из fragmentArray с сохранением в директорию fragments. Исходный аудиофайл разбивается на отдельные части, соответствующие выделенным речевым сегментам. Каждый фрагмент сохраняется в папке fragments с уникальным именем, связанным с исходным файлом и номером сегмента;

– повторение процесса для всех аудиофайлов в директории Sound. Алгоритм повторяет все шаги для каждого файла из директории Sound, обеспечивая массовую обработку данных. Это особенно полезно при работе с большими объемами данных, при обработке серии лекций.

Блок-схема алгоритма анализа и сегментации аудио представлена на рисунках 10-12.

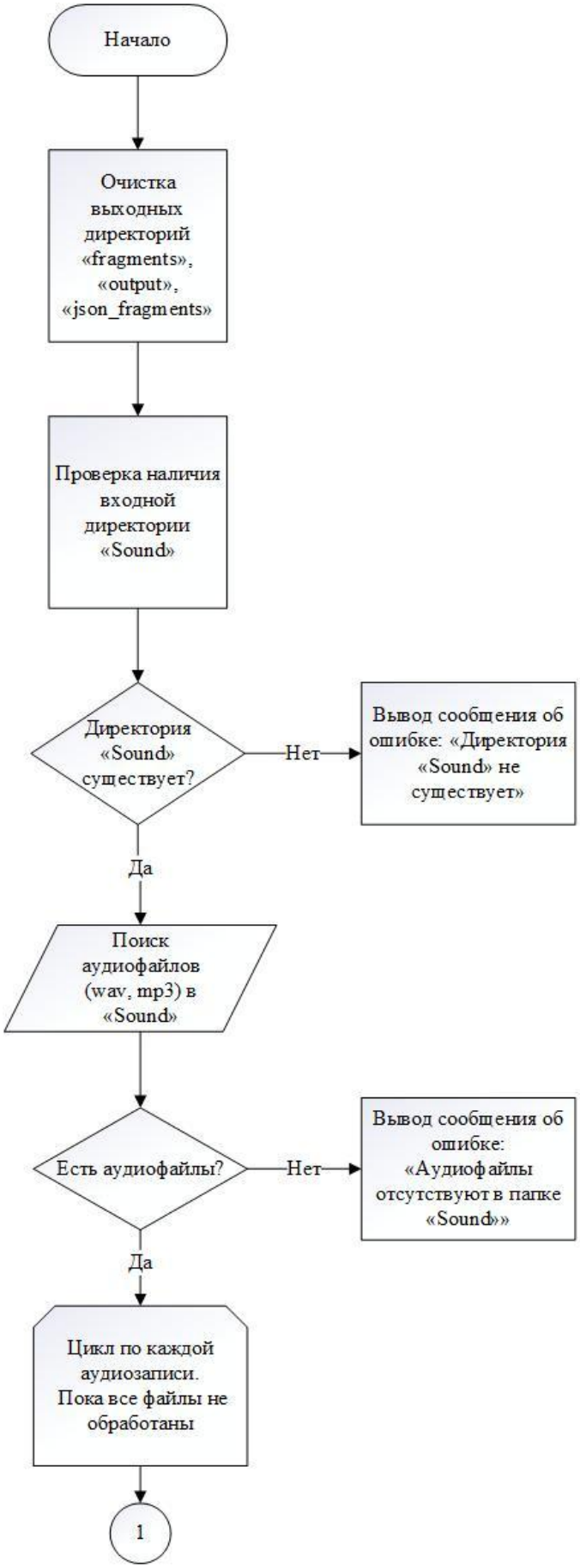


Рисунок 10 – Блок-схема алгоритма анализа и сегментации аудио

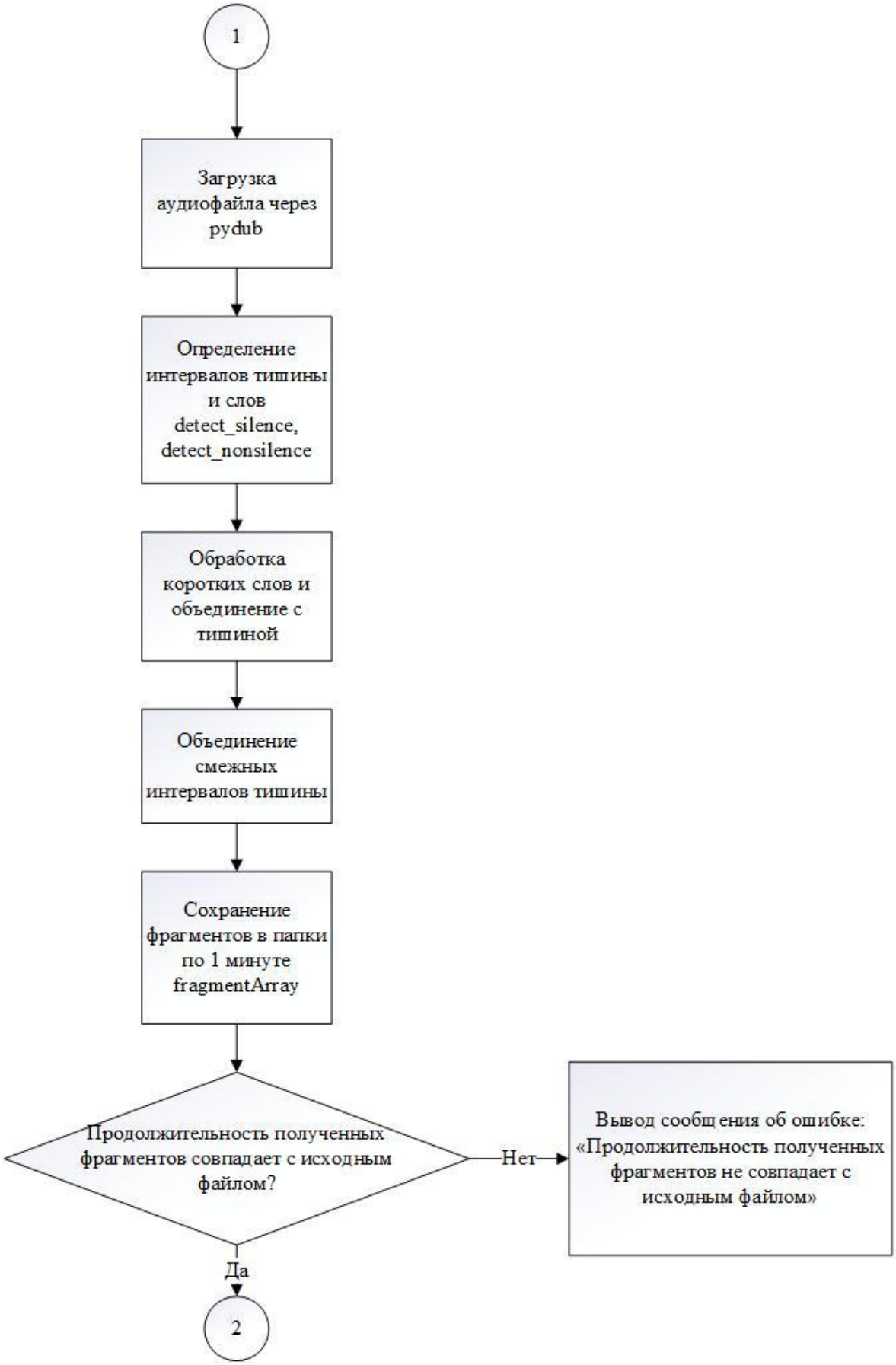


Рисунок 11 – Блок-схема алгоритма анализа и сегментации аудио



Рисунок 12 – Блок-схема алгоритма анализа и сегментации аудио

Таким образом, алгоритм представляет собой комплексное решение для анализа аудиозаписи голоса «аватара», обеспечивающее точное выделение

произносимых слов и формирование структурированных временных меток. Это позволяет создать надежную основу для синхронизации аудиодорожки с визуальными компонентами, что особенно важно для корректной работы «аватара».

2.2.3 Алгоритм определения фонем

Следующим этапом обработки аудиофайла становится определение фонем в выделенных речевых фрагментах, реализованное с помощью отдельного алгоритма на основе библиотеки Allosaurus [65-66]. Эта библиотека представляет собой предобученную нейросетевую модель, разработанную для межъязыкового анализа речи. Она позволяет выделять фонемы в аудиозаписях, обеспечивая высокую точность даже для сложных и длинных слов [67]. Библиотека основана на глубоких нейросетях, которые анализируют спектрограммы аудиосигналов и сопоставляют их с заранее известными паттернами фонем. Однако результаты работы модели представлены в собственном формате, который требует дополнительного преобразования для совместимости с моделью «аватара». Для анализа аудио применяется функция recognize, принимающая следующие аргументы:

– audio_file – путь к обрабатываемому аудиофайлу;

– emission – параметр, регулирующий чувствительность модели к фонемам. Значение по умолчанию 1 подходит для большинства случаев, обеспечивая баланс между точностью и устойчивостью к шумам. Увеличение параметра может быть полезно для некоторых языков, но требует дополнительной фильтрации лишних результатов;

– lang – выбор языка, для более точного определения фонем. Для русского языка используется значение rus, что позволяет учитывать специфику произношения и повысить качество анализа фонем.

Помимо определения фонем в аудиозаписи, не менее важным этапом является точное установление временных меток их начала и окончания. Функция, используемая для анализа, возвращает не только информацию о самой фонеме, но и её

длительность, что позволяет с высокой точностью определить моменты, когда звук начинается и заканчивается. Эти данные становятся основой для дальнейшего анализа и преобразования, обеспечивая синхронизацию между звучащей речью и визуальной моделью «аватара». Без корректной привязки к временной шкале даже правильно распознанные фонемы не смогут обеспечить естественное воспроизведение речи в анимации.

После получения фонем с помощью функции Allosaurus возникает необходимость их адаптации к формату, который распознаёт модель генерации «аватара». Визуальная система генерации оперирует 40 различными фонемами, включая одну, отвечающую за тишину. Это позволяет точно передавать как звуковые элементы, так и паузы между ними. Для фонем, соответствующих произносимым словам, выполняется преобразование в специальный формат, совместимый с моделью «аватара», где каждая фонема связана с определённой артикуляцией губ. В то же время моменты тишины обрабатываются отдельно — для них используется специальная фонема, которая указывает на отсутствие звука и позволяет модели корректно возвращаться в нейтральное состояние между фразами. Такой подход обеспечивает целостность и плавность анимации, делая её согласованной с аудио дорожкой.

Процесс анализа и получения фонем из аудиозаписи алгоритмом состоит из следующих шагов:

– очистка выходной директории phoneme_results. Перед началом обработки система полностью очищает папку phoneme_results от ранее сохранённых файлов. Это исключает конфликты между старыми и новыми данными, предотвращая возможные ошибки при записи результатов. Если в папке остались JSON-файлы от предыдущих запусков, они могут быть неверно использованы при работе других алгоритмов обработки;

– получение списка папок из директории output. Алгоритм сканирует папку output, где хранятся аудиофрагменты, разделённые на сегменты [68]. Система собирает имена всех подпапок для последовательной обработки. Если директория output

пуста, процесс завершается с уведомлением о проблеме, чтобы избежать попытки выполнения операций без данных;

– получение списка аудиозаписей в текущей папке. Внутри каждой подпапки модуль ищет аудиофайлы в форматах .wav и .mp3. Проверяется целостность файлов. При наличии файла, который имеет неправильную структуру или формат, программа выдаст сообщение об ошибке и завершит работу;

– определение типа первого фрагмента (Word/Silence). Система анализирует начальный аудиофрагмент в папке, выявляя, начинается ли он с речевого сегмента или паузы. Эта проверка необходима для корректного задания последовательности обработки, так как процесс обработки фонем предполагает чередование слов и тишины. Однако аудиозаписи могут начинаться как с произнесения первого слова, так и с кратковременной паузы. Данный этап реализован для учёта различных вариантов;

– извлечение массива фонем. С помощью библиотеки Allosaurus аудиофайл разбивается на фонемы с временными метками. Модель определяет начало и конец каждой фонемы с точностью до миллисекунд, что критично для синхронизации с анимацией;

– получение длительности аудиофайла. Измеряется общая длительность фрагмента, чтобы убедиться, что временные метки фонем не выходят за его пределы. Это позволяет выявлять ошибки в определении границ звука;

– преобразование фонем в стандартизированный формат. Фонемы, выделенные моделью Allosaurus, конвертируются в формат, совместимый с моделью «аватара». Для пауз используется специальный символ SIL, чтобы «аватар» оставался в нейтральном состоянии в моменты тишины;

– сохранение данных в JSON. Результаты обработки каждого аудиофайла записываются в JSON-файл. Он содержит информацию о фонемах, их таймингах и общей длительности, что обеспечивает достаточное количество информации для дальнейшего преобразования;

– вывод сообщения об успешности операции. После обработки всех файлов в папке выводится уведомление о завершении. Это позволяет пользователю отслеживать прогресс и своевременно выявлять ошибки;

– завершение работы алгоритма. Алгоритм проверяет, все ли папки в директории output были обработаны, освобождает используемые ресурсы и завершает работу.

Блок-схема алгоритма определение фонем в выделенных речевых фрагментах представлена на рисунках 13-15.

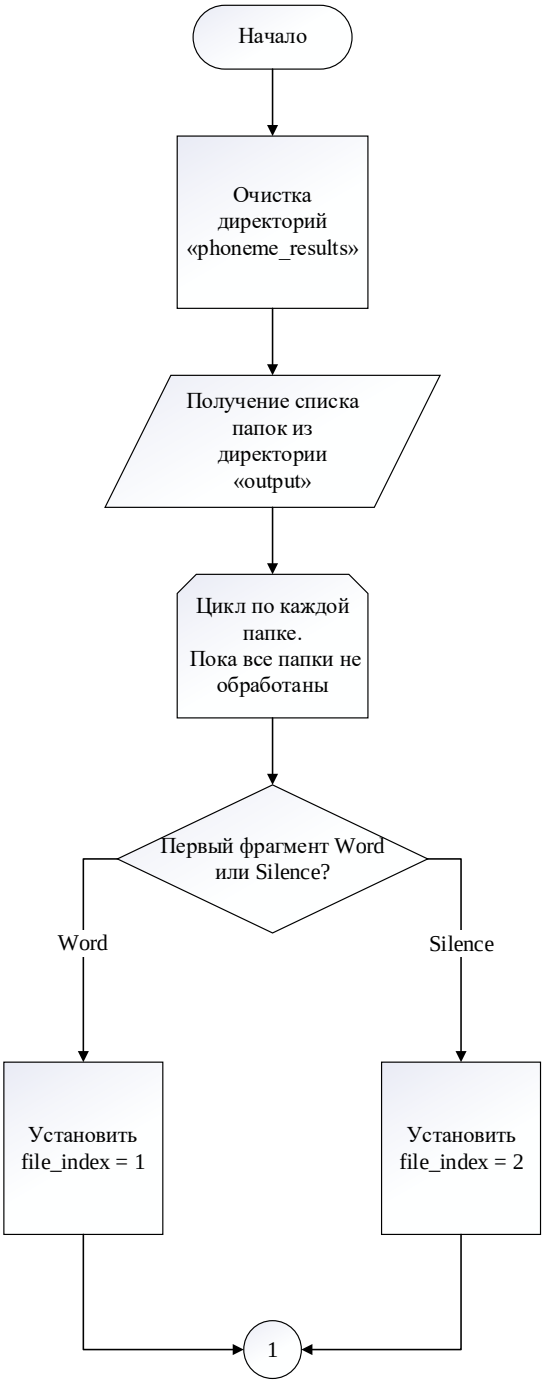


Рисунок 13 – Блок-схема алгоритма определение фонем

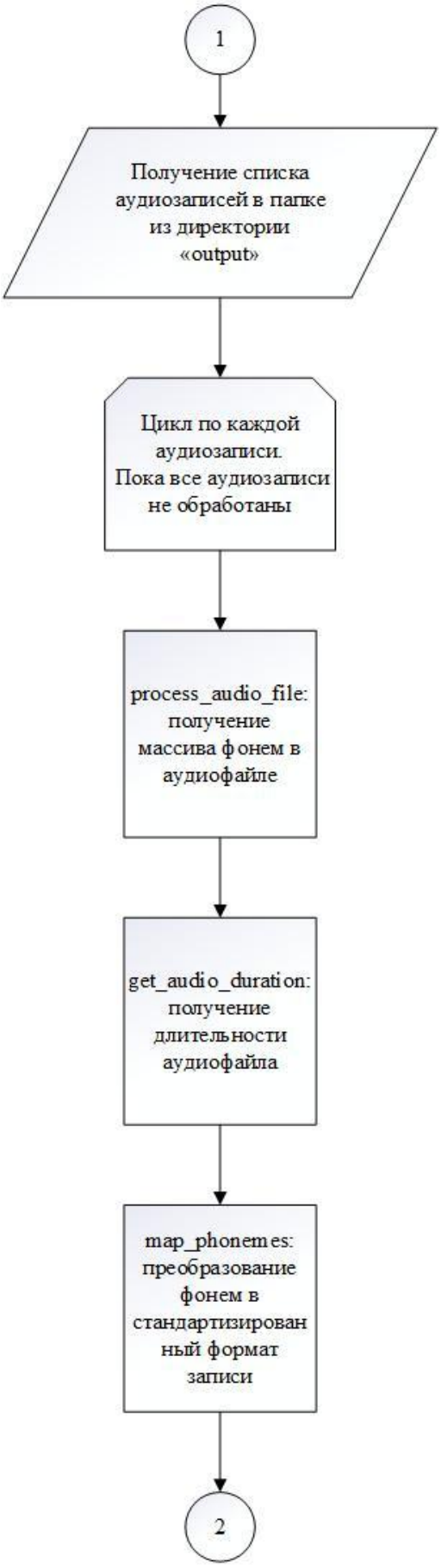


Рисунок 14 – Блок-схема алгоритма определение фонем



Рисунок 15 – Блок-схема алгоритма определение фонем

После завершения обработки алгоритм формирует структурированный список фонем с точными временными метками начала и окончания каждой из них. Это

позволит в дальнейшем создать точную последовательность произносимых фонем, для синхронизации аудио и видео составляющих генерируемого «аватара».

2.2.4 Алгоритм разметки фонем

Следующий этап в работе над подготовкой фонем для модели связан с их точной синхронизацией с фрагментами аудиозаписей, полученными ранее. Эта задача становится особенно критичной при работе с длительными аудиоматериалами. Небольшая погрешность в определении временных меток фонем может накапливаться, превращаясь в значительное отставание или опережение звука относительно визуального ряда. На записи продолжительностью более десяти минут ошибка в 10 миллисекунд на каждые пять секунд аудио приведет к рассинхронизации, превышающей одну секунду, что сделает невозможным корректное воспроизведение анимации губ «аватара» в такт речи. Для устранения подобных проблем используется алгоритм, основанный на анализе и сопоставлении временных меток аудиофрагментов и фонем.

Работа алгоритма начинается с загрузки аудиозаписей, разделенных на фрагменты, соответствующие отдельным словам. На основе этих данных извлекаются точные временные метки начала и конца каждого слова. Далее, используя предсказанные с помощью библиотеки Allosaurus длительности фонем внутри слова. После чего рассчитываются их начальные и конечные тайминги. Однако, иногда возникает несовпадение. Модель определения фонем для больших слов иногда выдает суммарную длительность фонем, которая не совпадает с реальной продолжительностью самого слова. Если сумма длительностей фонем превышает длину аудиофрагмента, тайминги перераспределяются пропорционально исходным весам, сохраняя относительные соотношения между фонемами. Если же сумма меньше, недостающее время равномерно добавляется к наиболее продолжительным гласным фонемам.

Также внимание уделяется обработке пауз, которые в моделях обычно обозначаются как SIL (silence). Для таких моментов вычисляются временные границы, соответствующие началу и концу тишины между словами. Это позволяет учитывать не только речь, но и естественные паузы, что необходимо для создания реалистичной

анимации. Результаты синхронизации сохраняются в формате JSON, который обеспечивает структурированное хранение данных. Каждая запись для слова содержит название аудиофрагмента, список фонем с точными временными метками начала и конца каждой фонемы. Для моментов пауз указывается код SIL, а также тайминги начала и конца фрагмента. В отличие от простых текстовых файлов, где информация представлена в виде неупорядоченного потока, JSON обеспечивает иерархию, где каждое слово, фонема и пауза имеют свои атрибуты с временными метками [69]. Такой подход упрощает интеграцию данных в дальнейшие этапы обработки и позволяет легко обрабатывать программно, а человеку – проверять вручную [70]. Фрагмент получаемой структуры данных изображен на рисунке 16.

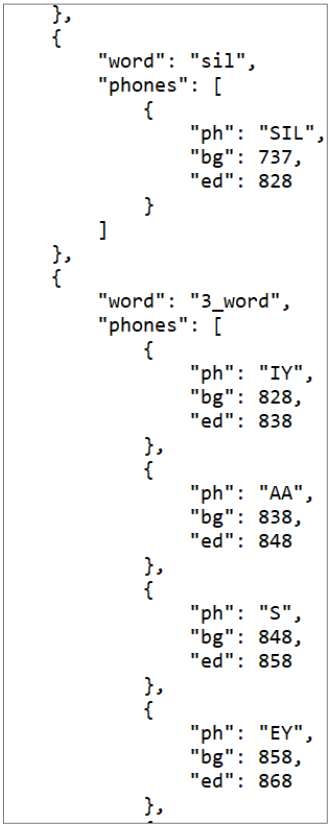


Рисунок 16 – Получаемая структура данных

Важным нюансом обработки данных является конвертация временных меток в сантисекунды. Такое преобразование необходимо для последующего перевода фонем из временной шкалы в покадровую, что обеспечивает точность синхронизации

анимации с аудиодорожкой. Процесс синхронизации фонем с аудиофрагментами состоит из следующих шагов:

– подготовка рабочей области. Перед началом работы система полностью очищает содержимое папки mapper, чтобы исключить влияние старых временных файлов. Это предотвращает возможные ошибки, связанные с конфликтами данных, и обеспечивает чистый старт для нового цикла обработки. Очистка позволяет избежать случайного использования устаревших метаданных, которые могли остаться после предыдущих запусков программы;

– получение списка папок с аудиофрагментами голоса. Программа сканирует директорию output и извлекает названия всех подпапок, содержащих аудиофрагменты. Эти подпапки разделены по продолжительности в 1 минуту. Если директория output пуста, процесс завершается с уведомлением об отсутствии данных для обработки.

– загрузка JSON-файлов с фонемами. Из директории phoneme_results считываются JSON-файлы, содержащие информацию о временных метках и типах фонем для каждого аудиофрагмента. Эти данные становятся основой для дальнейшей обработки и синхронизации с анимацией. Система проверяет корректность структуры и формата файлов, чтобы избежать ошибок при извлечении параметров;

– последовательная обработка папок с аудиофрагментами голоса. Для каждой папки в output запускается отдельный цикл обработки. Это позволяет систематизировать данные и избежать перегрузки памяти при работе с большими объемами информации. После завершения обработки одной папки алгоритм переходит к следующей, сохраняя целостность результатов;

– сортировка аудиофрагментов по порядку. Файлы внутри папок сортируются по возрастанию их числового индекса. Это обеспечивает правильную последовательность обработки и формирования списка обработки, соответствующую исходной аудиодорожке. Без сортировки возможны ошибки, связанные с системными особенностями операционной системы, на которой запускается программа;

– определение типа первого фрагмента (слово/тишина). Система проверяет, начинается ли аудио с речи или паузы, анализируя название первого файла. Это влияет на начальные тайминги анимации «аватара» и последовательность обработки;

– инициализация списка данных для хранения результатов. Создается структура данных, куда будут добавляться параметры каждого фрагмента: время начала, окончания, код фонемы или паузы. Эта структура заранее определяется для упрощения дальнейшей работы с массивами и минимизации риска ошибок при добавлении элементов;

– обработка каждого аудиофрагмента голоса в папке. Для каждого файла определяется его тип – слово или тишина. На основе этого выбирается конкретный алгоритм обработки. Извлечение таймингов фонем для слов или расчет длительности для пауз. Все данные собираются в единую структуру, которая будет использоваться для синхронизации с анимацией;

– расчёт длительности фрагментов и фонем. Для слов длительность делится между фонемами пропорционально, чтобы суммарное время совпадало с аудиофрагментом. Для пауз длительность берется напрямую из файла. Если сумма длительностей фонем не совпадает с реальным временем слова, значения корректируются для сохранения точности;

– формирование структуры данных для анимации. Каждая фонема или пауза записывается в список данных с точными временными метками указанных в сантисекундах. Это позволяет связать исходную аудиозапись с выявленными фонемами. Структура данных содержит как информацию о произносимых словах, так и о паузах;

– сохранение данных в JSON-формат. Сформированные данные записываются в JSON-файлы в директории mapper. JSON формат обеспечивает структурированность и удобство программного чтения, а также возможность ручной проверки результатов. Каждый файл содержит полную информацию о таймингах и типах фрагментов;

– уведомление об успешном завершении. После обработки всех файлов выводится сообщение о завершении операции. Это позволяет отслеживать прогресс и

выявлять возможные ошибки отсутствия данных или проблемы с кодировкой для отдельных фрагментов. Уведомления также служат подтверждением корректной работы алгоритма.

Блок-схема алгоритма синхронизации аудиофрагментов и списка полученных фонем представлена на рисунках 16-19.

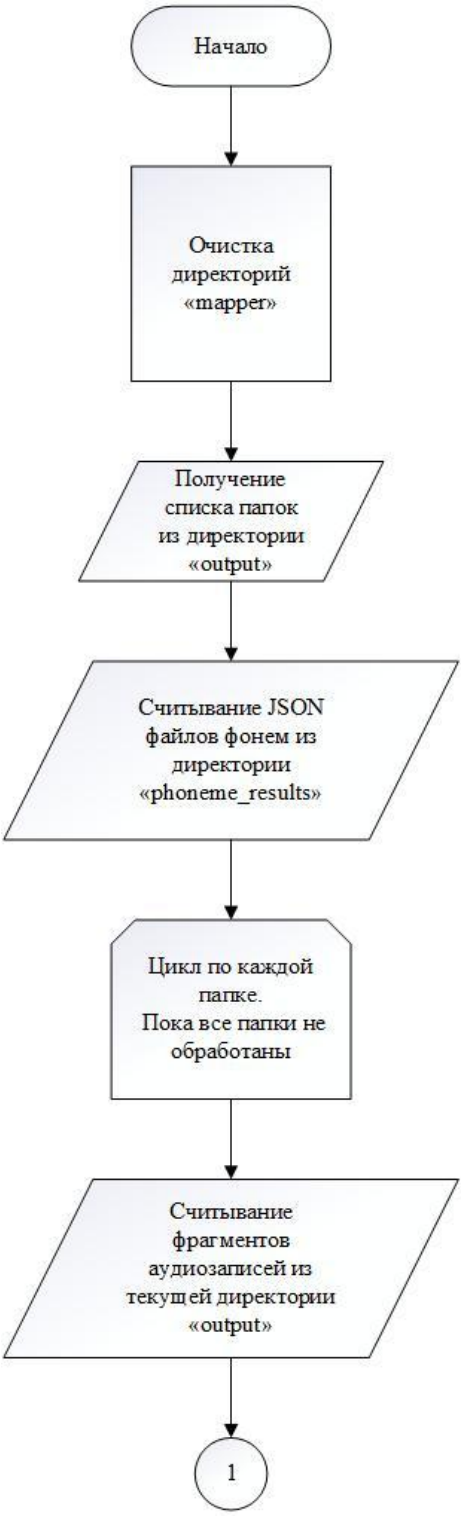


Рисунок 16 – Блок-схема алгоритма синхронизации аудиофрагментов и фонем

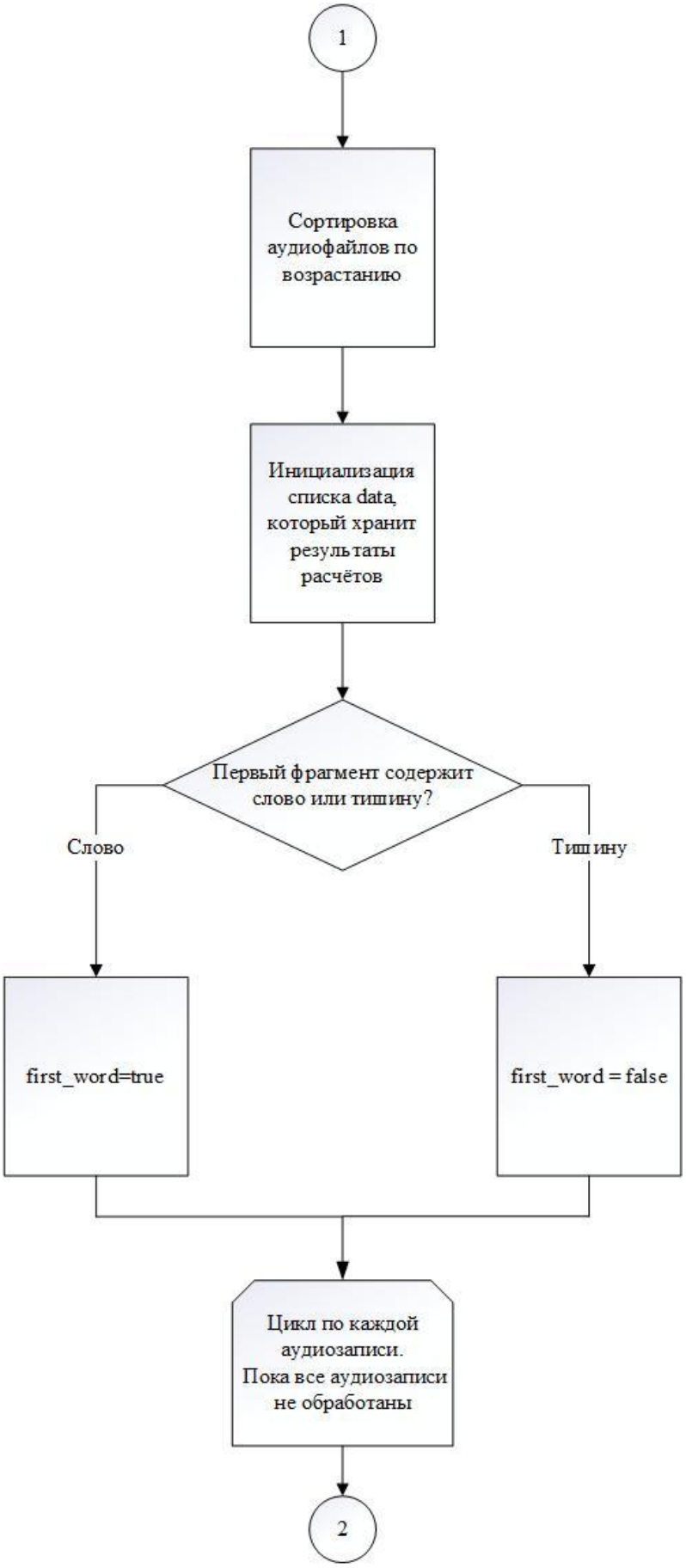


Рисунок 17 – Блок-схема алгоритма синхронизации аудиофрагментов и фонем



Рисунок 18 – Блок-схема алгоритма синхронизации аудиофрагментов и фонем



Рисунок 19 – Блок-схема алгоритма синхронизации аудиофрагментов и фонем

Сформированные структуры в формате JSON служат основой для последующей интеграции с системой генерации анимации. Они содержат детализированную информацию о временных интервалах фонем и пауз, адаптированную под требования модели «аватара».

2.2.5 Алгоритм преобразования фонем в покадровый формат

Последним алгоритмом в модуле получения фонем из аудиофайла голоса «аватара» является преобразование детализированного списка фонем, в котором указаны временные метки начала и конца произношения фонемы, в формат принимаемый системой генерации. Этот процесс требует адаптации данных под специфические требования модели, которая оперирует не временными интервалами, а дискретными кадрами. Основная задача заключается в создании списка, где каждый элемент соответствует отдельному кадру и содержит информацию о том, какая фонема должна быть отображена в этот момент.

Первым этапом становится перевод из временной шкалы в покадровую. Поскольку модель генерирует видеозапись с частотой 30 кадров в секунду, каждый кадр занимает 33 миллисекунды. На основе этого рассчитывается общее количество кадров, которое потребуется сгенерировать. Далее анализируются временные метки каждой фонемы. Для определения того, какая фонема должна отображаться в кадре, алгоритм сравнивает временные границы текущей фонемы и время текущего отображаемого кадра. Если время кадра находится в интервале текущей фонемы, то она фиксируется для отображения на этом кадре. Если временная метка кадра больше, чем текущий интервал фонемы, то алгоритм переходит к следующей фонеме в слове.

Важной особенностью является числовое кодирование фонем. Вместо символьных обозначений, которые используются в исходных данных, модель требует преобразования каждого символа в числовой идентификатор. Паузы обозначаются специальным кодом SIL, которому присваивается числовое значение 31. Обработка пауз требует особого внимания. Если текущий фрагмент аудио соответствует паузе, система вставляет код 31 в соответствующие кадры. Это обеспечивает возможность

создания переходов между словами. «Аватар» сохраняет нейтральное выражение лица во время тишины, что делает движение более естественным.

Цикл обработки кадров выполняется последовательно, начиная с первого и заканчивая последним кадром аудиозаписи. На каждом шаге алгоритм проверяет, какая фонема или пауза должна быть активной в данный момент, и формирует выходной список. Если в процессе обработки текущего слова заканчиваются фонемы, система автоматически переходит к следующему слову, продолжая заполнение списка до конца записи. Алгоритм преобразования списка фонем в требуемы для модели генерации «аватара» формат состоит из следующих шагов:

– очистка рабочей директории. Перед началом обработки система полностью очищает содержимое папки result, чтобы исключить влияние старых данных на новый цикл работы. Это предотвращает возможные конфликты, такие как случайное использование устаревших файлов, и гарантирует, что результаты будут соответствовать текущему набору данных;

– получение списка файлов. Из директории mapper считываются JSON-файлы, содержащие информацию о фонемах и их временных метках. Эти данные служат основой для дальнейшей обработки, так как в них хранятся точные временные границы каждой фонемы и паузы. Проверяется доступность файлов и соответствие их структуры ожидаемому формату, чтобы избежать ошибок при извлечении параметров;

– обработка каждого файла. Для каждого файла запускается отдельный цикл обработки, который включает чтение данных, анализ временных меток и преобразование фонем в покадровую последовательность. Это позволяет автоматизировать процесс обработки большого количество данных. Пропадает необходимость повторного запуска программы, для обработки нескольких голосовых аудиофрагментов. После завершения обработки одного файла алгоритм переходит к следующему, сохраняя целостность результатов;

– расчет общего числа кадров. На основе длительности аудиофрагмента и частоты 30 кадров/секунду определяется общее количество кадров frame_num. Расчет

учитывает возможные дробные значения, которые округляются для корректного отображения в кадровой шкале;

– инициализация переменных. Создаются счетчики word_index и phone_index, отслеживающие текущее положение в списке слов и фонем. Эти переменные позволяют последовательно перебирать элементы данных и корректно переходить между словами и фонемами. Также инициализируется список phoneset_list, который будет хранить результаты обработки кадров;

– добавление первой фонемы. В список phoneset_list помещается первая фонема из текущего слова, которая будет отображаться в первом кадре. Это обеспечивает корректный старт анимации, так как «аватар» начинает движение губ с первой фонемы. Если речь начинается с короткой паузы, в список добавляется соответствующий код, и «аватар» сохраняет нейтральное выражение лица до начала речи;

– цикл по всем кадрам. Для каждого кадра проверяется, изменилась ли фонема по сравнению с предыдущим кадром. Сравниваются временные метки текущей фонемы временем кадра в общей временной шкале. Если время кадра выходит за границы текущей фонемы, происходит переход к следующей. Этот процесс учитывает длительность кадра в 3.33 сантисекунды и корректно обрабатывает переходы между звуками и паузами;

– переход между фонемами и словами. Если текущая фонема закончилась, система переходит к следующей в рамках одного слова. Если слово полностью обработано, алгоритм переходит к следующему слову, используя данные из JSON-файла. Паузы обрабатываются аналогично словам, но вместо фонем добавляется специальный код, который указывает на отсутствие движения губ;

– конвертация фонем в индексы. После обработки всех кадров фонемы заменяются на числовые коды, необходимые для работы модели;

– сохранение результатов. Полученный массив сохраняется в JSON-файл в директории result и содержит покадровый список произносимых «аватаром» фонем;

– завершение работы алгоритма. По окончании обработки всех файлов выводится сообщение об успешном завершении операции. Это позволяет отслеживать прогресс и выявлять возможные ошибки;

Блок-схема алгоритма преобразования списка фонем в формат принимаемой системой генерации представлена на рисунках 20-22.

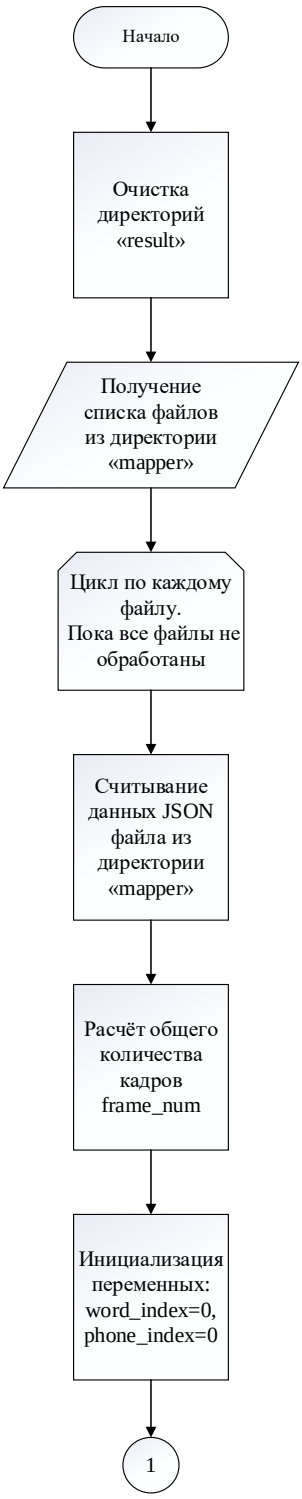


Рисунок 20 – Алгоритм преобразования списка фонем

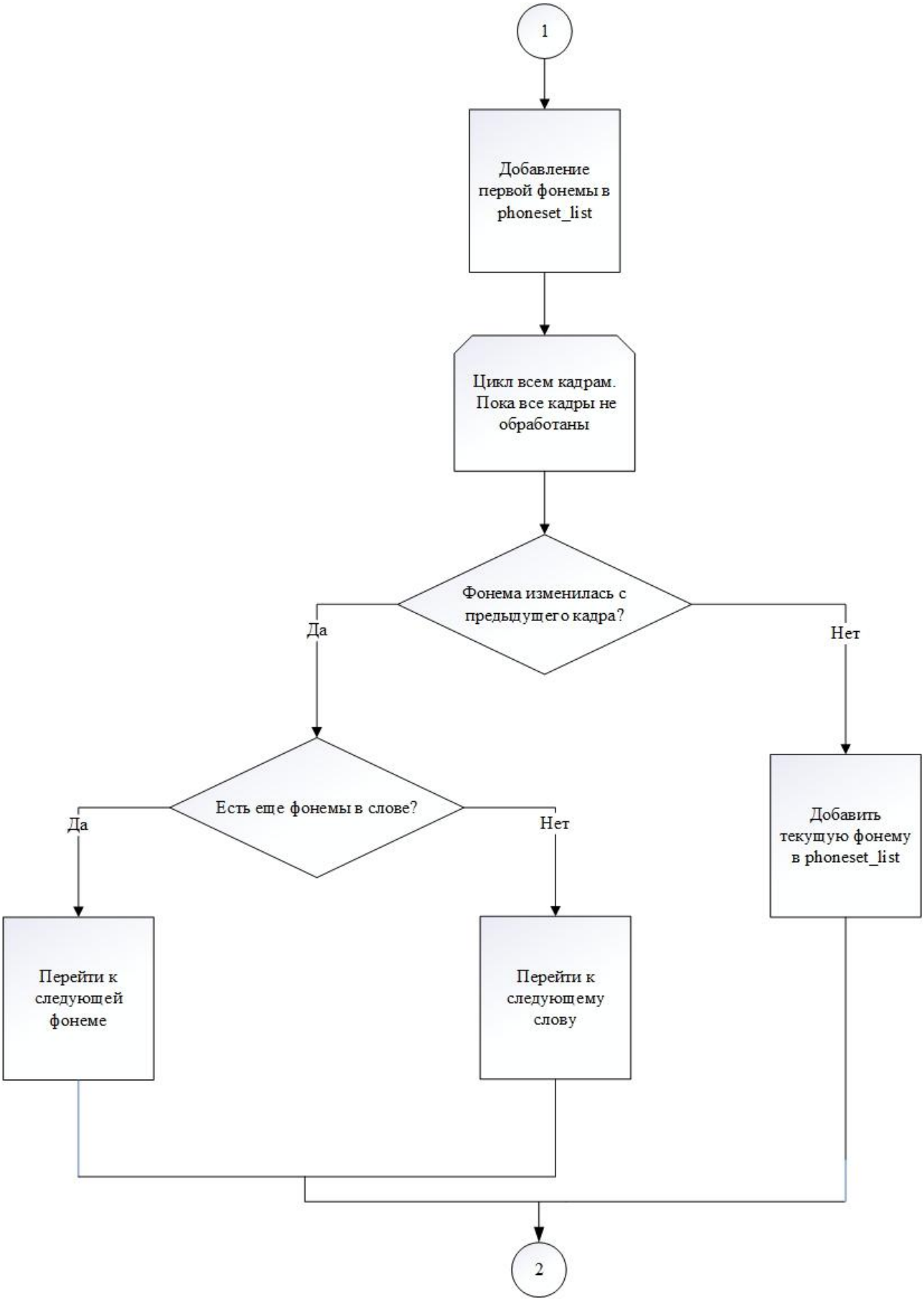


Рисунок 21 – Алгоритм преобразования списка фонем

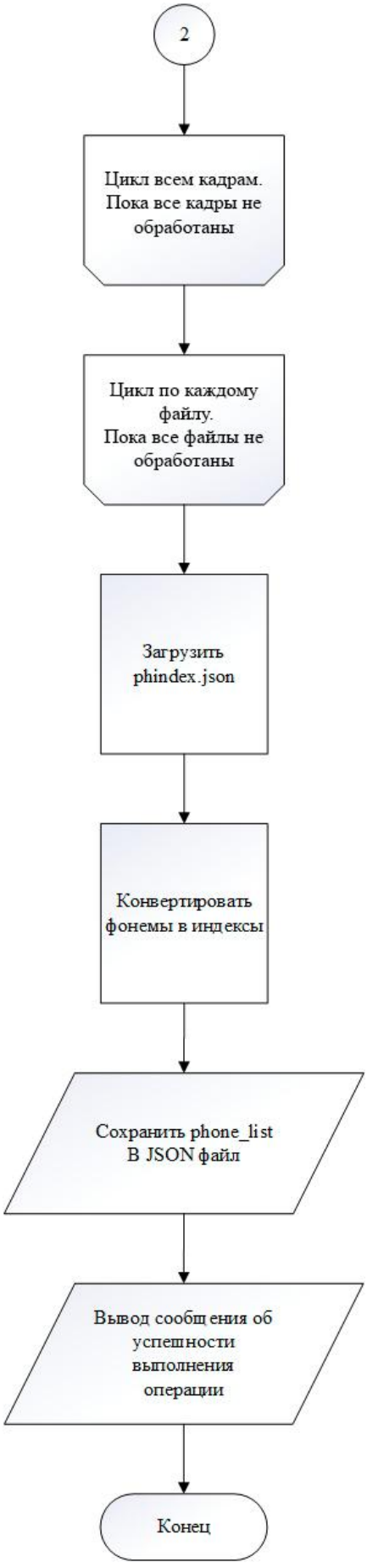


Рисунок 22 – Алгоритм преобразования списка фонем

Модуль преобразования списка фонем эффективно и удобно для пользователя анализирует аудиофайл с голосом «аватара» и формирует список фонем. На каждом этапе обработки реализованы механизмы валидации данных, включая проверку корректности форматов файлов и их целостности. Это позволяет минимизировать риски ошибок, вызванных некорректными входными данными, такими как повреждённые аудиозаписи или файлы с неподдерживаемыми расширениями и кодировками. Система поддерживает пакетную обработку, что делает её эффективной для работы с большими объёмами данных. Например, пользователь может одновременно загрузить десятки аудиофайлов, и модуль последовательно сгенерирует для каждого из них отдельный список фонем. Архитектура модуля построена с учётом гибкости и независимости от внешних компонентов. Благодаря модульному подходу, система легко адаптируется к внедрению новых методов распознавания фонем. Достаточно заменить соответствующий компонент, при этом не затрагивая остальные части алгоритма. Кроме того, модуль может быть адаптирован для работы с другими системами генерации, даже если они используют иные принципы синхронизации речи и визуализации.

2.3 Модуль захвата особенностей лица

2.3.1 Структура модуля

Данный модуль отвечает за формирование визуальной составляющей «аватара». Он реализует комплексный подход к анализу и реконструкции изображений, основываясь на параметрах 3DMM (3D Morphable Model) и методах компьютерного зрения [71]. Данная технология позволяет создавать трехмерные модели лиц с детализированной текстурой, на основе множества параметров. Этот подход позволяет кодировать лицевые структуры в виде математических коэффициентов [72].

Для сбора параметров лица и создания 3D модели, необходимо проанализировать исходное видео с рассказчиком. Для анализа кадра задействованы нейросетевые инструменты компьютерного зрения, обученные на больших датасетах изображений. Самым популярным и доступным инструментом на текущий момент является

библиотека OpenCV, которая предоставляет большое количество алгоритмов для обработки изображений и видео [73]. С её помощью можно выделять объекты в кадре, включая животных и людей. В данном модуле алгоритмы компьютерного зрения используются для определения рассказчика в кадре и считывания ключевых точек с лица. После идентификации пользователя модуль переходит к извлечению параметров [74]. С помощью предобученных моделей считываются геометрические особенности лица. Также анализируются и внешние параметры фона, распределения света, цветовой температуры и наличия теней. Эти данные преобразуются в нормализованный формат, что позволяет адаптировать их под требования 3DMM-модели. Собранные параметры используются для реконструкции трехмерной модели лица. Процесс включает построение полигональной сетки, наложение текстур и калибровку динамических свойств. Система автоматически корректирует форму подбородка, изгиб бровей и другие анатомические нюансы, обеспечивая соответствие цифрового двойника оригиналу [75]. С помощью дополнительных алгоритмов обеспечивается синхронизация виртуального освещения. Результирующий набор данных передается в модуль рендеринга, где параметры позиционирования, геометрии и фотометрии преобразуются в конечное изображение «аватара».

2.3.2 Алгоритм получения ключевых точек лица

Первый алгоритм, с которого начинается работа модуля это алгоритм получения ключевых точек с лица рассказчика. Его работа начинается с обработки исходного видео, содержащего речь человека [76]. Важным условием является формат кадра с разрешением 256 на 256 пикселей, который оптимизирован для работы используемой модели генерации «аватара». Такое разрешение позволяет сохранить детализацию лица и минимизировать нагрузку на вычислительные ресурсы, что критично при работе таких сложных моделей. Это позволяет использовать модель на персональных компьютерах, не прибегая к серверным и облачным решениям. Видео последовательно разбивается на отдельные кадры, так как алгоритмы анализа изображений оперируют статичными снимками, а не потоковым видео.

На следующем этапе для каждого кадра выделяются ключевые точки, которые являются цифровыми метками, которые описывают геометрию лица и его динамические изменения. В основе этого процесса лежит стандарт 3DMM, который определяет 68 точек, равномерно распределенных по лицевой области [77]. Эти точки позволяют фиксировать статичные черты лица и микродвижения, необходимые для воссоздания естественной мимики «аватара» [78]. Точность захвата особенностей лица обеспечивается продуманным расположением ключевых зон [79]:

– контур лица – 17 точек очерчивают границы от подбородка через скулы до линии роста волос;

– брови – 10 точек фиксируют форму, изгиб и положение относительно глазной впадины;

– глаза – 12 точек отмечают контуры век, уголки глаз и позицию зрачков;

– зрачки – 2 точки определяют направление взгляда с точностью до миллиметра;

– нос – 9 точек охватывают основание, переносицу и кончик носа;

– рот – 20 точек анализируют мимику, определяют форму, ширину и открытие рта. Внешний контур содержит 12 точек, внутренний 8 точек;

Алгоритм получения ключевых точек из опорного видео состоит из следующих шагов:

– считывание видеофайлов из директории input_dir. Программа рекурсивно сканирует указанную директорию выявления файлов, соответствующих форматам видео. Алгоритм определяет тип файла по расширению и фильтруют только медиафайлы, подходящие под обработку. Список найденных видео передаётся в очередь для последовательной обработки;

– цикл обработки для каждого видеофайла. Каждый видеофайл обрабатывается поочерёдно, что обеспечивает управление использованием памяти и ресурсов системы. Процесс выполняется последовательно, исключая одновременную загрузку нескольких файлов, что снижает риск перегрузки оборудования. После завершения обработки одного файла алгоритм переходит к следующему в очереди;

– создание выходной директории для текущего видео. Для каждого обрабатываемого видео создаётся уникальная поддиректория внутри основной выходной папки. Имя поддиректории формируется на основе названия исходного файла, что обеспечивает структурированное хранение результатов. Это необходимо, так как результатом выполнения программы является множество файлов, в которых указаны ключевые точки для каждого кадра видео;

– открытие видео с использованием cv2.VideoCapture. Библиотека OpenCV используется для инициализации потока чтения видео. Метод cv2.VideoCapture устанавливает соединение с файлом, определяет его параметры (разрешение, частоту кадров) и готовит к последовательному извлечению кадров. Проверяется успешность открытия файла перед началом обработки;

– цикл обработки каждого кадра. Алгоритм последовательно считывает каждый кадр из видео до достижения конца файла. Каждый кадр проходит цепочку преобразований. Цикл завершается после обработки всех доступных кадров;

– преобразование цветового пространства. Кадр преобразуется из цветового пространства BGR используемого в OpenCV в RGB, так как большинство инструментов визуализации и анализаторов изображений требуют именно этот формат. Конвертация выполняется через стандартные методы библиотеки OpenCV, сохраняя качество изображения;

– конвертация в PIL-изображение. Кадр преобразуется в объект PIL.Image для совместимости с библиотеками обработки изображений Pillow. Это позволяет применять функции детектирования объектов;

– закрытие cv2.VideoCapture. После подготовки всех кадров к анализу с помощью модели завершается, соединение с файлом через cv2.VideoCapture закрывается, освобождая занимаемые ресурсы;

– извлечение ключевых точек. Используя нейросетевую модель FaceAlignment, система выделяет ключевые точки на кадрах. Результаты фиксируются в структурированном формате для последующего сохранения;

– сохранение ключевых точек. Извлеченные данные сохраняются в указанную выходную директорию в текстовом формате, который требуется для следующего алгоритма;

– завершение работы алгоритма. По окончании обработки всех файлов система освобождает все ресурсы, записывает финальный отчёт в лог и завершает работу.

Блок-схема алгоритма получения ключевых точек из опорного видео представлена на рисунках 23-25.



Рисунок 23 – Блок-схема алгоритма определения ключевых точек



Рисунок 24 – Блок-схема алгоритма определения ключевых точек

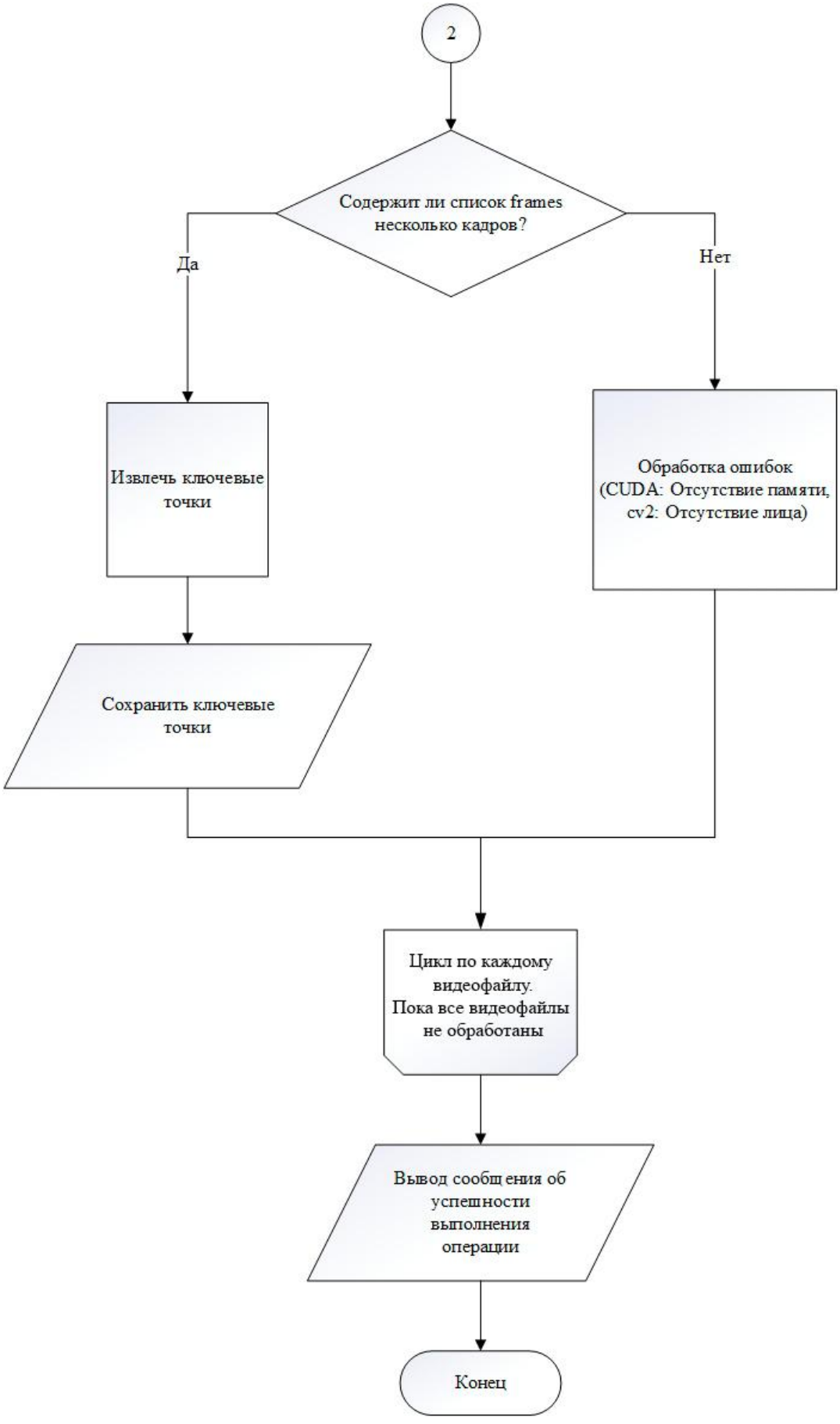


Рисунок 25 – Блок-схема алгоритма определения ключевых точек

Поскольку нейросетевой алгоритм работает на вероятностной основе, его результаты требуют визуальной проверки для подтверждения корректности определения ключевых точек. Визуализация ключевых точек поверх исходного изображения обеспечивает объективную оценку точности алгоритма. Вычисленные координаты корректно отражают контуры губ, носа, бровей, а также позиционирование глаз и направление взгляда.

2.3.3 Алгоритм захвата особенностей лица

Полученные ключевые точки позволяют проанализировать изображение намного детальнее. Для этого был разработан следующий алгоритм, который считывает особенности лица рассказчика. В ходе выполнения, алгоритм извлекает из изображения множество параметров. Далее данные структурируются в виде нормализованных векторов, описывающих как статические особенности лица, так и динамические параметры [80]. Векторы формируются на основе предсказаний предобученной на миллионах изображений нейросети. Также алгоритм геометрическую обработку изображения, что обеспечивает высокую точность получаемых данных [81]. Алгоритм выделяет следующее множество векторов, описывающих лицо рассказчика и его окружение:

– вектор уникальных черт лица (1x80). Содержит параметры, включающие расстояние между глазами, форму скул, длину подбородка, которые характеризуют индивидуальные анатомические особенности рассказчика;

– вектор мимики (1x64). Отражает динамику мышечных движений, связанных с выражением эмоций. Каждая компонента управляет деформацией конкретного участка лица, что позволяет детально считывать эмоции;

– вектор текстурных параметров (1x80). Характеризует микроструктуру кожи, включая пигментацию, шероховатость и распределение теней на лице. Данные используются для точного воссоздания внешности лица при генерации;

– вектор положения головы (1x3). Определяет ориентацию головы в пространстве через углы Эйлера (рыскание, тангаж, крен). Это позволяет учитывать повороты и наклоны головы рассказчика;

– вектор освещения и цветокоррекции (1x27). Включает коэффициенты, описывающие параметры яркости и направления света сцены, а также цветовые особенности кадра. Используется для цветовой нормализации изображения;

– матрица ключевых точек лица (2x68). Хранит ранее полученный координаты 68 точек, расположенных на контурах глаз, носа, рта и краях лица. Точки служат основой для генерации «аватара».

Интеграция всех векторов в единую систему обеспечивает комплексное описание как физических характеристик рассказчика, так и контекста съемки.

Полученные ключевые точки позволяют проанализировать изображение намного детальнее. Для этого был разработан следующий алгоритм, который считывает особенности лица рассказчика. В ходе работы алгоритма выделяется вектора, которые содержат нормализованные значения переменных. Множество векторов описывающие лицо рассказчика:

– вектор уникальных черт лица (1x80). Каждая компонента вектора кодирует индивидуальные особенности человека из исходного видео (расстояние между глазами, форму носа);

– вектор мимики (1x64). Используется для управления выражением лица. Каждая компонента вектора управляет отдельными участками лица;

– вектор текстурных параметров (1x80). Описывает визуальные свойства кожи на отдельных участках лица;

– вектор положения головы (1x3). Три эйлеровых угла, определяющие положение головы;

– вектор освещения и цветокоррекции (1x27). Коэффициенты для моделирования условий освещения и коррекции цвета;

– матрица ключевых точек лица (2x68). Хранит 68 точек, которые очерчивают контуры глаз, носа и рта рассказчика.

Это позволяет получить подавляющее количество информации как о самом рассказчике, так и окружающей его сцене. Алгоритм захвата особенностей лица рассказчика из опорного видео состоит из следующих шагов:

– считывание видеофайлов и ключевых точек. Алгоритм начинает работу с загрузки видеофайлов и данных о ключевых точках лица из указанных источников. Система проверяет форматы файлов и готовит их к дальнейшим операциям.

– инициализация модели. Подготовка нейросетевой модели Deep3DFaceRecon [82], которая считывает коэффициенты с изображения;

– проверка количества файлов. Система сравнивает количество загруженных видеофайлов с количеством файлов ключевых точек. Если числа не совпадают, то выдаётся сообщение об ошибке, и программа завершает свою работу;

– чтение видео. Если количество файлов совпадает, алгоритм открывает доступ к видеофайлам через библиотеку OpenCV. Это позволяет считывать кадры последовательно и передавать их в обработку. Проверяется успешность открытия каждого файла;

– извлечение кадров. Видео разделяется на отдельные кадры, которые сохраняются в массив для дальнейшей обработки. Каждый кадр преобразуется в стандартный формат RGB формат, что обеспечивает совместимость с моделью;

– загрузка ключевых точек. Данные о ключевых точках загружаются из файлов и приводятся к единому формату. Это необходимо для нахождения рассказчика на изображении;

– выравнивание изображений. Каждый кадр выравнивается относительно ключевых точек, что устраняет влияние поворотов головы и изменения масштаба. Операция выполняется с использованием геометрических трансформаций, из-за чего пропорционально лицо не изменяется;

– создание VideoPathDataset. Формируется датасет, который хранит пути к изображениям и соответствующие им ключевые точки. Это упрощает управление данными и обеспечивает эффективный доступ к ним в процессе работы модели;

– запуск модели предсказания коэффициентов. Модель обрабатывает изображения и ключевые точки, вычисляя коэффициенты, которые описывают параметры мимики, освещения, текстуры и другие. Результаты фиксируются в памяти для последующего объединения;

– получение коэффициентов. На выходе модели получается набор числовых значений, характеризующих особенности каждого кадра. Коэффициенты кодируют как статические черты лица, так и динамические изменения;

– объединение результатов. Рассчитанные коэффициенты собираются в единый структурированный файл. Данные группируются в общую таблицу, формат которой подходит для нейросети генерирующей конечное изображение;

– сохранение результатов в .mat файл. Итоговые данные сохраняются в формате .mat, который поддерживает хранение многомерных массивов;

– вывод сообщения об успешности операции. По завершении обработки всех кадров система выводит уведомление о результате выполнения алгоритма;

– завершение работы алгоритма. Алгоритм завершает работу, освобождая занимаемые ресурсы и закрывая соединения с файлами.

Блок-схема алгоритма захвата особенностей лица рассказчика из опорного видео представлена на рисунках 26-28.

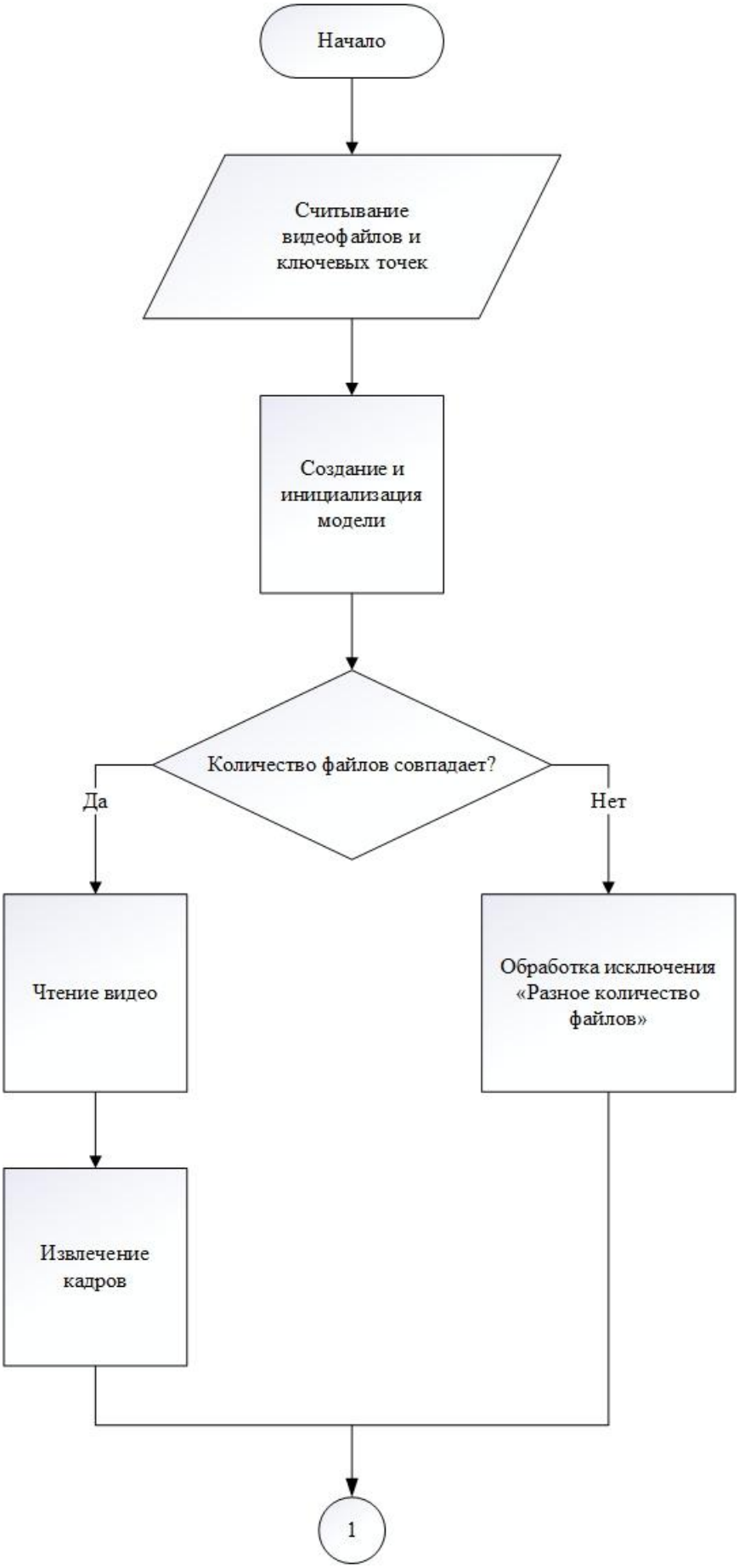


Рисунок 26 – Алгоритм захвата особенностей лица рассказчика



Рисунок 27 – Алгоритм захвата особенностей лица рассказчика

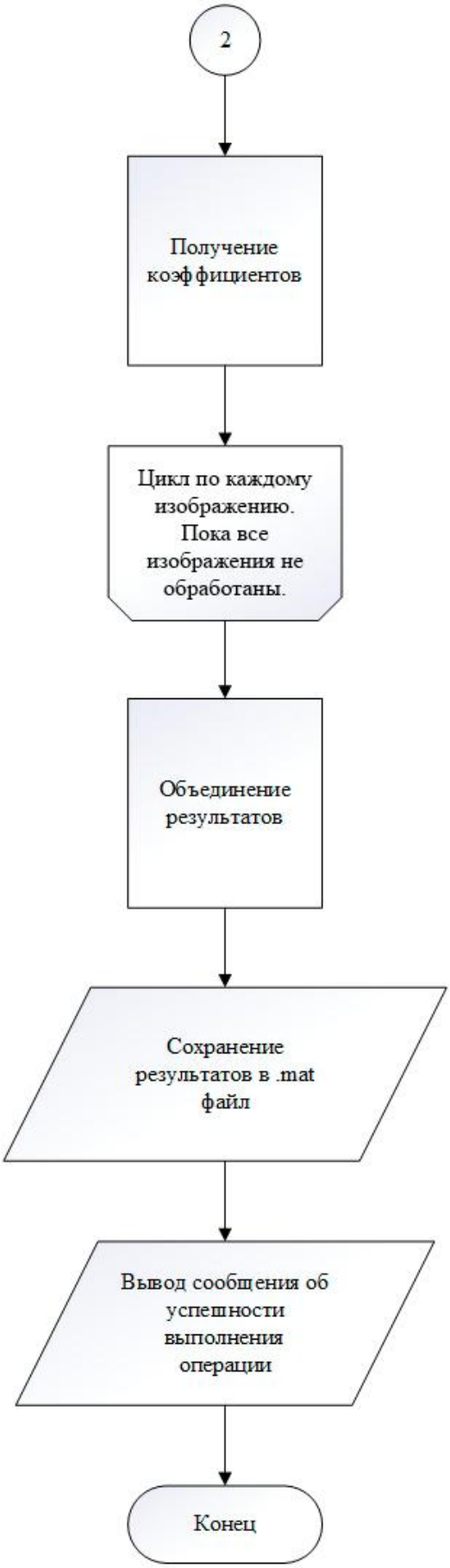


Рисунок 28 – Алгоритм захвата особенностей лица рассказчика

Модуль анализа лица обрабатывает исходное видео и извлекает параметры, необходимые для генерации «аватара». Технология повышает реалистичность за счёт использования реальных анатомических пропорций лица, а не синтетических значений. Полученные коэффициенты допускают повторное применение, что позволяет создавать профили рассказчиков с индивидуальными чертами и особенностями лица.

2.4 Модуль захвата движений головы

2.4.1 Структура модуля

Модуль захвата движений головы разработан для получения параметров ее движения, которые используются при генерации «аватара». Он позволяет точно определить и считать с опорного видео множество этих параметров. Естественные микродвижения головы во время речи сильно влияют на реалистичность «аватара». Во время речи человек, непроизвольно совершает множество движений головы, подчеркивая эмоции рассказчика. Эти едва уловимые смещения, повороты и важны для создания правдоподобного образа. «Аватар» с абсолютно статичной головой на протяжении всей речи воспринимается как неестественный и ненастоящий. Модели, способные учитывать динамику головы в реальном времени, являются прорывными, что позволяет значительно повысить визуальную достоверность.

Для извлечения необходимых параметров движения головы из опорного видео был разработан специализированный алгоритм. Этот процесс начинается с этапа подготовки видеоданных. В качестве основного инструмента для анализа используется модель Deep3DFaceRecon [83]. Данная модель применялась в предыдущем модуле, но она также хорошо подходит для считывания параметров движения головы. Работа модели сфокусирована на трекинге головы, не реализовывая потенциал в получении особенных параметров лица в целях экономии ресурсов. Первым шагом подготовки является преобразование исходного видео в разрешение 256x256 пикселей. Это стандартизирует входные данные и значительно ускоряет время вычисления параметров. После изменения разрешения видео разделяется на отдельные кадры. Анализ положения головы и извлечения параметров выполняется независимо для каждого кадра.

Далее обрабатывая каждый кадр, необходимо обнаружить лицо рассказчика и точно считать его лицевые ориентиры. Для решения этой задачи применяются современные методы компьютерного зрения, которые способны детектировать лицо человека в кадре, отделяя его от других объектов. Наиболее продвинутые алгоритмы способны считывать отдельные ключевые точки. Для получения ключевых точек можно использовать алгоритм, разработанный для модуля захвата особенностей лица. Успешное получение и сохранение координат точек лица завершают подготовительный этап.

После того как на изображении определено положение рассказчика и выделены ключевые точки лица, следует этап вычисления параметров движения головы. На этом этапе алгоритм принимает на вход подготовленные изображения и соответствующий им массив координат ключевых точек. Перед непосредственным анализом изображения подвергаются предобработке. Они нормализуются по яркости, контрасту и цветовой гамме, чтобы повысить точность результатов. Далее изображения и ключевые точек выравниваются относительно эталонных 3D-координат lm3d_std. Эти заранее определенные координаты представляют собой стандартизированную, нормализованную модель положения ключевых точек лица в нейтральной позе. Использование lm3d_std позволяет привести все входные кадры к единой системе отсчета. После запуска модели происходит предсказание параметров, описывающих положение головы в каждом кадре. Полученный данные сохраняются в двух форматах. Формат .mat (MATLAB-файл) служит для хранения структурированных числовых данных, которые затем объединяются в единую таблицу, описывающую динамику головы на протяжении всего видео [84]. Параллельно генерируются изображения в формате .png, которые используются для визуального контроля. Такой подход обеспечивает возможность оперативной проверки корректности параметров. Для создания визуализаций применяется библиотека nvdiffrast [85], основанная на дифференцируемом рендеринге. Она позволяет синтезировать изображение лица, используя только предсказанные параметры, без опоры на исходный кадр. Полученная параметрическая маска отображает форму лица, ориентацию головы и даже тонкие детали

[86]. Такая визуализация полезна для выявления ошибок, связанных с плохим освещением в кадре или неточным определением ключевых точек.

2.4.2 Алгоритм вычисления параметров движения головы

После определения положения рассказчика, а также считывания ключевых точек, далее идет определение параметров положения головы. Алгоритм считывает изображения и полученные ключевые точки. Каждое изображение дополнительно преобразуется, чтобы соответствовать требованию модели. Однако, даже для определения положения головы модель считывает параметры текстуру лица и освещения. Далее используя изображения, а также значения ключевых точек, модель запускается и предсказывает параметры. После получения параметров, результаты сохраняются в разных форматах. Формат .mat используется для дальнейшего объединения покадровых данных в единую таблицу. Формат .png используется для визуального контроля качества полученных параметров. Чтобы получать изображения используется библиотека nvdifferast, которая позволяет рендерить изображение исходя из параметров. Получается параметрическая маска, которая строить только на основе считанных параметров. Алгоритм захвата параметров движения головы рассказчика из опорного видео состоит из следующих шагов:

– инициализация модели. Загружается предобученная нейросетевая модель Deep3DFaceRecon_pytorch для извлечения 3D-параметров лица. Модель переводится в режим оценки и распределяется на вычислительное устройство;

– загрузка путей к данным. Формируются списки путей к изображениям и соответствующим текстовым файлам с ключевыми точками. Пути к файлам с ключевыми точками генерируются автоматически, заменяя расширение изображения на .txt и добавляя подкаталог detections. Это упрощает работу с размеченными данными и обеспечивает соответствие между изображениями и их разметкой;

– чтение и преобразование данных. Изображение открывается с помощью библиотеки PIL и преобразуется в RGB-формат [87-88]. Ключевые точки загружаются из

файла и инвертируются по оси Y для соответствия системе координат модели. Далее изображение и ключевые точки выравниваются относительно эталонных 3D-координат lm3d_std [89-90];

– цикл обработки изображений. Алгоритм последовательно перебирает изображения из каталога. Для каждого кадра проверяется наличие файла с ключевыми точками. При отсутствии разметки изображение пропускается. Если данные корректны, запускается процесс инференса, включающий предсказание параметров 3D-модели;

– предсказание параметров 3D-модели. Модель Deep3DFaceRecon_pytorch принимает изображения и ключевых точек, рассчитывая коэффициенты положения головы;

– визуализация результатов. Результаты визуализируются с помощью модуля Visualizer, включая отображение исходного изображения и реконструированного лица;

– сохранение результатов. Предсказанные параметры записываются в файл .mat для дальнейшего анализа. Полученные файлы визуализации сохраняются в туже директорию в формате .png;

– завершение работы. После обработки всех изображений освобождаются ресурсы, занятые моделью и визуализатором. Программа возвращает статус выполнения, информируя пользователя о завершении процесса;

Блок-схема алгоритма захвата движения головы рассказчика из опорного видео представлена на рисунках 29-30.



Рисунок 29 – Алгоритм захвата движений головы рассказчика

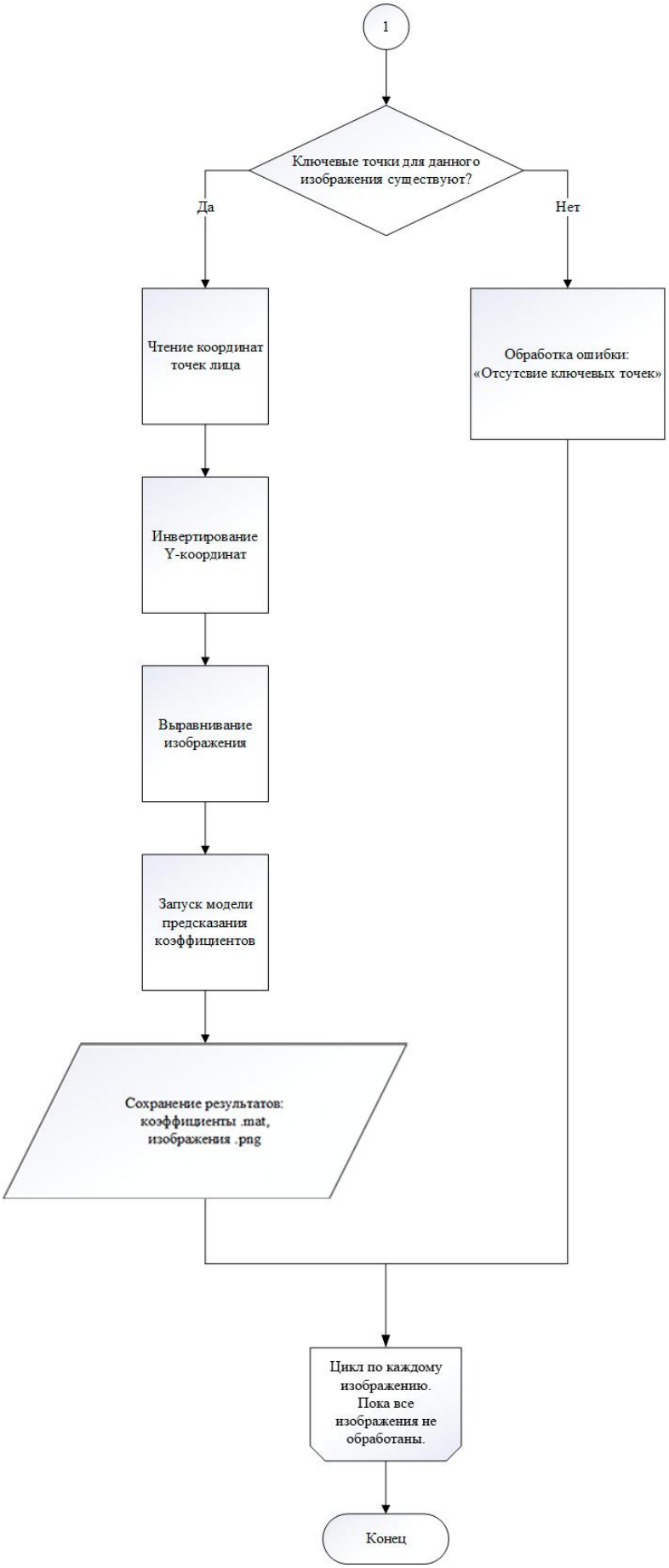


Рисунок 30 – Алгоритм захвата движений головы рассказчика

Алгоритм обрабатывает исходное видео, извлекая покадровые коэффициенты положения головы рассказчика. Эти коэффициенты точно описывают ориентацию (рыскание, тангаж, крен) и смещение головы в пространстве для каждого момента времени. Параллельно с извлечением данных, для каждого кадра генерируется специализированная визуализация. Она инструментом оперативного контроля качества, позволяя наглядно оценить точность соответствия параметров. После получения множества параметров движения головы необходимо объединить значения и привести к требуемому формату модели рендеринга. Для этого был разработан алгоритм преобразования исходных данных. Сначала из файлов .mat, где хранятся покадровые параметры, считываются значения, полученные на предыдущем этапе. Для корректной работы рендеринга модель требует дополнительных вычислений. Они связаны с определением центра лица и относительного масштаба для каждого кадра. Центр лица определяется напрямую из структуры ключевых точек. Масштаб вычисляется по принципу относительного изменения. За основу берется расстояние между двумя стабильными боковыми точками лица в первом кадре, которое принимается за базовое значение. Для каждого последующего кадра это же расстояние измеряется заново, и текущий масштаб рассчитывается как отношение текущего расстояния к базовому. Таким образом, для каждого кадра формируется вектор, содержащий пять ключевых элементов: ширину исходного изображения, высоту исходного изображения, вычисленный масштаб лица, X-координату центра лица и Y-координату центра лица. Множество этих векторов далее составят матрицу transform_params.

После этого обрабатываются исходные коэффициенты, предсказанные моделью Deep3DFaceRecon. Эти коэффициенты также структурируются в последовательные векторы, объединяются и записываются в матрицу coefficients. Алгоритм выполняет обработку для каждого исходного файла с параметрами. Он извлекает и преобразует данные, вычисляет необходимые дополнительные параметры на основе ключевых точек. Алгоритм объединения параметров и приведения в требуемый формат из следующих шагов:

– считывание данные. Из директории frames загружаются все .mat-файлы, представляющие отдельные кадры. Файлы содержат параметры 3D-модели и координаты ключевых точек. Формируется список для последовательной обработки;

– инициализация переменных. Создаются переменные для вычисления масштаба и временные вектора для хранения промежуточных значений transform_params и coefficients;

– организация циклической обработки. Алгоритм запускает последовательную обработку каждого файла из списка. Цикл продолжается до полной обработки всех файлов в директории;

– расчет масштаба. На основе координат 39-й и 42-й опорных точек рассчитывается расстояние. Для первого кадра это значение сохраняется как base_distance, а масштаб устанавливается в 1. Для последующих кадров масштаб вычисляется как отношение текущего расстояния к base_distance, что позволяет сохранить пропорции между кадрами;

– определение центра лица. В качестве центральной точки лица выбирается 33-я точка из массива ключевых точек. Извлекаются ее координаты X и Y в системе изображения. Эта точка используется для пространственного выравнивания:

– формирование параметров трансформации. Для каждого кадра создается вектор transform_params = [ширина изображения, высота изображения, масштаб, центр X, центр Y];

– компоновка коэффициентов. Из каждого .mat файла извлекаются параметры формы (id), мимики (exp), текстуры (tex), освещения (gamma), положения (trans). Параметры объединяются в единый вектор коэффициентов, формируя строку для матрицы coefficients;

– сохранение результатов. После обработки кадров и объединения всех векторов в матрицы, таблицы coefficients и transform_params сохраняются в единый .mat-файл;

– завершение работы. Алгоритм завершает выполнение, выводя сообщение о успешном сохранении данных.

Блок-схема алгоритма агрегации 3D параметров представлена на рисунках 31-32.

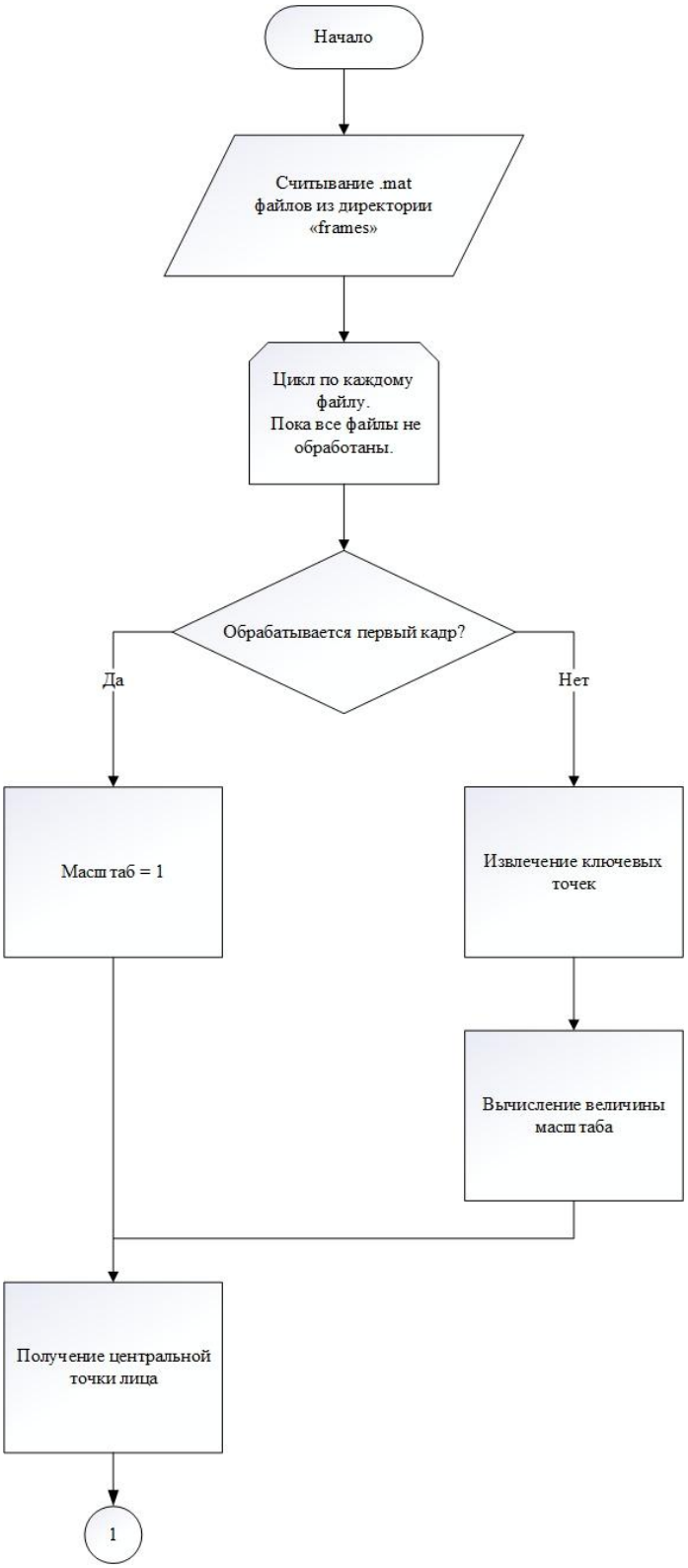


Рисунок 31 – Алгоритм агрегации 3D-параметров



Рисунок 32 – Алгоритм агрегации 3D-параметров

Алгоритм преобразует исходные данные в требуемый формат. Также от вычисляет дополнительные параметры необходимые для генерации. Особенностью полученных данных является возможность расширения исходных данных. Можно полученные дублировать коэффициенты, тем самым расширяя длину генерируемого фрагмента. Важно чтобы дублируемый фрагмент был 2-3 минуты, чтобы движение головы не повторялось слишком часто и не казалось странным.

Этот модуль вычисляет последние исходные данные, которые нужны чтобы создать видеолекцию с «аватаром». С помощью микродвижений головы дополняется реалистичность создаваемого персонажа [91].

2.5 Модуль генерации «аватара»

2.5.1 Структура модуля

Модуль генерации становится финальным звеном в процессе создания нейросетевого «аватара». Все ранее полученные и преобразованные данные находят своё воплощение. Для создания видео модели требуется 5 компонент. Исходное изображение служит основой для анимации, подвергаясь динамической трансформации. Для анимации речи используется список фонем. Каждая фонема кодируется определенным номером, после чего для каждой фонемы существует определенный паттерн произношения. Модель детально воспроизводит движение губ, языка и челюсти, автоматически синхронизируя из с речью. Это позволяет создать иллюзию живой речи. Чтобы повысить уровень реалистичности создаваемого персонажа используются коэффициенты особенностей лица. Они вносят в анимацию характерные асимметрии и индивидуальные особенности мимики, присущие человеку. Именно преднамеренное создание незначительных ассиметрий позволяют преодолеть эффект «зловещей долины», когда идеальные компьютерные модели воспринимаются как неестественные и отталкивающие. Это дополняется интеграцией микродвижений головы, рассчитанных на основе видеозаписи. Модель добавляет небольшие кивки, повороты и колебания, которые сопровождают эмоциональную речь. Сгенерированное видео синхронизируется со звуковой дорожкой лекции. Аудиофайл накладывается после визуального

рендеринга, который создавался с учетом произносимых слов. Таким образом получается добиться синхронизации видео сигнала с аудиопотоком.

Для удобства работы с модулем были разработаны профили, представляющие собой заранее подготовленные шаблоны. Каждый профиль включает изображение рассказчика и набор коэффициентов, которые описывают стилевые параметры мимики и движения головы. Использование профилей сокращает количество подготовительных действий, так как не требует повторного расчёта множества параметров для каждого нового запуска модели. Это позволяет быстро переключаться между различными «аватарами», сохраняя при этом индивидуальные особенности поведения и внешности. Также для профиля подготавливается пример голоса, чтобы при создании голосового аудиофайла речь «аватара» была одинаковой. Это позволяет сохранить тембр, интонацию и эмоциональную окраску. Также для профиля подготавливается пример голоса, чтобы при создании голосового аудиофайла речь «аватара» была одинаковой. Благодаря этой возможности, один и тот же профиль может использоваться для генерации видеолекций с одинаковым стилем речи и эмоциями.

2.5.2 Алгоритм создания «аватара»

Модуль для создания реалистичного «аватара» использует модели StyleTalk и FaceGenerator. Они работают отдельно и выполняют различные задачи. Работа начинается с модели StyleTalk, которая позволяет создавать реалистичную мимику [92]. Для создания мимики используется 3 функции, которые выполняются последовательно:

– content_encoder обрабатывает фонемы, извлечённые из аудиозаписи. Функция анализирует фонемы в контексте временного окна, охватывающего несколько кадров, чтобы учесть динамику. Произнесённые ранее фонемы влияют на выбор текущей, обеспечивая плавные переходы между звуками. На этом этапе формируется базовый уровень мимики, не учитывающий индивидуальные особенности говорящего;

– style_encoder корректирует мимику с учётом стилевых параметров. Он извлекает подготовленные стилевые параметры, чтобы придать естественности ранее созданным эмоциям. На этом этапе также учитывается движение головы, смещая положение рта, бровей и носа;

– decoder создаёт последовательность параметров выражений лица. Эта функция объединяет выходы content_encoder и style_encoder, создавая последовательность параметров выражений лица. Результатом работы декодера становится набор из 73 параметров для каждого кадра видео, описывающих положение мышц лица, форму губ и движения бровей.

Полученные данные передаются в модель FaceGenerator. Она преобразует числовые параметры в визуальные кадры. Модель использует статичное изображение лица рассказчика как отправную точку. Перед началом обработки изображение проходит этап подготовки, чтобы соответствовать требованиям модели. Сначала оно считывается с помощью библиотеки OpenCV, которая по умолчанию загружает изображения в цветовом пространстве BGR. Затем цветовая модель преобразуется из BGR в RGB [93]. Нейросетевая модель FaceGenerator обучена на данных, представленных в RGB формате. Далее изображение преобразуется с помощью библиотеки Pillow, которая позволяет представить его в виде numpy-массива [94]. Этот массив конвертируется в тензор, совместимый с фреймворком PyTorch [95-97]. При преобразовании в тензор, значения пикселей нормализуются и приводятся к диапазону от 0 до 1. Преобразования обеспечивают корректную работу модели, так как она рассчитана на строго определённый формат входных данных. Далее к ней применяется последовательность параметров мимики от StyleTalk. Для каждого кадра нейросеть вычисляет, как должны трансформироваться пиксели исходного изображения, чтобы воспроизвести заданные выражения. При этом учитываются анатомические ограничение, которые препятствуют резкому искажению пропорций лица и сохраняют реалистичность.

Мимика, полученная с помощью StyleTalk дальше используется моделью FaceGenerator. Она преобразует параметры мимики в кадры видео. Модель использует

исходное статичное изображение лица и последовательность параметров от StyleTalk. Для каждого кадра FaceGenerator определяет, как должны трансформироваться пиксели исходного изображения, чтобы воспроизвести заданные параметры мимики. Нейросеть применяет преобразования с учётом анатомических ограничений и сохранения фотореалистичности, генерируя последовательность правдоподобных выражений лица. Алгоритм рендеринга нейросетевого «аватара» состоит из следующих шагов:

– проверка выходной директории. Система проверяет наличие каталога output для хранения результатов. Если директория отсутствует, она создается автоматически, чтобы обеспечить корректное сохранение сгенерированных файлов. Это предотвращает ошибки при записи данных и упрощает организацию проекта;

– инициализация конфигурации. Загружаются параметры модели из файла конфигурации, включая пути к моделям. Это позволяет настроить работу моделей в соответствии с требованиями;

– загрузка моделей. Инициализируются нейросетевые модели StyleTalk и FaceGenerator. Модель StyleTalk используется для генерации мимики лица на основе множества параметров. Модель FaceGenerator создаёт кадры для конечного видео. Обе модели переводятся в режим оценки eval(), что гарантирует стабильность работы без обучения;

– цикл обработки фрагментов. Алгоритм формирует полные пути для каждого фрагмента. Для этого считывается название файла с фонемами, после чего заменяется название директории и расширение файла;

– загрузка списка фонем. Из JSON-файла считывается временная последовательность фонем. Данные используются для формирования мимики «аватара»;

– определение фонемного контекста. Функция get_audio_window выделяет контекстное окно фонем вокруг текущего кадра. Окно включает предыдущие и последующие фонемы для анализа динамики речи. Это обеспечивает плавность переходов при изменении эмоций;

– кодирование фонем. Контекстная выборка фонем подается на вход content_encoder модели StyleTalk. Она преобразует в фонемы в базовый уровень мимики, которая кодирует речь и произносимые слова;

– загрузка данных стиля. Из .mat файлов загружаются параметры особенностей лица и движения головы, которые используются для создание более реалистичного образа. Эти данные определяют индивидуальные черты анимированного лица;

– кодирование стиля. Параметры стиля обрабатываются style_encoder модели StyleTalk, формируя вектор стиля. Он определяет черты внешности и особенности рассказчика, которые остаются постоянными во время анимации;

– загрузка данных положения лица. Из файла извлекаются координаты ключевых точек лица. Эти данные необходимы для корректировки пространственной позиции лица в кадре, обеспечивая естественные повороты и наклоны;

– комбинирование параметров. Полученные вектора содержания и стиля комбинируются с данными положения лица. В результате получается матрица параметров, которая описывает экспрессию, стиль и положение для каждого кадра. Это позволяет создать согласованную анимацию речи и движения головы;

– сохранение параметров. Комбинированные параметры сохраняются в файл .npy, чтобы их можно было использовать для последующего этапа генерации с помощью модели FaceGenerator;

– преобразование исходного изображения. Базовое фото лица нормализуется и адаптируется под требования модели FaceGenerator. Также изображение конвертируется в тензор и масштабируется для совместимости;

– разделение данных на части. Для повышения эффективности обработки вся последовательность кадров разбиваются на блоки по 4 кадра. Это уменьшает нагрузку на GPU и ускоряет генерацию;

– генерация изображений. Модель FaceGenerator получает на вход преобразованное изображение и вектор параметров. На выходе формируются синтезированные кадры видео с учетом мимики, стиля и положения на экране. Каждый кадр представляет собой реалистичное лицо с определенной выражаемой эмоцией;

– нормализация кадров. Сгенерированные кадры преобразуются из диапазона [-1, 1] в значения пикселей [0, 255]. Это необходимо для возможности сохранения в стандартных форматах;

– соединение кадров. Все сгенерированные кадры объединяются в единое видео;

– добавление звука. К сгенерированному видео применяется аудиофайл с помощью библиотеки ffmpeg [98]. Это обеспечивает синхронизацию речи и движений лица, создавая естественное восприятие анимации;

– сохранение результатов. Готовое видео сохраняет в директорию output в формате .mp4. После обработки всех фрагментов алгоритм завершает работу, освобождая ресурсы и выводя сообщение о завершении.

Блок-схема алгоритма рендеринга нейросетевого «аватара» представлена на рисунках 33-36.

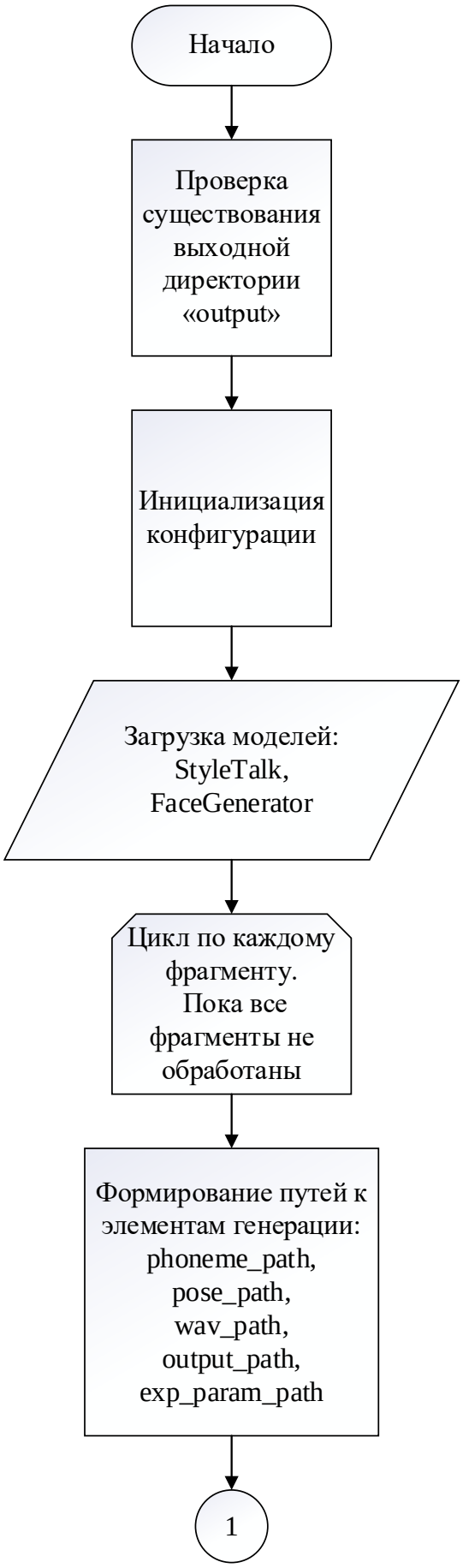


Рисунок 33 – Алгоритма рендеринга нейросетевого «аватара»

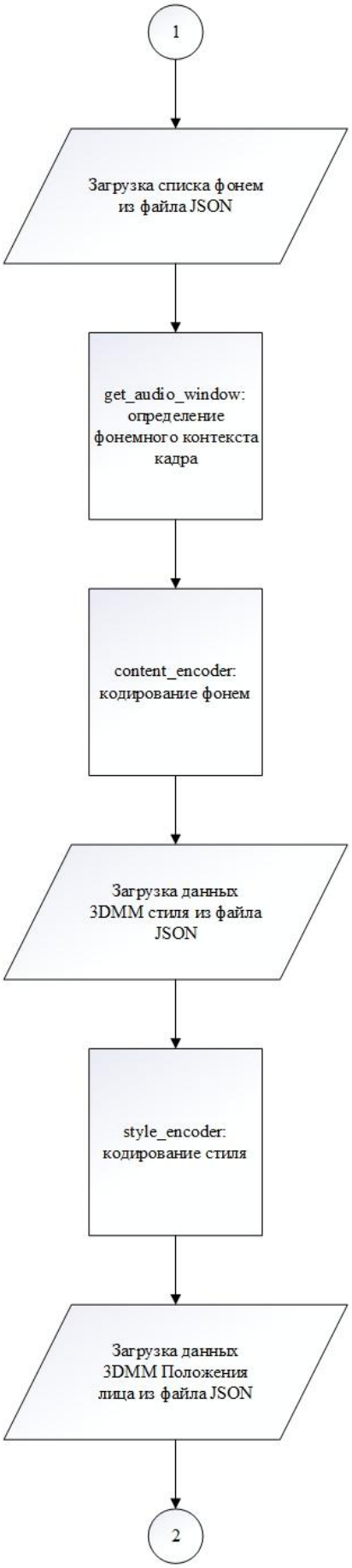


Рисунок 34 – Алгоритма рендеринга нейросетевого «аватара»



Рисунок 35 – Алгоритма рендеринга нейросетевого «аватара»



Рисунок 36 – Алгоритма рендеринга нейросетевого «аватара»

Алгоритм рендеринга позволяет создавать реалистичное видео персонажа, опираясь на множество параметров, включая мимику, движения головы и синхронизацию с аудио. Для создания одноминутного видеофрагмента требуется порядка 6 гигабайт видеопамяти, что делает процесс доступным даже на оборудовании среднего уровня. Благодаря этому алгоритм не требует использования дорогостоящего серверного или облачного оборудования. Его можно запускать на потребительской видеокарте с поддержкой технологии CUDA [99-100]. Для повышения производительности возможно уменьшение длины создаваемого фрагмента. Это снизит потребление видеопамяти, но незначительно увеличит общее время генерации. Модуль поддерживает конвейерную обработку данных, позволяя одновременно передавать на генерацию любое количество фрагментов. Алгоритм обрабатывает их автоматически, без необходимости вмешательства пользователя. Все разработанные модули для создания нейросетевого «аватара» образуют экосистему, которая позволяет из сырых данных (видео, аудиозапись, изображение) получать компоненты для генерации и создавать видео с «аватаром» на основе собственных данных.

3 Алгоритмизация и программная реализация образовательной цифровой площадки

3.1 Базы данных информационного обеспечения образовательной цифровой площадки

3.1.1 Архитектура базы данных

Проектирование базы данных является важным шагом при создании любого крупного проекта, где требуется управлять разнообразными данными. Реляционные СУБД позволяют организовывать взаимосвязи между таблицами, что обеспечивает гибкость при добавлении и извлечении информации через SQL-запросы. Центральной особенностью является использование JOIN-соединений [101]. с помощью внешних ключей можно объединять данные из разных таблиц, связывая пользователей с их результатами тестирования или видеолекции с соответствующими лабораторными работами. Такая структура помогает уменьшить количество дублированных данных. Вместо хранения одинаковой информации в нескольких местах используется ссылка на уникальный идентификатор. Однако необходимо соблюдать баланс. Чрезмерная нормализация приводит к усложнению запросов, снижению производительности и созданию избыточного количества таблиц, что сильно затрудняет поддержку системы [102]. СУБД хранит данные зарегистрированных пользователей и предоставляет библиотеку видеолекций для онлайн-просмотра. Она содержит систему тестирования с банком вопросов, которая динамически формирует для каждого студента индивидуальный, случайный набор вопросов. Также СУБД обеспечивает хранение данных системы лабораторных работ, организованной в виде множества блоков.

3.1.2 Таблицы общего назначения

Центральным элементом базы данных выступает таблица Users, хранящая сведения о зарегистрированных участниках системы. Участники делятся на три категории, включая студентов, изучающих видеолекции и проходящих тестирование.

Для студентов предусмотрены отдельные функциональные блоки: видеобиблиотека, система генерации тестов и модули лабораторных работ. Преподаватели могут загружать видеолекции и получают результаты тестирования и прохождения лабораторной работы студентов. Администраторы имеют доступ к коду платформы и управлению базы данных. Каждая запись включает в себя фамилию и имя пользователя, логин посредством электронной почты и пароль. Поля с типом varchar позволяют хранить данные переменной длины, а использование enum для поля user_type гарантирует корректность ролей. Таблица также поддерживает хранение медиа через ссылки и фиксирует время регистрации пользователя. Таблица Users содержит следующие поля:

– id первичный ключ, уникальный идентификатор пользователя (тип: int11, auto_increment);

– first_name имя пользователя (тип: varchar, максимальная длина 255 символов);

– last_name фамилия пользователя (тип: varchar, максимальная длина 225 символов);

– email электронная почта пользователя (тип: varchar, максимальная длина 225 символов);

– password_hash хешированный пароль пользователя (тип: varchar, максимальная длина 512 символов);

– created_at дата и время создания учетной записи (тип: timestamp);

– study_group название учебной группы пользователей (тип: varchar, максимальная длина 30 символов);

– user_type роли пользователей: студент, преподаватель, администратор (тип: enum('student', 'teacher', 'admin')).

Такая структура таблицы позволяет сохранить всю необходимую информацию о пользователе. Пароль сохраняется в виде хеша, что необходимо для обеспечения информационной безопасности, предотвращая возможность утечки учетных записей пользователей. Поле email является логином пользователя при авторизации и

позволяет восстановить забытый пароль. Также реализовано разделение по разным учебным группам для возможности разделения контента по возрасту.

Для хранения данных о видеолекциях необходимо создать отдельную таблицу Videos. Каждая запись должна содержать название видео, чтобы было понятно к какому курсу относится видеолекция. Кроме название также хранится краткое описание лекции, предоставляя дополнительную информацию о её содержании. Для хранения видео используется сторонние видеоресурсы. Предусмотрено поле для хранения длительности видео. Также как и видео, превью изображение хранится на видеоресурсе, ссылка на который сохраняется в записи. Также реализован флаг is_public, позволяющий скрывать материал, пока для него не готов тест или лабораторная работа. Каждая запись сохраняет метаданные, точное время загрузки видео, счётчик количества его просмотров и идентификатор пользователя, который загрузил материал. Метаданные необходимы для обеспечения дополнительного контекста видеолекции. Таблица Videos содержит следующие поля:

– id первичный ключ, уникальный идентификатор видео (тип: int11, auto_increment);

– title название видео (тип: varchar, максимальная длина 225 символов);

– description подробное описание видео (тип: text, текст произвольной длины);

– url ссылка на видео (тип: varchar, максимальная длина 512 символов);

– preview_image_url ссылка на превью изображение видео (тип: varchar, максимальная длина 512 символов);

– duration продолжительность видео в секундах (тип: int11);

– is_public флаг доступности видео (тип: int1, 0 скрывает видео, 1 делает видео общедоступным);

– upload_date дата и время загрузки видео (тип: datetime);

– views количество просмотров видео (тип: int11);

– user_id идентификатор пользователя, который загрузил видео (тип: int11, внешний ключ, связывающий с таблицей Users).

Такая структура таблицы Videos позволяет сохранить всю необходимую информацию для работы библиотеки видеозаписей на главной странице сайта. Использование сторонних ресурсов для хранения сильно снижают требования к объёму хранилища базы данных. Возможность скрывать видео из доступа позволяет полностью подготовить и проверить материал, перед его публикацией.

3.1.3 Таблицы системы тестирования

Следующие таблицы формируют основу системы проведения тестирований. Система тестирования платформы должна обеспечивать формирование уникальных тестов из банка вопросов. Для корректной реализации данного требования, необходимо корректно произвести нормализацию структуры базы данных. Вместо единой таблицы Tests данные будут разделены на несколько логически связанных таблиц. Первая таблица должна хранить общее описание теста, содержащее его основные параметры и метаданные. Вторая таблица призвана обеспечить сохранение самих вопросов, составляющих банк тестовой системы. Третья таблица необходима для хранения множества возможных ответов на эти вопросы. Кроме того, требуется отдельная таблица для записи результатов тестирования пользователей. Необходимо также обеспечить взаимосвязь между видео и тестом, для корректного отображения теста под нужной лекцией. Нормализованная структура базы данных обеспечит эффективное и гибкое хранение данных. Таким образом, архитектура системы тестов основана на нескольких взаимодействующих таблицах.

Основная таблица Tests должна содержать общее описание теста. В каждой записи хранится название теста, краткое описание теста для обеспечения дополнительного контекста. Также в этой таблице храниться количество вопросов для каждого теста студента. Хранится ограничение по времени для каждого теста, а также дата добавления теста в базу данных. Таблица Tests имеет следующую структуру:

– id первичный ключ, уникальный идентификатор теста (тип: int11, auto_increment);

– test_name название теста (тип: varchar, максимальная длина 255 символов);

– description краткое описание теста (тип: text, текст произвольной длины);

– time_limit ограничение времени на прохождение теста в минутах (тип: int11);

– num_questions количество вопросов в тесте (тип: int11);

– created_at дата и время создания теста (тип: timestamp).

Таблица Tests базовая в системе оценки знаний. Именно через нее производится взаимосвязь с другими таблицами, обеспечивая целостность данных. Для возможности отслеживания актуальности теста таблица содержит дату и время создания теста, а также краткое описание содержащихся в тесте вопросов. Эта дополнительная информация помогает студентам лучше подготовиться к вопросам. Для администраторов эти поля позволяют контролировать актуальности теста в контексте текущей видеолекции.

Для хранения вопросов тестов была создана таблица Questions. Она предназначена для хранения текста каждого вопроса, а также его типа и метаданных. Это обеспечивает дополнительную гибкость создания тестов, с возможностью добавления различных типов вопросов без изменения структур остальных таблиц. Эти вопросы и составляют банк тестовой системы. В таблице реализовано два типа вопросов. Вопросы с одним правильным вариантом и с несколькими обязательными вариантами, где для корректного ответа необходимо выбрать все верные ответы. Таблица также включает поля created_at и updated_at, фиксирующие дату создания и последнего редактирования вопроса. Это позволяет администраторам отслеживать актуальность содержимого и своевременно обновлять устаревшие вопросы. Такая структура обеспечивает эффективное хранение банка вопросов, поддерживает возможность повторного использования вопросов в разных тестах и позволяет легко масштабироваться, создавая новые типы вопросов. Таблица Questions имеет следующую структуру:

– id первичный ключ, уникальный идентификатор вопроса (тип: int11, auto_increment);

– question_text текст вопроса (тип: text, текст произвольной длины);

– question_type тип вопроса (тип: enum('single', 'multiple'), single вопрос содержит один правильный ответ, multiple несколько правильных ответов);

– created_at дата и время создания вопроса (тип: timestamp);

– updated_at дата и время последнего изменения вопроса (тип: timestamp).

С помощью записей этой таблицы формируется централизованный банк вопросов, который позволит создавать динамические тесты. Каждый вопрос в системе сопровождается предопределенными вариантами ответов, из которых студент выбирает правильные. При этом архитектура таблицы предусматривает возможность расширения функционала. Например, добавление поддержки открытых вопросов, где пользователь вводит ответ вручную, а не выбирает из предложенных.

Для добавления и хранение вариантов ответов была создана таблица Answers. Записи этой таблицы представляют собой варианты ответов на вопросы из таблицы Questions. Для реализации взаимосвязи создан внешний ключ, связывающий вариант ответа с вопросом. Также каждый вариант ответа содержит флаг правильности, 1 если вариант верный, 0 если неверный. Таблица также содержит поле display_order, которое позволяет установить порядок отображения вариантов ответов. Особенностью такой структуры хранения вопросов и ответов является обеспечение дополнительной вариативности. Можно хранить разное количество вариантов ответов к разным вопросам. Это повышает удобство в работе с банком вопросов, позволяя адаптировать тесты и варьировать сложность вопросов. в создании контрольных вопросов. Таблица Answers имеет следующую структуру:

– id первичный ключ, уникальный идентификатор ответа (тип: int11, auto_increment);

– question_id внешний ключ, ссылка на вопрос, к которому относится ответ (тип: int11);

– answer_text текст варианта ответа (тип: text, текст произвольной длины);

– is_correct флаг правильности ответа (тип: int1, если 1, то ответ верный, если 0 – ответ неверный);

– display_order порядок отображения вариантов ответа (тип: int11).

Таблица позволяет создать произвольное количество вариантов ответов для каждого вопроса. Это важно для разнообразия заданий. Имеется возможность задать порядок отображения ответов администратором, что полезно при составлении теста. Подготавливая тест правильный ответ можно всегда фиксировать первым, а затем с помощью изменения порядка перемешивать их перед показом студентам.

Для связи таблиц Questions, содержащей вопросы тестов, и Tests, описывающей их структуру, используется дополнительная таблица Test_questions. Она реализует гибкую связь через внешние ключи question_id и test_id, позволяя добавлять метаданные, не изменяя сами вопросы. Это позволяет избегать массового редактирования вопросов при обновлении тестов. Например, в этой таблице можно хранить сложность вопроса, его вес в тесте или порядок отображения, что расширяет функциональность без вмешательства в исходные данные. Такой подход обеспечивает вариативность при создании тестов и поддерживает нормализацию базы данных. Связующая таблица Test_questions имеет следующую структуру:

– id первичный ключ, уникальный идентификатор записи (тип: int11, auto_increment);

– test_id ссылка на тест (тип: int11, внешний ключ, связь с таблицей Tests);

– question_id ссылка на вопрос (тип: int11, внешний ключ, связь с таблицей Questions);

– display_order порядок отображения вопроса в тесте (тип: int11).

Дополнительные связующие таблицы часто используются для обеспечения гибкости и расширяемости системы. Они позволяют связывать один объект с несколькими другими объектами. Это используется в взаимосвязи вопроса и нескольких ответов. Также это позволяет адаптировать проект под новые требования без переписывания существующих данных. Еще одна связующая таблица Video_tests связывает видеолекцию с тестом. Это позволяет привязывать один тест к разным видео и наоборот. Это полезно при обновлении контента. Когда лекция перерабатывается, новое видео может использовать существующий тест, сохраняя преемственность проверки знаний. В тоже время старая версия видео остаётся доступной с тем же набором

вопросов. Такая структура исключает дублирование тестов для схожих материалов. Связующая таблица Video_tests имеет следующую структуру:

– id первичный ключ, уникальный идентификатор записи (тип: int11);

– video_id ссылка на видео (тип: int11, внешний ключ, связь с таблицей Videos);

– test_id ссылка на тест (тип: int11, внешний ключ, связь с таблицей Tests).

Описанные ранее таблицы формируют целостную систему для хранения тестов, вопросов и вариантов ответов. Они позволяют создавать оригинальные тесты и обеспечивают вариативность при создании контрольных вопросов. Вариативность выражаться как в сложности вопросов, так и в их разнообразии. Завершающим элементом системы стала таблица Test_sessions, которая фиксирует результаты прохождения тестов студентами. Она сохраняет не только итоговые баллы, но и детализированную информацию для разрешения спорных ситуаций. Каждая запись включает список вопросов и ответов в формате JSON, где сопоставляются question_id и текст ответа пользователя, что позволяет при необходимости восстановить полную картину тестирования. Дополнительно в таблице хранится время начала и завершения попытки. Это помогает анализировать временные затраты на тест и определять уровень его сложность. Имеются поля, которые хранят количество правильных ответов и общее количество вопросов. Это позволяет исключить ошибки, связанные с изменением состава теста после его создания. При изменении общего количества вопросов, полученный ранее балл будет недействительным. Для этого эта информация дополнительно дублируется. Также сохраняется полученный балл от 0 до 100, что позволяет задавать проходной порог. Поля test_id и user_id обеспечивают связь каждого сохраненного ответа с конкретным тестом и пользователем, гарантируя целостность данных и упрощая хранение. Таблица Test_sessions имеет следующую структуру:

– id первичный ключ, уникальный идентификатор сессии тестирования (тип: int11, auto_increment);

– user_id ссылка на пользователя (тип: int11, внешний ключ, связь с таблицей users);

– test_id ссылка на тест (тип: int11, внешний ключ, связь с таблицей tests);

– start_time время начала прохождения теста (тип: timestamp);

– end_time время окончания прохождения теста (тип: timestamp);

– total_score итоговая оценка за тест (тип: decimal(5,2), число с двумя десятичными знаками);

– correct_answers количество правильных ответов (тип: int11);

– total_questions общее число вопросов в тесте (тип: int11);

– answers_data детальная информация ответов пользователя в формате JSON (тип: longtext).

Таблица сохраняет значительный объем дополнительной информации, обеспечивая возможность ручной проверки работы алгоритмов. Это полезно при масштабировании, когда возможны ошибки в результате изменения кодовой базы. При этом система с детализированными ответами пользователя и дублированием общего количества вопросов продумана заранее.

Разработанная система хранения данных теста является высоко нормированной. Такая структура позволяет уменьшить количество избыточно дублирующийся информации без полезной нагрузки. Архитектура позволяет расширять таблицы и добавлять функционал, что критично для развивающегося проекта. Схема взаимодействия таблиц системы тестирования представлена на рисунке 37.

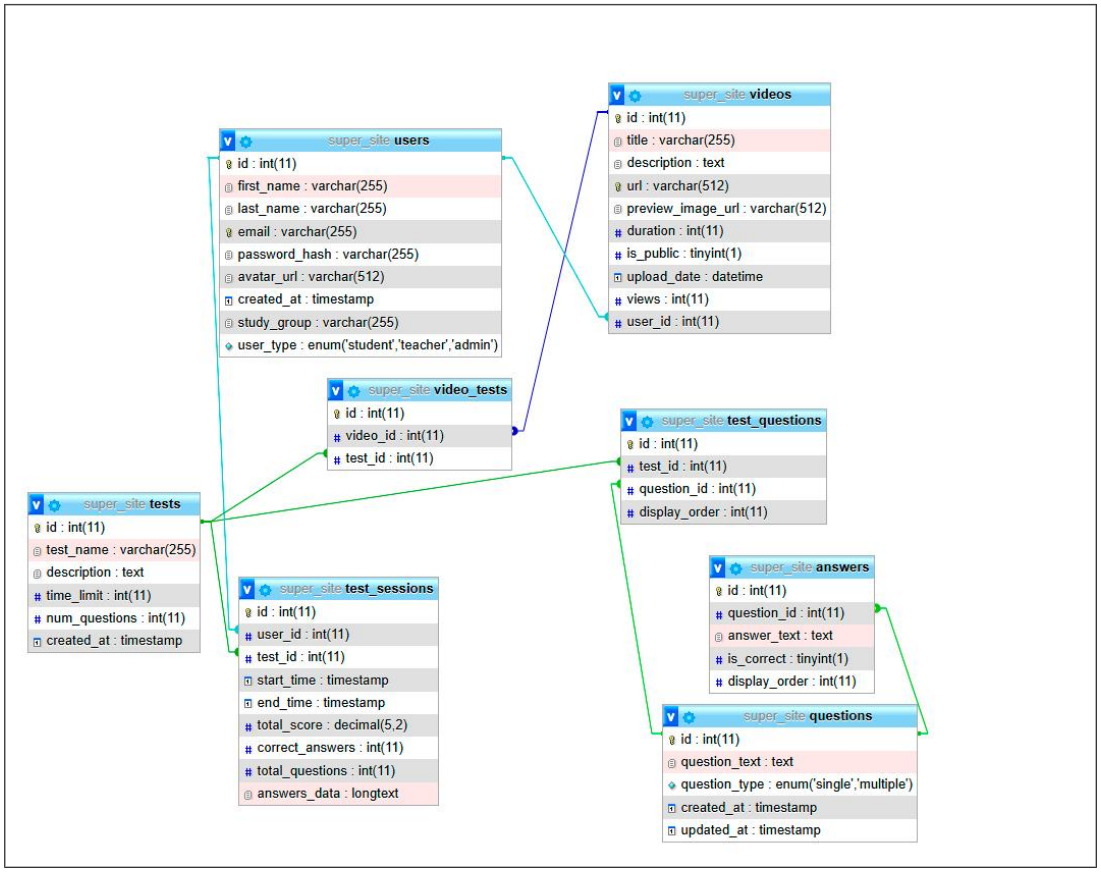


Рисунок 37 – Схема взаимодействия таблиц системы тестирования

Система получилась комплексной, с множеством связей между таблицами, что свидетельствует о тщательной проработке связей. Таблицы формируют устойчивую систему, где каждая таблица выполняет строго определенную роль. Взаимодействие между таблицами поддерживает целостность данных и адаптируется под рост проекта.

3.1.4 Таблицы системы лабораторных работ

Для образовательной платформы разработана система лабораторных работ, также использующая множество взаимосвязанных таблиц. Лабораторная работа состоит из заданий, каждое из которых основано на взаимодействии блоков. Задания предполагают установку блоков в правильную последовательность, выбор верных элементов и отсеивание неверных. Структура таблиц должна обеспечивать хранение готовых блоков для сборки в правильную последовательность.

Вся система связана с базовой таблицей lab, которая содержит записи с общим описанием задания. Имеются поля названия и краткого описания, поясняя суть и тему работы. Поле time_limit обеспечивает ограничение по времени на решение лабораторной работы. Временные рамки ориентированы не на усложнение, а на предотвращение списывания и перебора всех вариантов, что побуждает студентов к осмысленному решению задач. Это важно, так как лабораторные работы используют методы геймификации материала. Взаимодействие с блоками на практике помогает лучше усвоить лекционный материал, закрепляя знания через активное применение, а не заучивание. Также в таблице указывается общее количество заданий. Таблица lab имеет следующую структуру:

– id первичный ключ, уникальный идентификатор лабораторной работы (тип: int11);

– lab_name название лабораторной работы (тип: varchar, максимальная длина 255 символов);

– description краткое описание лабораторной работы (тип: text, текст произвольной длины);

– time_limit ограничение времени на выполнение лабораторной работы в минутах (тип: int11);

– num_tasks количество задач в лабораторной работе (тип: int11).

Такая структура основной таблицы уже включает все необходимые для лабораторных работ поля. При необходимости добавления новых метаданных достаточно расширить эту таблицу одним дополнительным полем. Это обеспечивает минимальные трудозатраты на модификацию системы. Нормализация базы данных гарантирует гибкость при развитии функционала платформы без перестройки архитектуры.

Каждая лабораторная работа наполняется множеством заданий. Для хранения заданий используется таблица lab_tasks. Таблица позволяет хранить задания и связывать множество заданий с одной лабораторной работой. Каждое задание содержит краткое название, в котором указана тема вопроса. Имеется поле для детального описания вопроса. Также имеется поле для определения типа задания. Система

поддерживает два ключевых типа. Первый тип с кодом «move» требует расставить все предоставленные блоки в единственно верной последовательности. Второй тип с кодом «move-wrong» требует удалить лишние, некорректные блоки из набора, а затем выстроить оставшиеся правильные блоки в требуемом порядке. Такая типизация позволила реализовать градацию сложности. Начальные задания используют простые комбинации из 3-4 исключительно правильных блоков, где требуется только их упорядочивание. Финальные задания усложняются до 8-10 блоков, которые включают специально добавленные неверные варианты, которые необходимо отсеять. Для поддержания актуальности контента каждая запись в lab_tasks содержит метки времени создания created_at и последнего обновления updated_at, позволяя отслеживать историю изменения заданий. Таблица lab_tasks имеет следующую структуру:

– id первичный ключ, уникальный идентификатор задачи (тип: int11);

– name название задачи (тип: varchar, максимальная длина 255 символов);

– description подробное описание задачи (тип: text, текст произвольной длины);

– type тип задачи (тип: text, тип «move» требует верной последовательности, «move-wrong» убрать лишние блоки и установить в правильном порядке);

– lab_id ссылка на лабораторную работу (тип: int11, внешний ключ, связь с таблицей lab);

– created_at дата и время создания задачи (тип: timestamp);

– updated_at дата и время последнего изменения задачи (тип: timestamp).

Таблица lab_tasks служит для хранения заданий, связанных с конкретной лабораторной работой. Она включает текст вопроса, тип задачи и внешний ключ lab_id. Использование внешнего ключа позволяет добавлять несколько заданий в одну лабораторную работу.

Для каждого задания необходимо создать множество блоков, которые содержат текстовое описание элементов. Например, это могут быть части SQL–запроса, этапы XSS–атаки или описание уязвимостей. Хранение этих блоков обеспечивается таблицей lab_blocks, где каждый блок содержит уникальное текстовое описание элемента.

Технически текст может быть любой длины, однако блоки с большим количеством текста занимают много места на экране и не позволяют создать много блоков. В таких случаях оптимальным решением является разделение большого блока на несколько маленьких, что улучшает восприятие и гибкость компоновки. Каждая запись содержит поле correct_order, который указывает правильную на позицию блока в последовательности, для получения правильного результата. Таким образом реализуется хранение правильного результата. Для реализации режима «move-wrong» было добавлено correct, указывающее корректность блока. Для неправильного блока присваивается значение correct_order = -1, чтобы он не участвовал в расчете правильного порядка. Дополнительно введена концепция двух линий для режима «move-wrong». Основная линия создана для составления правильной позиции. Вторая линия является «корзиной» для отбракованных блоков. Таблица lab_blocks имеет следующую структуру:

– id первичный ключ, уникальный идентификатор блока (тип: int11, auto_increment);

– task_id ссылка на задачу (тип: int11, внешний ключ, связь с таблицей);

– content текстовое наполнение блока (тип: text, текст произвольной длины);

– correct флаг правильности блока (тип: int1, 1 если блок правильный, 0 если неправильный);

– correct_order номер блока в правильной последовательности (тип: int11, для неправильного блока correct_order всегда -1);

– line линия, на которой располагается блок (тип: int11).

Такая структура таблицы lab_blocks позволяет создать произвольное количество блоков для каждого вопроса из таблицы lab_tasks через внешний ключ task_id. Для каждого блока хранится множество дополнительной информации, чтобы реализовать разные режимы работы заданий, позволяя разнообразить лабораторную работу.

Для лабораторной работы, как и в случае теста создана отдельная таблица для хранения результатов lab_results. Эта таблица связана с помощью внешних ключей с

таблицей Users, позволяя указывать id пользователя, который проходит лабораторную работу. Это позволяет однозначно идентифицировать пользователя не дублируя данные. Также lab_results связана с базовой таблицей lab, обеспечивая возможность связи с номером лабораторной работы. Каждая запись включает полный ответ пользователя в формате JSON, обеспечивая возможность ручного проверки работы алгоритма или разрешения спорных вопросов. Имеется поле, которое хранит дату и время начала пользователем попытки прохождения лабораторной работы, что полезно для контроля сроков выполнения лабораторной работы. Также сохраняется итоговый результат пользователя. Лабораторная работа не допускает частичного выполнения. Пользователь может сохранить результат только после решения всех заданий. Это реализовано через обязательную проверку всех заданий перед завершением. При этом пользователь имеет возможность неограниченно проверять и корректировать свои ответы, что усиливает обучающий эффект. Лабораторные работы спроектированы не как инструмент оценки, а как интерактивный метод закрепления материала. Путем многократного взаимодействия с элементами, лабораторная работа способствует глубокому усвоению знаний через практику. Таблица lab_results имеет следующую структуру:

– id первичный ключ, уникальный идентификатор записи (тип: int11, auto_increment);

– user_id ссылка на пользователя (тип: int11, внешний ключ, связь с таблицей users);

– lab_id ссылка на лабораторную работу (тип: int11, внешний ключ, связь с таблицей lab);

– results детальная информация о результатах выполнения лабораторной работы в формате JSON (тип: longtext);

– attempt_time дата и время когда пользователь начал выполнение лабораторной работы (тип: datetime);

– total_score итоговый балл за лабораторную работу (тип: int11).

Таблица lab_results сохраняет всю необходимую информацию для сбора результатов. Далее с помощью комбинированных запросов, использующих JOIN-соединения с таблицами Users, Lab, lab_tasks и lab_blocks, можно извлекать данные о выполнении лабораторных работ студентами в удобном формате. Такая организация данных обеспечивает целостность и доступность информации. Так же она способствует масштабируемости системы и возможности адаптации под новые задачи.

Для обеспечения взаимодействия системы лабораторных работ с видеолекциями была создана таблица video_lab_links. Она обеспечивает взаимосвязь таблицы videos с таблицей заданий lab_tasks. Это позволяет закреплять видеоматериалы за конкретными заданиями лабораторных работ, а не за всей лабораторной работой в целом. Отличительной особенностью такого подхода является гибкость в организации учебного процесса. Одно и то же видео может быть привязано к разным заданиям, а каждое задание может иметь собственные видеопояснения. Такое решение оправдано тем, что темы лекций могут пересекаться между собой, и такое решение обеспечит наименьшее дублирование информации. Это также упрощает обновление и адаптацию контента, так как изменения в связях касаются только конкретных заданий и не затрагивает структуру лабораторной работы в целом. Связующая таблица video_lab_links имеет следующую структуру:

– id первичный ключ, уникальный идентификатор записи (тип: int11, auto_increment);

– video_id ссылка на видео (тип: int11, внешний ключ, связь с таблицей videos);

– task_id ссылка на задачу лабораторной работы (тип: int11, связь с таблицей lab_tasks).

Таким образом была построена структура хранения данных для создания системы лабораторных работ на образовательной платформе. Каждая созданная таблица имеет свою определенную задачу. С учетом принципов нормализации получилось минимизировать дублирование информации. Каждая таблица выполняет свою специфическую задачу. Таблица lab_tasks управляет разнообразными типами заданий, lab_blocks хранит блоки, необходимые для их выполнения, а lab_results

фиксирует результаты прохождения студентами. Благодаря связям через внешние ключи система интегрируется с основными таблицами users, videos и lab, обеспечивая целостность данных и упрощая взаимодействие между компонентами. Связующая таблица video_lab_links реализует связь между таблицами, позволяя объединить теоретический материал и практические упражнения. Схема взаимодействия таблиц системы лабораторной работы представлена на рисунке 38.

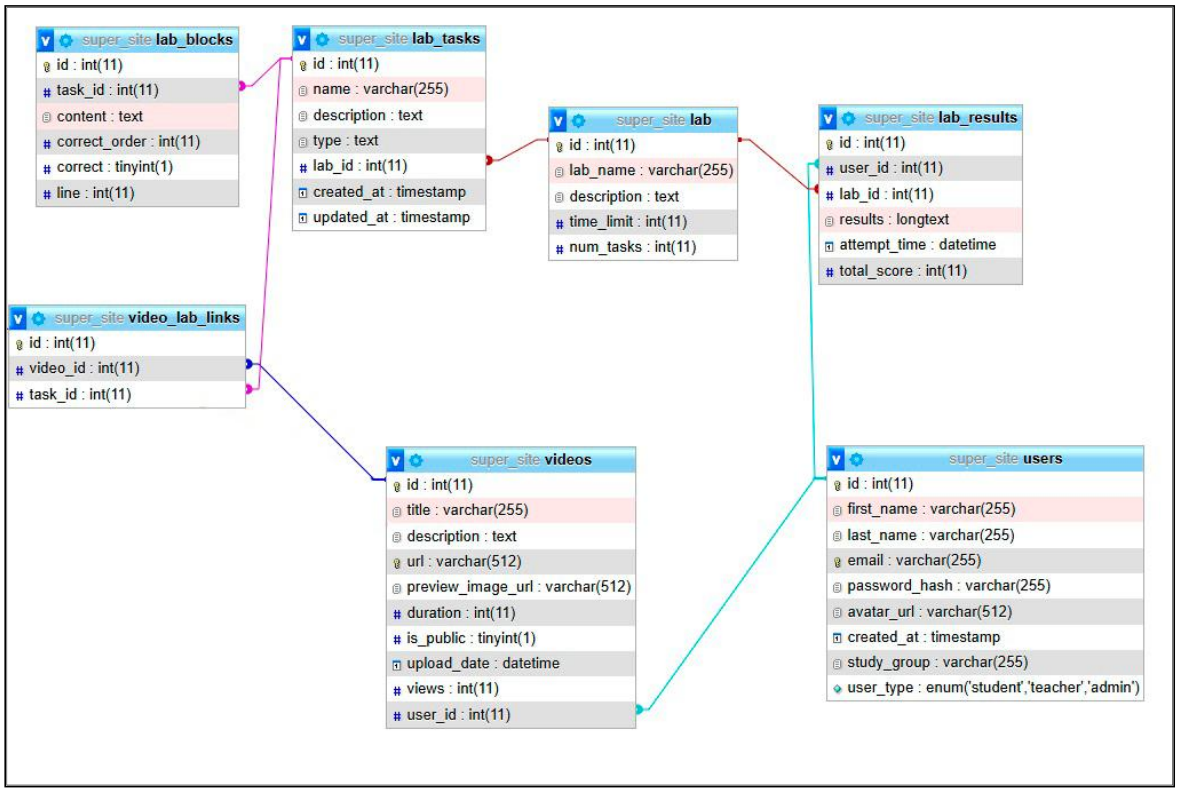


Рисунок 38 – Схема взаимодействия таблиц системы лабораторных работ

На схеме видно большое количество взаимосвязей между таблицами. Каждая таблица имеет хотя бы одну связь, что говорит о качественном нормировании системы и отсутствии избыточности. Такая структура гарантирует возможность получения любой запрашиваемой информации с помощью комплексных JOIN-запросов, обеспечивая целостность данных и эффективность выборки.

3.2 Образовательная цифровая площадка

3.2.1 Подсистема просмотра видеолекций

Образовательная платформа просмотра видеолекций реализована с использованием современных подходов к алгоритмическому проектированию и программной интеграции веб-приложений. Основу системы составляет комплексное взаимодействие между клиентской и серверной частями, обеспечиваемое через RESTful API [103]. Система предоставляет возможность онлайн-просмотра видеолекций, интегрированных с внешними платформами, такими как YouTube и Rutube. Это позволяет оптимизировать ресурсы хранения и обеспечивает стабильное воспроизведение контента. Для оценки знаний реализована встроенная система тестирования, основанная на общем банке вопросов. Тесты формируются случайным образом из заданного набора, что исключает повторяемость заданий и минимизирует риск списывания. Система лабораторных работ включает элементы геймификации, где пользователи взаимодействуют с интерактивными блоками, собирая решения в требуемой последовательности. Уровень сложности заданий постепенно нарастает, что способствует глубокому усвоению материала через практическое применение. Платформа интегрируется с внешними сервисами для хранения видео и отправки email-уведомлений, что позволяет оптимизировать ресурсы и обеспечить масштабируемость.

Главная страница приложения, которая отображается сразу после входа пользователя в образовательную платформу, представляет собой библиотеку видеолекций. Основной задачей этой страницы является структурированная подача всех доступных видеолекций. Для этого каждое видео сопровождается текстовым названием и превью изображением. Страница библиотеки видеолекций изображена на рисунке 39.

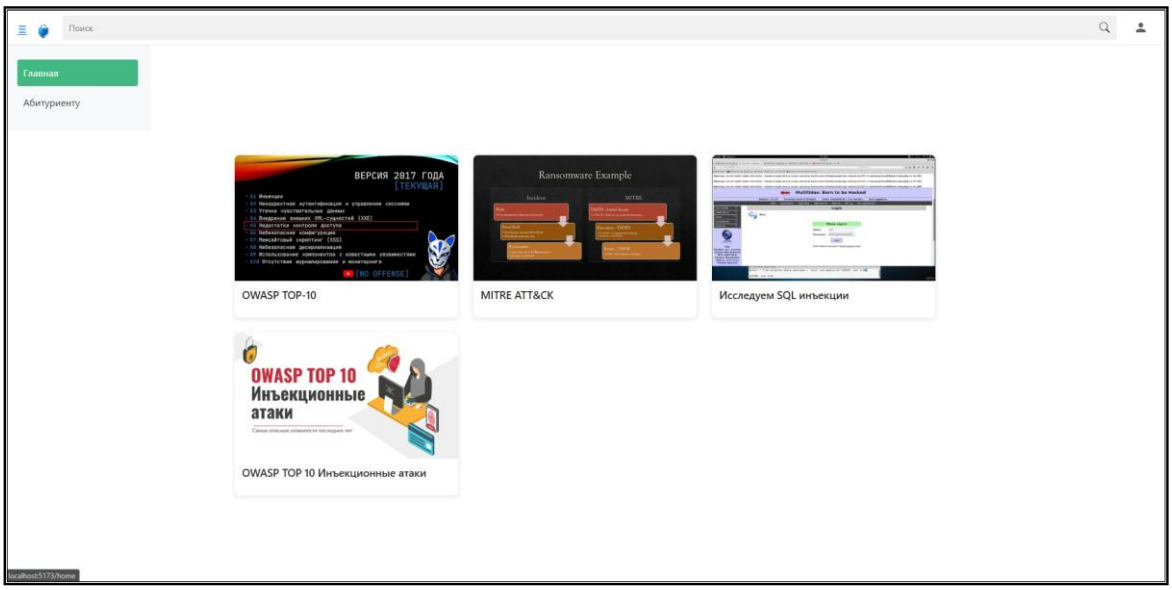


Рисунок 39 – Библиотека видеолекций

Для получения списка доступных видео осуществляется запрос к серверной части по адресу /api/videos. На сервере при этом выполняется SQL-запрос типа SELECT к таблице videos. Из неё извлекается идентификатор, название видео, ссылка видео, адрес превью изображения и длительность. Особенностью этого запроса является фильтрация записей с помощью флага is_public, который определяет, должно ли видео быть доступным для пользователей. Такой подход позволяет исключать из выборки материалы, которые находятся на стадии подготовки. После обработки запроса на клиентской стороне отображаются превью-изображения и названия опубликованных видео. При этом сами видеофайлы хранятся на сторонних видеоплатформах, что позволяет сэкономить ресурсы на хранение.

После выбора нужной видеолекции пользователь нажимает на неё и переходит на страницу видеоплеера. На этой странице отображается название видео и краткое описание. Ниже указываются условия для прохождения тестирования и размещены кнопки для запуска тестирования и лабораторной работы. Если материал для проведения теста или лабораторной работы не готов, кнопки становятся неактивными и отображаются серым цветом. Плеер поддерживает воспроизведение видео с платформ Rutube и Youtube. Данная страница выполняет роль связующего звена между

разными системами платформы. Страница просмотра видеолекций изображена на рисунке 40.



Рисунок 40 – Страница просмотра видеолекций

Для получения полной информации о видеолекции выполняются три последовательных запроса к API. Первый запрос направляется на эндпоинт /api/videos/:id, на который передаётся идентификатор видео, полученный ранее на странице библиотеки видеозаписей. В качестве ответа сервер возвращает название видеолекции, её описание и ссылку на видео. Далее клиентская часть приложения делает запрос к эндпоинту /api/video-tests/:videoId для получения данных о связанном с видео тестировании. В ответе указывается количество вопросов в тесте и ограничение по времени на прохождение. Последний запрос выполняется к эндпоинту /api/video-labworks/:videoId. В ответ сервер передаёт номер лабораторной работы, её название и общее количество заданий. Благодаря такому взаимодействию пользователь получает доступ к просмотру видеолекции, а также к прохождению тестирования и лабораторной работы. Разделение функционала на три независимых эндпоинта обеспечивает более читаемую структуру кода с минимальным количеством логики. Это упрощает поиск ошибок и позволяет эффективнее расширять функционал системы.

3.2.2 Подсистема аутентификации

Для прохождения тестирования и лабораторных работ пользователь должен пройти аутентификацию. С этой целью была реализована система аутентификации, которая включает страницу входа, страницы регистрации и восстановления пароля. Чтобы перейти на страницу входа, необходимо нажать кнопку в верхнем правом углу главной страницы. На странице входа пользователь вводит электронную почту в качестве логина и пароль. По умолчанию вводимые символы пароля маскируются. Для отображения пароля требуется нажать кнопку «Показать». Это позволяет дополнительно защитить пользователя от несанкционированного доступа. Страница аутентификации представлена на рисунке 41.

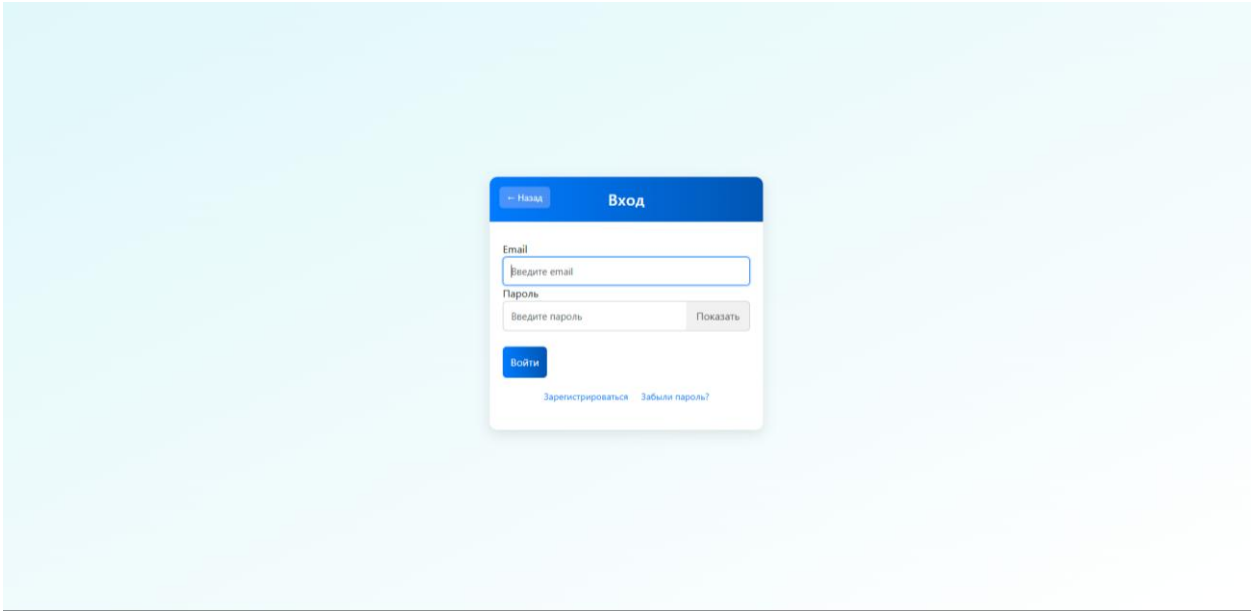


Рисунок 41 – Страница аутентификации

После ввода данных на странице аутентификации пароль пользователя хешируется с помощью алгоритма SHA-256 через библиотеку crypto-js. Преобразованный пароль передаётся на серверную часть приложения через эндпоинт /api/login. На сервере сначала проверяются обязательные поля email и password. Если одно из них отсутствует, возвращается ошибка 400. Затем происходит обращение к таблице users базы данных для поиска пользователя с указанным адресом электронной почты. Если пользователь найден, система сравнивает хеш введённого пароля с сохранённым в

базе данных значением, используя библиотеку crypto. При совпадении хешей генерируется JWT-токен, содержащий идентификатор пользователя и электронную почту со сроком действия 24 часа. Для безопасности секретный ключ для подписи токена берётся из переменной окружения JWT_SECRET. После успешной проверки токен и данные пользователя передаются обратно на клиентскую часть приложения. Сгенерированный токен сохраняется в контейнере для последующего автоматического входа. Блок-схема эндпоинта, реализующего систему аутентификации представлена на рисунке 42.

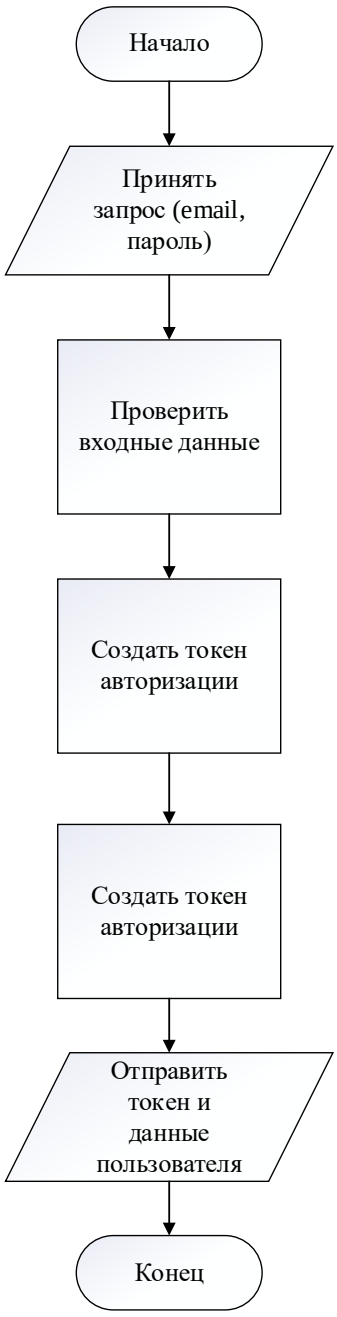


Рисунок 42 – Блок-схема эндпоинта системы аутентификации

Такая реализация исключает передачу пароля в открытом виде благодаря предварительному хешированию с использованием SHA-256. Также обеспечивается безопасное хранение данных аутентификации через JWT-токен, который автоматически обновляется по истечении срока действия 24 часа. Это минимизирует риски длительного хранения чувствительной информации на клиентской стороне.

Для создания учётной записи добавлена страница регистрации. Регистрация требует ввода имени, фамилии, электронной почты и выбора типа пользователя. При регистрации вводится пароль и дополнительно подтверждается для исключения опечаток при вводе. Пароль имеет строгие требования. Необходимо использовать минимум 8 символов, обязательное использование заглавных и строчных букв, цифр и спецсимволов. Для удобства при вводе выполнение требований динамически подсвечивается зелёным цветом. Вводимый пароль скрывается, предотвращая его просмотр. Высокие требования снижают риск взлома учетных записей методом перебора. Страница регистрации отображена на рисунке 43.

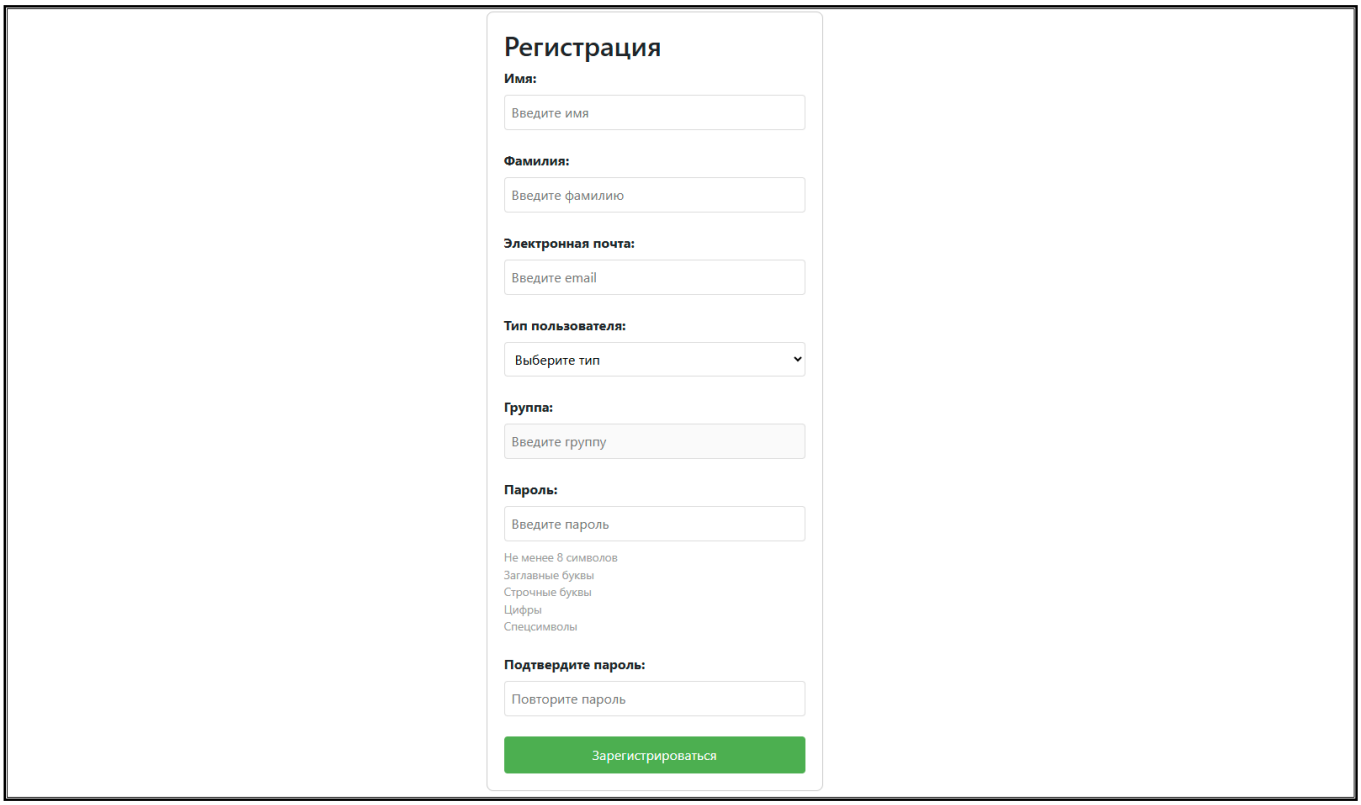


Рисунок 43 – Страница регистрации

После заполнения всех полей на странице регистрации и нажатия кнопки «Зарегистрироваться» данные пользователя передаются на сервер через эндпоинт /api/register. На серверной стороне выполняется запрос к базе данных для проверки наличия пользователя с указанным адресом электронной почты. Если такой пользователь уже существует, система возвращает ошибку 400. В случае отсутствия совпадения, имя, фамилия, электронная почта, тип профиля, учебная группа и хешированный пароль с помощью алгоритма SHA-256 добавляются в таблицу users используя INSERT-запрос. Все запросы к базе данных в веб-приложении реализованы с использованием параметризованных выражений, что предотвращает уязвимости, связанные с SQL-инъекциями. Успешная регистрация подтверждается ответом сервера со статусом 200 и соответствующим сообщением. Блок-схема эндпоинта, реализующего систему регистрации представлена на рисунке 44.

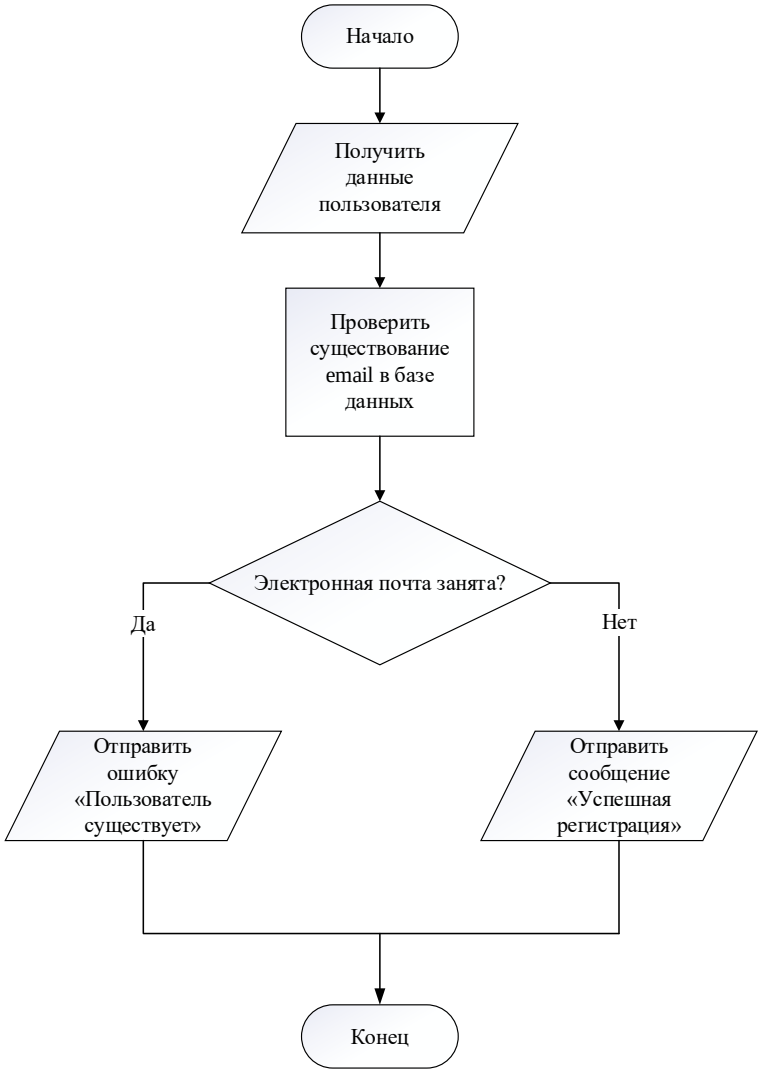


Рисунок 44 – Блок-схема эндпоинта системы регистрации

Система регистрации предоставляет пользователям возможность создать профиль для полноценного использования возможностей веб-приложения. В случае утраты доступа к учетной записи реализована функция восстановления пароля. На странице аутентификации предусмотрена кнопка «Забыли пароль», при нажатии на которую пользователь перенаправляется на форму восстановления. Далее необходимо ввести электронную почту профиля указанной при регистрации. После подтверждения создаётся уникальный токен восстановления пароля с ограниченным сроком действия, и на почту автоматически отправляется сообщение со ссылкой для сброса. Ссылка ведёт на защищённую страницу, где пользователь устанавливает новый пароль, соответствующий требованиям сложности. Изображение электронного письма сброса пароля изображено на рисунке 45.

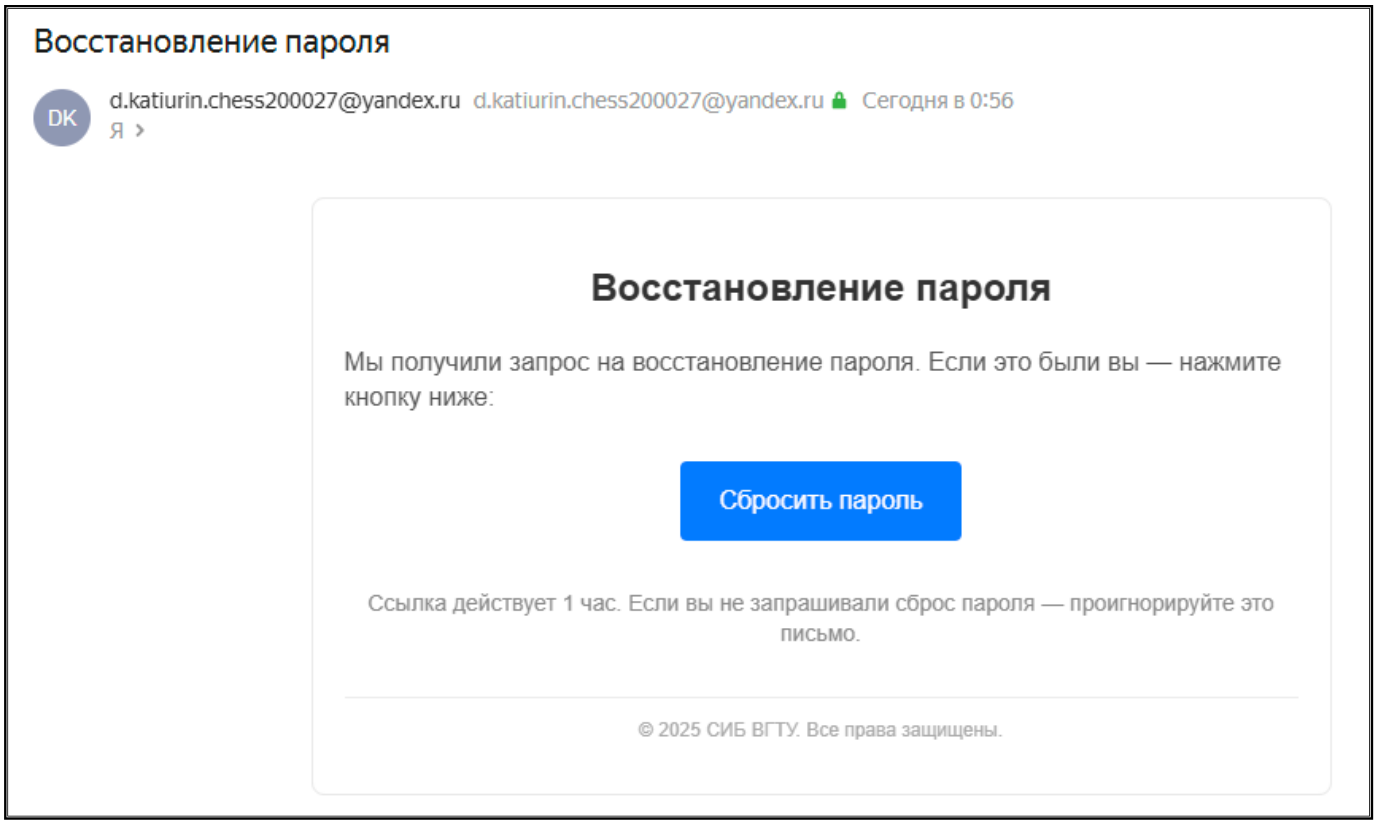


Рисунок 45 – Электронное письмо восстановления пароля

Для настройки системы автоматической отправки писем сброса пароля используется библиотека nodemailer, сконфигурированная для работы с почтовым сервером Yandex. Настройка библиотеки включает указание логина и пароля аккаунта, с которого осуществляется рассылка. Когда пользователь вводит свою электронную почту

и нажимает кнопку отправки, данные направляются на сервер через эндпоинт /api/reset-password. На серверной стороне выполняется дополнительный SQL-запрос для поиска пользователя с указанным адресом электронной почты. Если такой пользователь отсутствует, система возвращает сообщение «Пользователь не найден». При успешной проверке генерируется уникальный токен сброса пароля через библиотеку uuidv4. Он сохраняется в таблице password_reset_tokens базы данных и имеет ограничение по времени 60 минут. Это повышает безопасность, ограничивая возможность повторного использования токена. После этого формируется ссылка восстановления пароля, которая включает ранее созданный токен. Электронное письмо содержит красивый подготовленный текст с HTML-разметкой, загружаемый из файла reset-password.html. Готовое письмо отправляется на ранее указанный адрес. Блок-схема эндпоинта, реализующего эндпоинт для страницы сброса пароля представлен представлен на рисунке 46.

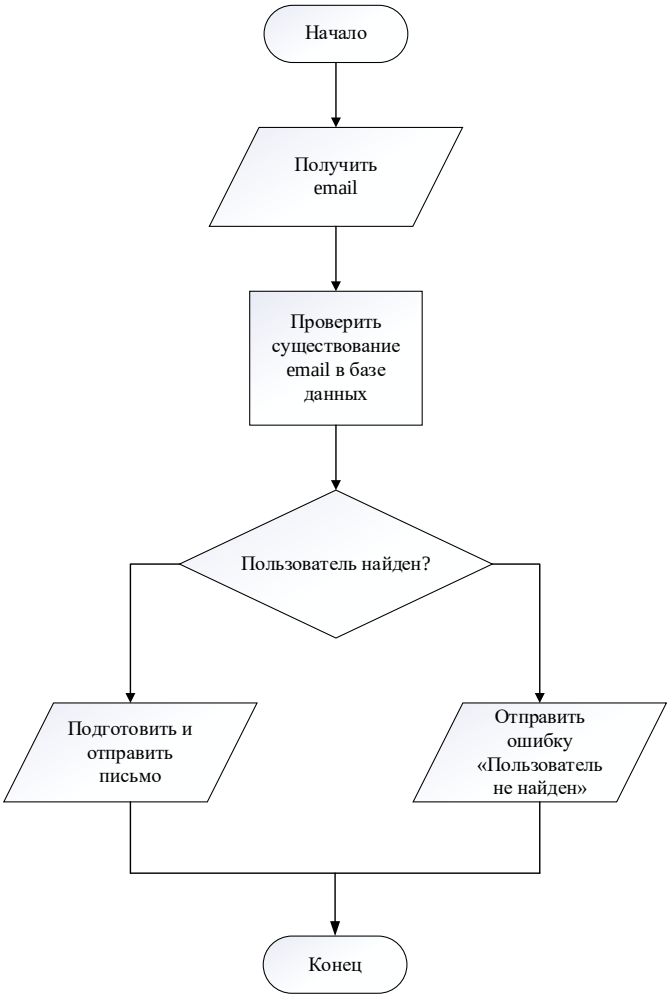


Рисунок 46 – Блок-схема эндпоинта системы сброса пароля

После перехода по ссылке из письма пользователь вводит новый пароль и подтверждает его. Пароль должен соответствовать необходимым требованиям. Необходимо использовать минимум 8 символов, обязательное использование заглавных и строчных букв, цифр и спецсимволов. При вводе выполненные требований динамически подсвечивается зеленым цветом для удобства. Корректно введённый пароль хешируется и отправляется на эндпоинт /api/reset-password/confirm. Хеширование перед отправкой гарантирует, что пароль не передаётся в открытом виде при сбросе. На сервере токен сброса сразу удаляется через DELETE-запрос. Это исключает повторное использование ссылки сброса пароля. После этого пароль пользователя обновляется через UPDATE-запрос. Автоматизированный сброс соответствует современным стандартам веб-разработки.

3.2.3 Подсистема тестирования

После успешной аутентификации пользователь получает полный доступ к функционалу веб-приложения, включая тесты и лабораторные работы. Система тестирования создаёт уникальные варианты с помощью банка вопросов, который создаётся для каждого теста. При создании теста вопросы случайно выбираются из банка, что при достаточном объёме вопросов исключает повторение вариантов. Для повышения объективности оценки каждый тест имеет ограничение по времени выполнения. Такой подход предотвращает списывание и обеспечивает честность результатов. Страница прохождения тестирования представлена на рисунке 47.

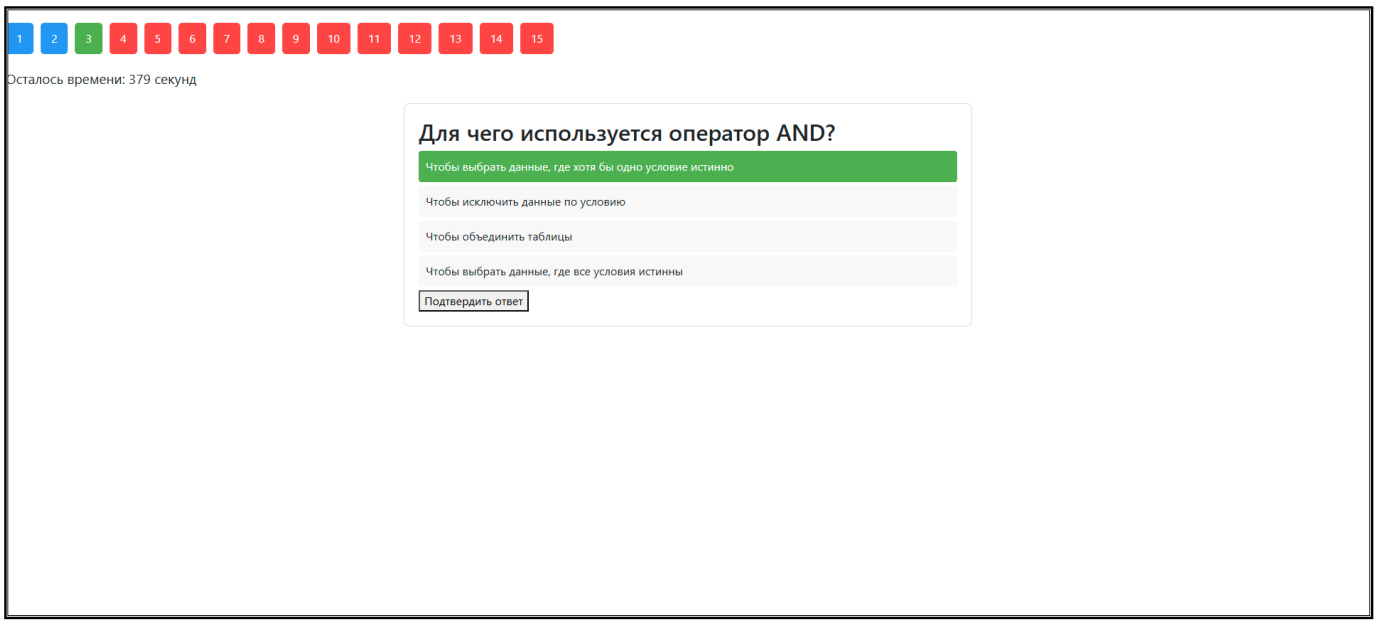


Рисунок 47 – Страница прохождения тестирования

Страница прохождения тестирования состоит из области с текущим вопросом, навигационной панели для перемещения между заданиями и таймера, отсчитывающего время до завершения теста. В окне вопроса отображается текст задачи и варианты ответов. Для выбора ответа пользователь кликает на нужный вариант и подтверждает выбор кнопкой «Подтвердить». Выбранный ответ можно изменить до окончания теста, а для удобства он выделяется зеленым цветом. Навигационная панель содержит кнопки, соответствующие каждому вопросу. Красным цветом отмечаются неотвеченные вопросы, зеленым текущий активный вопрос, а синим уже завершенные вопросы. Под навигационной панелью отображается таймер в секундах, по истечении которого тест завершается автоматически, а все ответы засчитываются. Если пользователь завершил тест раньше, он может нажать кнопку «Завершить тестирование» и перейти к странице результатов. На странице результатов отображается процент правильных ответов, количество верно решённых вопросов и детализация по каждому вопросу. Если вопрос решен верно он подсвечивается зеленым цветом, если неверно красным. Пример отображения результатов тестирования представлен на рисунке 48.

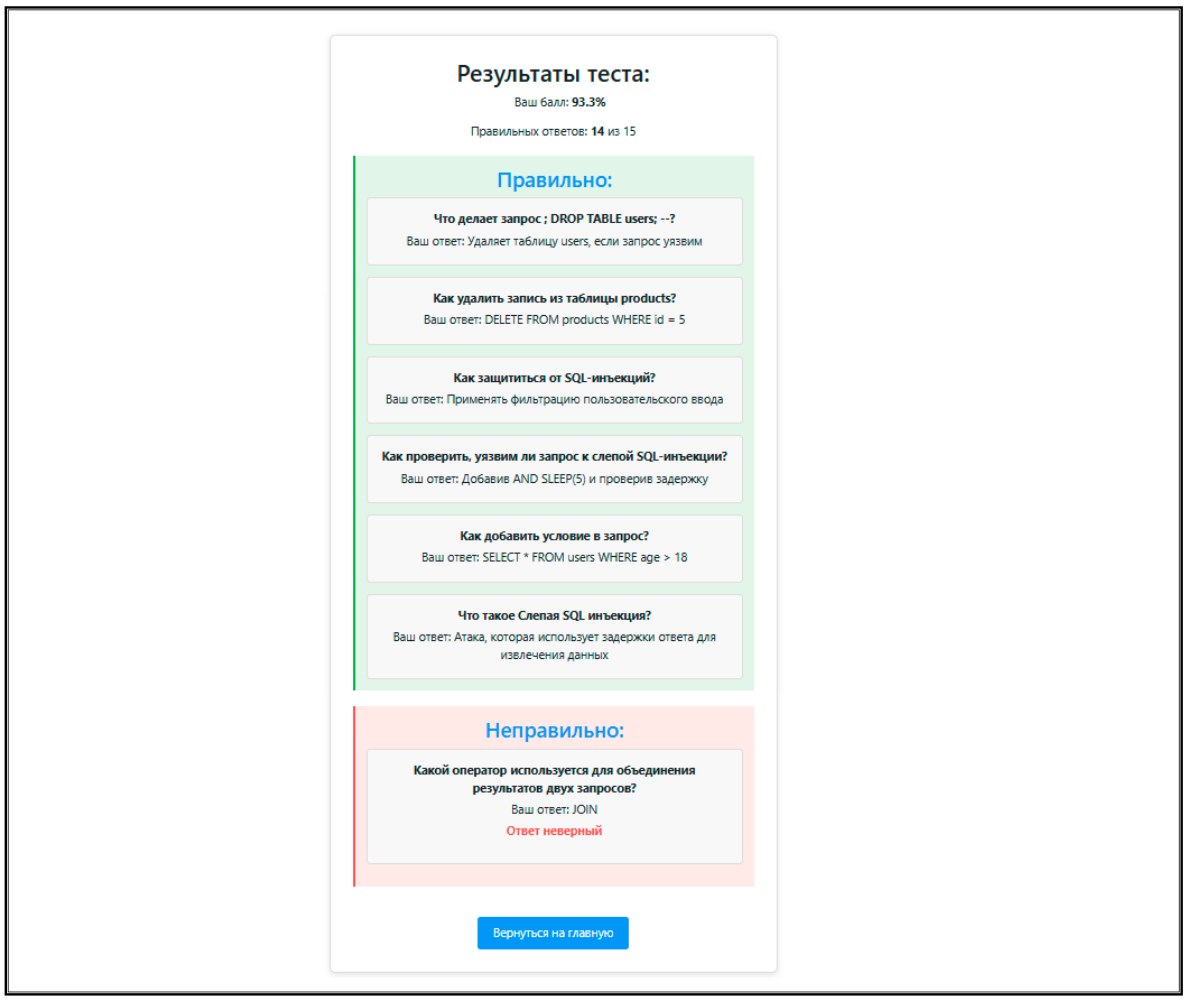


Рисунок 48 – Вывод результатов тестирования

Для получения данных о тесте используется эндпоинт /api/tests/:id, где дополнительно указывается идентификатор теста. Количество вопросов и длительность тестирования уже были получены на странице просмотра видеолекции. При обработке запроса сервер сначала проверяет существование теста. Далее выполняется комплексный SQL-запрос. В запросе используется оператор JOIN, который объединяет таблицы question, answer и test_questions для извлечения связанных данных. Для случайного выбора вопросов применяется функция RAND() в SQL-запросе. Ограничение количества заданий выполняется через LIMIT с учетом параметра num_questions из таблицы tests. Полученные данные структурируются с использованием GROUP_CONCAT и JSON_OBJECT, чтобы преобразовать ответы в JSON-формат на сервере. Затем структурированные данные на клиентской части приложения парсятся для отображения. Если тест с указанным ID не найден, сервер возвращает ошибку

404. Это сделано для дополнительной защиты, так как при отсутствии теста кнопка прохождения тестирования неактивна. осле завершения теста результаты пользователя сохраняются в базе данных через эндпоинт /api/test-sessions. Для этого используется INSERT-запрос, включающий идентификатор пользователя, номер теста, набранный балл, количество правильных ответов, общее число вопросов и детализированные данные о каждом ответе в JSON-формате. Чтобы преобразовать в JSON формат используется встроенная функция JSON.stringify. Запросы для сохранения результатов к эндпоинту /api/test-sessions требуют аутентификации по JWT-токену, что гарантирует защиту данных. Реализация системы тестирования с общим банком вопросов обеспечивает уникальность вариантов для студентов, что способствует глубокому усвоению материала.

3.2.4 Подсистема лабораторных работ

Для дополнительного закрепления разработаны лабораторные работы с элементами геймификацией. В ходе выполнения заданий студент взаимодействует с интерактивными блоками, собирая решения в требуемой последовательности. Уровень сложности заданий нарастает постепенно. На начальных этапах требуется упорядочить готовые блоки. Для последних самых сложных заданий нужно выбрать корректные элементы из общего банка блоков и расположить их в правильной комбинации. Результаты лабораторных работ сохраняются в базе данных аналогично тестам. Интерфейс лабораторных работ динамически адаптируется под размер экрана, обеспечивая удобство использования на любых устройствах. Страница прохождения лабораторной работы представлена на рисунке 49.

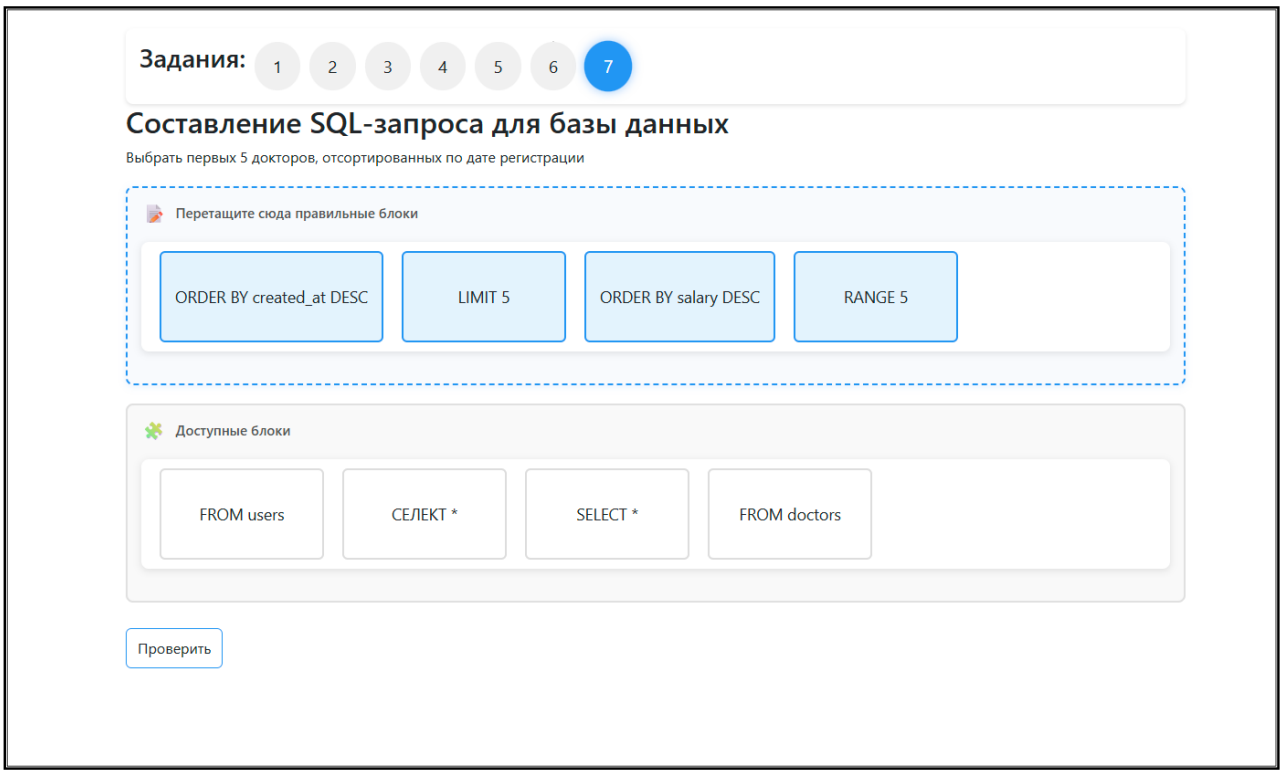


Рисунок 49 – Страница прохождения тестирования

Для работы системы тестирования клиентская часть приложения запрашивает задания лабораторной работы через эндпоинт /api/lab/:labId/tasks. На сервере выполняется SQL-запрос, который извлекает все задания для указанной лабораторной работы, предварительно отсортированные по дате создания ORDER BY created_at. Дополнительно для каждого задания отправляется отдельный запрос на эндпоинт /api/lab/tasks/:id. Он используется чтобы получить связанные с заданием блоки, из которых формируется задание. Блоки сортируются по полю correct_order, что позволяет определить правильную последовательность при проверке. Все блоки интерактивны. Их можно перемещать между строками и менять местами. После завершения задания пользователь нажимает кнопку «Проверить», которая сравнивает текущий порядок блоков с эталонным. При несоответствии выводится сообщение об ошибке, а при верном расположении задание считается выполненным. Результаты сохраняются через эндпоинт /api/lab/results. Он требует аутентификации по JWT-токену для аутентификации пользователя. Для сохранения используется INSERT-запрос в таблицу lab_results. Сохраняется идентификатор пользователя, номер лабораторной работы,

общий балл, полные данные о выбранных блоках и их порядке в формате JSON. Разделение системы на отдельные конечные точки API обеспечивает гибкость в её использовании и упрощает процесс расширения функционала. Также это позволяет вносить изменения в отдельные компоненты без необходимости переписывать крупные блоки кода, что снижает риск возникновения ошибок. Конечные точки изолированы, что позволяет реализовывать аутентификацию на уровне отдельных запросов через JWT токены, обеспечивая защиту данных при тестировании и сохранении результатов. Архитектура на основе микросервисов гарантирует отказоустойчивость. Сбой одного компонента не парализует всю систему.

В результате создана полноценная образовательная платформа с возможностью просмотра видеолекций в режиме онлайн. Интегрированная система тестирования с общим банком вопросов автоматически генерирует уникальные варианты, исключая ручное составление тестов. Дополнительно веб-приложение предоставляет инструмент создания лабораторных работ с интерактивными элементами, что способствует эффективному закреплению материала. Автоматическая проверка решений в реальном времени обеспечивает мгновенную обратную связь, ускоряя процесс обучения.

3.3 Обеспечение безопасности веб-приложения образовательной цифровой площадки

Для обеспечения комплексной безопасности веб-приложения образовательной платформы разработаны и внедрены меры защиты, охватывающие все компоненты системы. Реализована многоуровневая защита, включающая настройку базы данных, использование JWT-токенов для аутентификации и авторизации на сервере, а также строгую валидацию пользовательского ввода на клиентской стороне. Далее представлены технические детали каждого из этих решений.

Базовой защитой является использование протокола HTTPS, который зашифровывает передаваемые данные. HTTPS требует SSL-сертификата и секретного ключа [104]. Сертификат подтверждает подлинность домена, а ключ обеспечивает криптографическую защиту соединения. Для локальной настройки HTTPS был создан

самоподписанный SSL-сертификат сгенерирован утилитой mkcert [105-106]. Эта программа позволяет создать локальный центр сертификации для создания собственного SSL-сертификата. Сертификат выпущен для Common Name localhost, что позволяет проверить отсутствие ошибок безопасности при локальном тестировании. Для публикации в интернете потребуется заменить сертификат на вариант от доверенного центра [107]. Обновление сертификата потребует замены файла сертификата и секретного ключа, при этом сохраняя всю остальную используемую инфраструктуру без изменений.

В проекте реализовано хранение конфигурационных данных в файле. env, что позволяет динамически управлять параметрами приложения без изменения кода [108]. Уязвимые переменные, такие как IP-адреса серверной и клиентской частей, логины, пароли, секретный ключ JWT и другие уязвимые данные, вынесены в отдельный файл [109]. Использование. env файла обеспечивает простоту настройки. При изменении адресов или учетных данных достаточно обновить значения в одном месте. Файл. env исключается из системы контроля версий, что предотвращает утечку конфиденциальной информации. Такой подход позволяет безопасно передавать код для аудита и совместной разработки, сохраняя секреты вне репозитория.

Для обеспечения безопасности базы данных реализован комплекс мер, основанный на принципе наименьших привилегий. В рамках этого подхода создан пользователь app_user, который используется исключительно для взаимодействия с базой данных приложения. Для пользователя сгенерирован надежный 32-символьный пароль, включающий заглавные и строчные буквы, цифры и спецсимволы [110]. Пользователю app_user предоставлены минимально необходимые права на выполнение операций SELECT, INSERT, UPDATE и DELETE только для таблиц, используемых в приложении. При этом удаление и просмотр системных таблиц для этого пользователя заблокировано. Пароль root пользователя также был заменен на криптографически устойчивый.

При взаимодействии с базой данных установлено ограничение на максимальное количество одновременных подключений [111]. Это снижает нагрузку на сервер базы данных. Настройка также предотвращает DoS-атаки, связанные с массовым

использованием API-запросов, блокируя исчерпание доступных ресурсов. Ограничение реализовано с помощью параметра connectionLimit при создания пула подключения на серверной части приложения. Для взаимодействия с базой данных используются исключительно параметризованные SQL-запросы. Такой подход полностью исключает возможность SQL-инъекций, так как пользовательский ввод передается в запросах в виде отдельных параметров, а не подставляется напрямую в SQL-запрос [112]. База данных автоматически экранирует все потенциально опасные системные символы, которые используются для создания запросов. Также параметризованные запросы упрощают поддержку кода, улучшая читабельность запросов. Дополнительно улучшается производительность, за счет кэширования недавних запросов в базе данных.

Для предотвращения несанкционированного доступа к учетным данным все пароли в базе данных хранятся в захешированном виде с использованием алгоритма SHA-256. Этот криптографический алгоритм обеспечивает высокую степень защиты [113]. Он преобразует пароли в уникальные строки фиксированной длины, которые невозможно восстановить обратно. Хеширование не даёт возможности чтения паролей в открытом виде при компрометации базы данных. Дополнительно реализовано хеширование паролей на этапе передачи данных между клиентской и серверной частью приложения. Алгоритм SHA-256 выбран за счет эффективности вычисления и криптографической стойкости.

Учетные записи пользователей разделены на роли, что ограничивает доступ к функционалу в соответствии с правами. Студенты могут просматривать видеолекции, проходить лабораторные работы и тестирования. Преподаватели могут добавлять видеолекции, создавать тесты и лабораторные задания. При запросах проверяется соответствие роли и доступных действий, что предотвращает попытки несанкционированного использования функционала. Система ролей предотвращает возможность использования функций, не предусмотренных для определенной роли.

В приложении реализована система аутентификации на основе JWT-токенов [114-115]. Они используются для идентификации пользователя и сохранения состояния авторизации. После входа в систему токен сохраняется на стороне клиента, и при

повторном открытии приложения пользователь остается залогиненным. При взаимодействии с эндпоинтами API токен передается вместе с запросом в заголовке, что позволяет проверить аутентификацию пользователя. Срок действия каждого токена составляет 24 часа, после чего требуется повторная аутентификация. JWT-токены обеспечивают целостность данных. При изменении содержимого токена делает его недействительным, что станет известно сразу же при любом запросе. Сервер генерирует токен, подписывая его с использованием секретного ключа. При получении токена от расшифровывается, сверяется срок действия и извлекаются данные пользователя. Если токен истек или поврежден, токен блокируется. После этого пользователю необходимо заново авторизоваться в системе.

Для дополнительной защиты веб-приложения реализовано ограничение CORS [116]. Этот механизм контролирует, какие внешние домены могут отправлять запросы на сервер, исключая несанкционированный доступ. Сервер принимает запросы только от доверенных источников, указанных в белом списке. В этом списке указывается адрес источника, используемый протокол и порт. CORS предотвращает атаки типа CSRF (межсайтовая подделка запроса), блокируя запросы с непроверенных доменов. Настройка CORS включает указание разрешённых заголовков и используемых для работы методов. Такой подход обеспечивает защиту API от использования злоумышленниками в сторонних вредоносных приложениях [117].

На каждой странице, где пользователь вводит данные, реализована дополнительная валидация входных данных. Это позволяет предотвратить ошибки, связанные с некорректным вводом. Также проверка входных данных позволяет защитить приложение от XSS и SQL-инъекции. Проверка включает наличие данных, экранирование специальных символов и ограничение длины вводимых данных. Валидация выполняется на клиентской стороне, чтобы указать пользователю на неверно вводимые данные. На серверной стороне данные проверяются для защиты от многочисленных атак, связанных с переполнением буфера и внедрения вредоносного кода.

Для отслеживания и анализа HTTP-запросов на этапе разработки используется библиотека Morgan. Она автоматически записывает информацию о каждом запросе. Логи Morgan включают в себя используемый метод передачи (GET/POST), адрес

получателя, статус ответа, время обработки и размер ответа. Логирование позволяет оперативно выявлять ошибки в работе API.

При создании веб-приложения большое внимание было уделено информационной безопасности. Было реализовано большое количество защитных мер, которые позволяют обезопасить данные пользователей и инфраструктуру приложения. Использование HTTPS с SSL-сертификатами, параметризованных SQL-запросов, хеширования паролей и JWT-токенов обеспечило защиту на уровне передачи данных, базы данных и аутентификации. Валидация вводимых данных, ограничение прав доступа к БД и настройка CORS предотвратили уязвимости, связанные с несанкционированным доступом. Комплексный подход к безопасности включает логирование запросов, динамическое управление конфигурациями через. env-файл и ограничение на использование ресурсов, позволил создать устойчивую безопасную систему. Она защищена от множества типов атак, а также имеет потенциал для масштабируемости приложения в будущем.

ЗАКЛЮЧЕНИЕ

В ходе исследования, проведённого при подготовке выпускной квалификационной работы, были подробно изучены особенности разработки электронного нейросетевого курса, направленного на освоение основ кибербезопасности. Основным результатом работы стало создание специализированного нейросетевого инструментария, способного генерировать видеолекции произвольной продолжительности с возможностью персонализации визуального облика и звучания виртуального «аватара». Помимо этого, была разработана образовательная цифровая платформа, предназначенная для хранения и воспроизведения подготовленных материалов, а также для проведения апробации учебного курса с целью повышения его эффективности.

Таким образом, основная цель исследования алгоритмического и программного обеспечения нейросетевой реализации электронного курса по основам кибербезопасности была успешно решена посредством создания нейросетевого инструментария и цифровой платформы. Достижение поставленной цели потребовало последовательного решения ряда взаимосвязанных задач:

– анализ и техническое описание особенностей системы: были выявлены ключевые особенности и характеристики реализации электронного курса, включая параметры генерации нейросетевого «аватара», ограничения используемых технологий, а также архитектура взаимодействия компонентов разработанного инструментария. Определены структура и логика функционирования веб-приложения цифровой платформы, включая взаимосвязи её элементов;

– разработка нейросетевого инструментария генерации «аватара»: разработан и детально описан нейросетевой инструментарий, включающий пять функциональных модулей. Данный инструментарий обеспечивает создание персонализированного «аватара» с возможностью настройки его визуального облика, голоса, моделей поведения и передачи эмоций. Описаны ключевые алгоритмические решения и программные аспекты реализации системы;

– создание веб-приложения цифровой платформы: разработано и описано веб-приложение цифровой образовательной платформы. Платформа предназначена для размещения и демонстрации сгенерированных видеолекций, проведения апробации курса для оценки его эффективности и проверки знаний студентов. Архитектура платформы реализована как совокупность специализированных компонентов, взаимодействующих для формирования целостной системы.

Перспективы дальнейшего использования результатов исследования заключаются в применении разработанных нейросетевых технологий для создания адаптивных электронных курсов не только по кибербезопасности, но и в других технических и гуманитарных дисциплинах. Данные технологии позволяют гибко адаптировать контент под актуальные требования, автоматизировать проверку знаний и анализ эффективности обучения. Это открывает возможность не только для повышения уровня подготовки специалистов за счет персонализированных траекторий обучения, но и обеспечить широкий доступ к качественному образованию в различных предметных областях.

СПИСОК ЛИТЕРАТУРЫ

1 Евенко, Е. В. Цифровая трансформация образования и современные образовательные технологии / Е. В. Евенко, О. А. Гливенкова // Вестник Майкопского государственного технологического университета. – 2023. – Т. 15, № 4. – С. 83-92. – Текст: непосредственный.

2 Молодцова, Н. Г. От клипового мышления к визуальному: проблема и способ разрешения / Н. Г. Молодцова // Хроноэкономика. – 2021. – № 5(33). – С. 27-29. – EDN NBNDKZ. – Текст: непосредственный.

3 Сашенков, С. А. Использование феномена "Клиповое мышление" в образовательном процессе с использованием информационно-коммуникационных технологий / С. А. Сашенков // Общественная безопасность, законность и правопорядок в III тысячелетии. – 2020. – № 6-3. – С. 286-289. – Текст: непосредственный.

4 Панченко, Т. А. Организация учебной деятельности студентов с учетом особенностей клипового мышления / Т. А. Панченко // Проблемы высшего образования. – 2017. – № 2. – С. 141-143. – Текст: непосредственный.

5 Кубанцева, Д. И. Клиповое мышление в контексте образовательного процесса / Д. И. Кубанцева // Проблемы современного образования. – 2022. – № 6. – С. 70-79. – Текст: непосредственный.

6 Крохин, А. Л. О когнитивной теории мультимедийного обучения (CTML) Р. Майера и взаимосвязи вербальной и визуальной компонент лекционной презентации математических дисциплин / А. Л. Крохин // Новые образовательные технологии в вузе : Сборник тезисов докладов участников конференции, Екатеринбург, 18–20 февраля 2014 года / редактор: А.В. Поротникова. – Екатеринбург: Федеральное государственное автономное образовательное учреждение высшего профессионального образования Уральский федеральный университет им. первого Президента России Б.Н. Ельцина, 2014. – С. 807-816. – Текст: непосредственный.

7 Использование искусственного интеллекта в информационной безопасности URL: https://b-152.ru/iskusstvennyj-intellekt-v-ib – Текст: электронный.

8 AnyoneNet: Synchronized Speech and Talking Head Generation for Arbitrary Persons / X. Wang, Q. Xie, J. Zhu [et al.] // IEEE Transactions on Multimedia. – 2023. – Vol. 25. – P. 6717-6728. – Текст: непосредственный.

9 Flow2Flow: Audio-visual cross-modality generation for talking face videos with rhythmic head / Zh. Wang, W. He, Yu. Wei, Yu. Luo // Displays. – 2023. – Vol. 80. – P. 102552. – Текст: непосредственный.

10 Tan, Sh. Style2Talker: High-Resolution Talking Head Generation with Emotion Style and Art Style / Sh. Tan, B. Ji, Ye. Pan // Proceedings of the AAAI Conference on Artificial Intelligence. – 2024. – Vol. 38, No. 5. – P. 5079-5087. – Текст: непосредственный.

11 Теория поколений X, Y, Z, A URL: http://spsu.ru/news/4525-teoriya-pokolenij-x-y-z-i-a-al-fa-kto-est-kto – Текст: электронный.

12 Минакова, И. В. Конкуренция между США и Китаем за технологическое лидерство на мировом рынке искусственного интеллекта / И. В. Минакова, Е. В. Шергин // Экономические и социальные проблемы регионального развития в современных условиях : Сборник научных трудов Международной научно-практической конференции. В 2-х томах, Курск, 06 февраля 2025 года. – Курск: ЗАО "Университетская книга", 2025. – С. 375-380. – Текст: непосредственный.

13 Евдокименко, Е. Ю. Понятие информационного шума в социально-гуманитарных науках / Е. Ю. Евдокименко // Молодой ученый. – 2013. – № 10. – С. 564-566. – Текст: непосредственный.

14 Колобаев, В. К. Клиповое мышление - новый этап в развитии мышления современных учащихся / В. К. Колобаев, Т. А. Синицына // Наукосфера. – 2022. – № 2-1. – С. 57-62. – Текст: непосредственный.

15 Колобаев, В. К. Понятийно-логическое мышление vs клиповое мышление / В. К. Колобаев, Т. А. Синицына // Инновации. Наука. Образование. – 2021. – № 26. – С. 1159-1164. – Текст: непосредственный.

16 Богомолов, А. И. Геймификация образовательного процесса в высшем учебном заведении / А. И. Богомолов, В. П. Невежин // Новые информационные технологии в образовании: Сборник научных трудов 19-й международной научно-

практической конференции, Москва, 29–30 января 2019 года / Под общей редакцией Д.В. Чистова. Том Часть 2. – Москва: Общество с ограниченной ответственностью "1С-Паблишинг", 2019. – С. 129-131. – Текст: непосредственный.

17 Как создать цифровой аватар с помощью нейросетей URL: https://ssl-team.com/blog/kak-sozdat-tsifrovoy-avatar-s-pomoschyu-neyrosetey – Текст: электронный.

18 Проблемы генерации видео, 3D-моделей и синтеза речи URL: https://in-crussia.ru/understand/u-ii-bolshie-problemy – Текст: электронный.

19 Шестерин, Н. О. Феномен "зловещей долины" в цифровых медиа / Н. О. Шестерин // Медиа в современном мире. 63-и Петербургские чтения : Сборник материалов Международного научного форума. В 2-х томах, Санкт-Петербург, 18–20 апреля 2024 года. – Санкт-Петербург: ООО "Медиапапир", 2024. – С. 126-128. – Текст: непосредственный.

20 Илюшичев, В. Д. Когнитивные маркеры эффекта «Зловещей Долины» / В. Д. Илюшичев, А. В. Чистопольская, А. А. Курицын // Вестник Ярославского государственного университета им. П. Г. Демидова. Серия Гуманитарные науки. – 2024. – Т. 18, № 2(68). – С. 318-327. – Текст: непосредственный.

21 Методы ускорения обучения нейронных сетей URL: https://habr.com/ru/companies/magnus-tech/articles/840902 – Текст: электронный.

22 Talking-Head Generation with Rhythmic Head Motion / L. Chen, G. Cui, Z. Kou [et al.] // Lecture Notes in Computer Science. – 2020. – Vol. 12354 LNCS. – P. 35-51. – Текст: непосредственный.

23 X2Face: A network for controlling face generation URL: https://goo.su/2WUzRlI – Текст: электронный.

24 Pose-Controllable Talking Face Generation URL: https://github.com/Hangz-nju-cuhk/Talking-Face_PC-AVS – Текст: электронный.

25 One-shot Talking Head Generation with Controllable Speaking Styles URL: https://github.com/FuxiVirtualHuman/styletalk – Текст: электронный.

26 Dynamic Neural Radiance Fields for Monocular 4D Facial Avatar Reconstruction URL: https://github.com/gafniguy/4D-Facial-Avatars – Текст: электронный.

27 Nikitaeva, A. Y. Institutional framework for the development of artificial intelligence in the industry / A. Y. Nikitaeva, A. B. M. Salem // Journal of Institutional Studies. – 2022. – Vol. 14, No. 1. – P. 108-126. – Текст: непосредственный.

28 Васильева, А. А. Искусственный интеллект как инновационный тренд в обрабатывающей промышленности / А. А. Васильева, К. А. Назарова // XXXVII Международные Плехановские чтения : Сборник статей участников конференции. В 4-х томах, Москва, 09–12 апреля 2024 года. – Москва: Российский экономический университет им. Г.В. Плеханова, 2024. – С. 257-261. – Текст: непосредственный.

29 Оптимизация процессов инфлюенс-маркетинга с использованием искусственного интеллекта / Е. В. Мищенко, А. Б. Хмельницкая, А. Г. Шахбандарян [и др.] // Russian Economic Bulletin. – 2025. – Т. 8, № 1. – С. 329-339. – Текст: непосредственный.

30 Хэ, Ю. Будущее графического дизайна: влияние искусственного интеллекта и машинного обучения / Ю. Хэ // Оригинальные исследования. – 2025. – Т. 15, № 2. – С. 123-127. – Текст: непосредственный.

31 Доржиева, В. В. Национальные приоритеты развития промышленного искусственного интеллекта в условиях новых технологических вызовов / В. В. Доржиева // Вопросы инновационной экономики. – 2022. – Т. 12, № 1. – С. 111-122. – Текст: непосредственный.

32 История развития системы reCAPTCHA URL: https://habr.com/ru/articles/853748 – Текст: электронный.

33 Система Coqui text to speech URL: https://docs.coqui.ai/en/latest/models/xtts.html – Текст: электронный.

34 Обухов, Д. С. Клонирование и конверсия произвольного голоса с использованием генеративных потоков / Д. С. Обухов // Автоматика и телемеханика. – 2022. – № 10. – С. 80-93. – Текст: непосредственный.

35 Python Speech Recognition 2025 URL: https://www.assemblyai.com/blog/the-state-of-python-speech-recognition – Текст: электронный.

36 Обнаружение человека в кадре с помощью OpenCV URL: https://the-code.media/cv2 – Текст: электронный.

37 Поиск лиц на изображении с использованием OpenCV в Python URL: https://waksoft.susu.ru/2021/04/02/raspoznavanie-licz-s-ispolzovaniem-opencv-v-python – Текст: электронный.

38 Web Application Architecture URL: https://www.softkraft.co/web-application-architecture – Текст: электронный.

39 Как работают веб-приложения на HTML/JS/CSS URL: https://habr.com/ru/articles/450282 – Текст: электронный.

40 Как построить правильную архитектуру приложения URL: https://ru.hexlet.io/blog/posts/arhitektura-prilozheniya – Текст: электронный.

41 Зачем нужен React. Плюсы и минусы URL: https://habr.com/ru/sandbox/241172 – Текст: электронный.

42 Способы использования VUE URL: https://ru.vuejs.org/guide/extras/ways-of-using-vue – Текст: электронный.

43 Блокирующий ввод/вывод PHP URL: https://habr.com/ru/articles/553098/ – Текст: электронный.

44 Фреймворк Flask URL: https://skillbox.ru/media/code/freymvork-flask-kak-on-rabotaet-i-zachem-nuzhen – Текст: электронный.

45 Node.js плюсы и минусы URL: https://reg.cloud/blog/node-js-prostymi-slovami – Текст: электронный.

46 Система управления базой данных MariaDB URL: https://en.wikipedia.org/wiki/MariaDB – Текст: электронный.

47 15 причин использовать MariaDB URL: https://habr.com/ru/articles/855802 – Текст: электронный.

48 Построение API endpoint URL: https://tquality.ru/blog/kak-pisat-endpointy – Текст: электронный.

49 Базовые знания в REST API URL: https://wp-kama.ru/handbook/rest/basic – Текст: электронный.

50 Архитектура современных веб-сервисов и способы их защиты URL: https://goo.su/aWrsnZe – Текст: электронный.

51 Вовк, И. Ю. Разработка системы синтеза речи для русского языка при помощи глубоких нейронных сетей / И. Ю. Вовк, В. С. Гогорян // Межвузовская научно-техническая конференция студентов, аспирантов и молодых специалистов им. Е.В. Арменского, Москва, 18–28 февраля 2019 года. – Москва: Московский институт электроники и математики НИУ ВШЭ, 2019. – С. 61-63. – Текст: непосредственный.

52 Multimodal Emotion Recognition With Temporal and Semantic Consistency / B. Chen, Qi. Cao, M. Hou [et al.] // IEEE/ACM Transactions on Audio, Speech and Language Processing. – 2021. – Vol. 29. – P. 3592-3603. – Текст: непосредственный.

53 Сравнение цифровых аудиоформатов URL: https://en.wikipedia.org/wiki/Comparison_of_audio_coding_formats – Текст: электронный.

54 Головинов, А. О. Особенности преобразования текста современными системами синтеза речи / А. О. Головинов, В. В. Колмакова // Культурология, филология, искусствоведение: актуальные проблемы современной науки : сборник статей по материалам XIV-XV международной научно-практической конференции. Том № 9-10 (11) : Ассоциация научных сотрудников "Сибирская академическая книга", 2018. – С. 39-43. – Текст: непосредственный.

55 Чопоров, О. Н. О параллельных вычислениях на GPU, Nvidia CUDA / О. Н. Чопоров, Э. К. Юлис // Междисциплинарный вектор развития современной науки: теория, методология, практика : Сборник статей III Международной научно-практической конференции, Петрозаводск, 09 декабря 2020 года. – Петрозаводск: Международный центр научного партнерства «Новая Наука» (ИП Ивановская Ирина Игоревна), 2020. – С. 216-220. – Текст: непосредственный.

56 Жарков, Р. А. Системы компьютерного синтеза речи / Р. А. Жарков, Е. Г. Алексеев // XLIX Огарёвские чтения : материалы научной конференции: в 3 частях, Саранск, 07–13 декабря 2020 года. Том Часть 1. – Саранск: Национальный исследовательский Мордовский государственный университет им. Н.П. Огарёва, 2021. – С. 198-201. – Текст: непосредственный.

57 Баянбай, А. А. Исследование масштабируемости архитектуры микросервисов: сравнение с другими стилями архитектуры программного обеспечения и

факторами, влияющими на масштабируемость / А. А. Баянбай // Научный аспект. – 2024. – Т. 14, № 5. – С. 1886-1897. – Текст: непосредственный.

58 Определение фонемы URL: https://ru.wikipedia.org/wiki/%D0%A4%D0%BE%D0%BD%D0%B5%D0%BC%D0%B0 – Текст: электронный.

59 Идентификация голоса и анализ аудио URL: https://habr.com/ru/articles/572496/ – Текст: электронный.

60 Хусаинов, А. Ф. Программный комплекс для анализа речи (на примере распознавания фонем татарского языка) / А. Ф. Хусаинов // Доклады Томского государственного университета систем управления и радиоэлектроники. – 2013. – № 3(29). – С. 129-133. – Текст: непосредственный.

61 Конвейер данных URL: https://habr.com/ru/articles/900640 – Текст: электронный.

62 Sushil, N. Multilingual Video Viewing Subtitle to Audio Translator / N. Sushil // International Journal for Research in Applied Science and Engineering Technology. – 2024. – Vol. 12, No. 5. – P. 697-703. – Текст: непосредственный.

63 Распознование речи в видеоконтексте URL: https://habr.com/ru/companies/neoflex/articles/850608/ – Текст: электронный.

64 Руководство по задачам, возникающим при использовании речевой аналитики URL: https://habr.com/ru/articles/825162 – Текст: электронный.

65 Распознование устной речи URL: https://dl.acm.org/doi/10.1016/j.compeleceng.2022.108549 – Текст: электронный.

66 Библиотека распознавания фонем Allosaurus URL: https://github.com/xinjli/allosaurus – Текст: электронный.

67 Multilingual acoustic modeling for speech recognition based on subspace Gaussian mixture models / L. Burget, P. Schwarz, O. Glembek [et al.] // ICASSP, IEEE International Conference on Acoustics, Speech and Signal Processing - Proceedings : 2010 IEEE International Conference on Acoustics, Speech, and Signal Processing, ICASSP 2010, 14–19 марта 2010 года / sponsors: The Institute of Electrical and Electronics Engineers,

Signal Processing Society. – Dallas, TX, 2010. – P. 4334-4337. – Текст: непосредственный.

68 Рекурсивное чтение файлов из подпапок на Python URL: https://sky.pro/wiki/python/rekursivnoe-chtenie-faylov-iz-podpapok-na-python – Текст: электронный.

69 Описание формата JSON URL: https://habr.com/ru/articles/554274 – Текст: электронный.

70 Подгорный, П. А. Преимущества использования формата JSON в современных web-приложениях с точки зрения информационной безопасности / П. А. Подгорный // Вестник науки. – 2022. – Т. 4, № 11(56). – С. 242-245. – Текст: непосредственный.

71 3D Morphable Models URL: https://www.h-ka.de/en/isrg/projects/3d-morphable-models – Текст: электронный.

72 Генеративная техника 3DMM URL: https://en.wikipedia.org/wiki/3D_Morphable_Model – Текст: электронный.

73 Основы библиотеки OpenCV URL: https://opencv.org/about – Текст: электронный.

74 Быстрый старт и начало работы с изображениями OpenCV URL: https://habr.com/ru/articles/678260 – Текст: электронный.

75 Семенова, А. П. Анализ мимических выражений для задачи распознавания эмоций / А. П. Семенова, В. Н. Павлыш // Проблемы искусственного интеллекта. – 2020. – № 4(19). – С. 69-79. – Текст: непосредственный.

76 Семенова, А. П. Поиск ключевых точек лица для задачи распознавания эмоций / А. П. Семенова, В. Н. Павлыш // Информатика и кибернетика. – 2021. – № 1-2(23-24). – С. 59-64. – Текст: непосредственный.

77 Санников, А. О. Использование технологии нейронных сетей для построения 3D модели головы человека в дополненной реальности / А. О. Санников, Е. А. Слободюк // CPT2019 Международная научная конференция Нижегородского государственного архитектурно-строительного университета и Научно-исследовательского центра физико-технической информатики : Труды Международной научной

конференции, ЦарьГрад, 13–17 мая 2019 года. – ЦарьГрад: Нижегородский государственный архитектурно-строительный университет, Научно-исследовательский центр физико-технической информатики, 2019. – С. 277-283. – Текст: непосредственный.

78 Спижевой, А. С. Автоматическое определение статических и динамических выражений лица человека по изображениям и видео / А. С. Спижевой, Д. В. Баландин // Информационные системы и технологии ИСТ-2020 : Сборник материалов XXVI Международной научно-технической конференции, Нижний Новгород, 24–28 апреля 2020 года / Нижегородский государственный технический университет им. Р.Е. Алексеева. – Нижний Новгород: Нижегородский государственный технический университет им. Р.Е. Алексеева, 2020. – С. 1078-1086. – Текст: непосредственный.

79 Описание расположения ключевых точек лица URL: https://habr.com/ru/companies/okko/articles/417329/ – Текст: электронный.

80 Орлова, Ю. А. Анимирование эмоций человека и параметризация формы лица на основе модели Суррея / Ю. А. Орлова, В. Л. Розалиев, А. С. Стешенко // Известия Юго-Западного государственного университета. Серия: Управление, вычислительная техника, информатика. Медицинское приборостроение. – 2019. – Т. 9, № 4(33). – С. 76-92. – Текст: непосредственный.

81 3D-реконструкция лиц по фотографии и их анимация с помощью видео URL: https://habr.com/ru/companies/yandex/articles/421353 – Текст: электронный.

82 Нейросетевая модель Deep3DFaceRecon URL: https://github.com/sicxu/Deep3DFaceRecon_pytorch – Текст: электронный.

83 Киба, А. С. Совместная 3D реконструкция лица и плотностное выравнивание с помощью регрессионной сети карты положений / А. С. Киба // Актуальные проблемы авиации и космонавтики: Сборник материалов VI Международной научно-практической конференции, посвященной Дню космонавтики. В 3-х томах, Красноярск, 13–17 апреля 2020 года / Под общей редакцией Ю.Ю. Логинова. Том 2. – Красноярск: Федеральное государственное бюджетное образовательное учреждение

высшего образования "Сибирский государственный университет науки и технологий имени академика М.Ф. Решетнева", 2020. – С. 144-146. – Текст: непосредственный.

84 Шпаргалка по форматам файлов с данными в Python URL: https://habr.com/ru/articles/481140/ – Текст: электронный.

85 Библиотека визуализации nvdiffrast URL: https://nvlabs.github.io/nvdiffrast – Текст: электронный.

86 Использование библиотеки визуализации nvdiffrast URL: https://habr.com/ru/news/828946 – Текст: электронный.

87 Энциклопедия библиотеки Pillow URL: https://dvmn.org/encyclopedia/modules/pillow – Текст: электронный.

88 Андриенко, И. С. Работа с изображением с помощью модуля Pillow на языке программирования Python / И. С. Андриенко // Постулат. – 2021. – № 11(73). – Текст: непосредственный.

89 Обработка изображений с помощью библиотеки Python Pillow URL: https://habr.com/ru/articles/681248 – Текст: электронный.

90 Перегон картинок из Pillow в NumPy/OpenCV URL: https://habr.com/ru/articles/545850 – Текст: электронный.

91 Истратова, Е. Е. Разработка интеллектуальной системы для распознавания лиц на основе нейронных сетей / Е. Е. Истратова, Д. Н. Достовалов, Е. А. Бухамер // International Journal of Open Information Technologies. – 2021. – Т. 9, № 4. – С. 41-45. – Текст: непосредственный.

92 StyleTalk: One-Shot Talking Head Generation with Controllable Speaking Styles / Y. Ma, S. Wang, Zh. Hu [et al.] // Proceedings of the AAAI Conference on Artificial Intelligence. – 2023. – Vol. 37, No. 2. – P. 1896-1904. – Текст: непосредственный.

93 Преобразование цветовых RGB пространств OpenCV URL: https://docs.opencv.org/3.4/de/d25/imgproc_color_conversions.html – Текст: электронный.

94 Абдураманов, З. Ш. Методы обработки изображений в Python / З. Ш. Абдураманов, Г. С. Сейдаметов // Информационно-компьютерные технологии в

экономике, образовании и социальной сфере. – 2021. – № 4(34). – С. 80-86. – Текст: непосредственный.

95 PyTorch – фреймворк глубокого обучения URL: https://habr.com/ru/articles/334380 – Текст: электронный.

96 Бовшик, П. П. Анализ фреймворков для нейронных сетей / П. П. Бовшик // Молодежная школа-семинар по проблемам управления в технических системах имени А.А. Вавилова. – 2021. – Т. 1. – С. 19-21. – Текст: непосредственный.

97 Глубокое обучение на PyTorch URL: https://habr.com/ru/companies/piter/articles/354912 – Текст: электронный.

98 Библиотека декодирования видео ffmpeg URL: https://habr.com/ru/articles/467537 – Текст: электронный.

99 Якимов, В. Н. Технология CUDA и параллельная обработка данных на графических процессорах NVIDIA / В. Н. Якимов, С. С. Гулин // Международная конференция по мягким вычислениям и измерениям. – 2014. – Т. 2. – С. 238-240. – Текст: непосредственный.

100 Вычисления с GPU-ускорением на Python URL: https://habr.com/ru/companies/timeweb/articles/853578 – Текст: электронный.

101 SQL Join запросы URL: https://www.w3schools.com/sql/sql_join.asp – Текст: электронный.

102 Нормализация СУБД URL: https://habr.com/ru/articles/254773/ – Текст: электронный.

103 Воронин, В. В. Проектирование и разработка RESTful API веб-сервера / В. В. Воронин, П. А. Громов // Информационные технологии XXI века : сборник научных трудов / Ответственный за выпуск Е.А. Шеленок. – Хабаровск : Тихоокеанский государственный университет, 2015. – С. 474-480. – Текст: непосредственный.

104 HTTPS, TLS, SSL. Создание собственных X.509 сертификатов URL: https://habr.com/ru/articles/593507 – Текст: электронный.

105 Mkcert: валидные HTTPS-сертификаты для localhost URL: https://habr.com/ru/companies/globalsign/articles/435476 – Текст: электронный.

106 Сертификат SSL / Б. Р. Минахватов, В. А. Стригунов // Проблемы совершенствования организации производства и управления промышленными предприятиями: Межвузовский сборник научных трудов. – 2021. – № 1. – С. 217-222. – Текст: непосредственный

107 OWASP Transport Layer Security Cheat Sheet URL: https:// cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html – Текст: электронный.

108 Использование. env файла в разработке программного обеспечения URL: https://vc.ru/dev/1739633-fail-env-i-prichem-tut-bezopasnost – Текст: электронный.

109 Обеспечение безопасности MariaDB URL: https://www.rldp.ru/mysql/mysql80/security.html – Текст: электронный.

110 Рекомендаций по безопасности в MySQL / MariaDB URL: https://blog.sedicomm.com/2019/11/18/12-rekomendatsij-po-bezopasnosti-v-mysql-mariadb-dlya-linux – Текст: электронный.

111 Алиева, Е. М. Актуальность атак с использованием SQL-инъекций / Е. М. Алиева, Ш. З. Ширинова // In the World of Science and Education. – 2025. – № 3. – С. 163-167. – Текст: непосредственный.

112 Алгоритм шифрования SHA-256 URL: https://habr.com/ru/articles/729260 – Текст: электронный.

113 JSON Web Tokens URL: https://gist.github.com/zmts /802dc9c3510d79fd40f9dc38a12bccfc/339778999d35a9c81f115a3755ea3035681f2702 – Текст: электронный.

114 Рябов, Н. В. Управление доступом к защищенным ресурсам при помощи Json web tokens / Н. В. Рябов, М. В. Шавловский, В. В. Подкопаев // Системный анализ в науке и образовании. – 2018. – № 3. – С. 44-50. – Текст: непосредственный.

115 Cross-Origin Resource Sharing URL: https://developer.mozilla.org /ru/docs/ Web/HTTP /Guides/CORS – Текст: электронный.

116 Вафин, И. И. Значение CORS для безопасности веб-приложений / И. И. Вафин // Совершенствование науки и образования в области естественных и технических исследований : Материалы XXXVI Всероссийской научно-практической

конференции, Ставрополь, 21 декабря 2023 года. – Ставрополь: Общество с ограниченной ответственностью "Ставропольское издательство "Параграф", 2023. – С. 22-24. – Текст: непосредственный.

117 Использование библиотеки Morgan для Node.js URL: https://www.npmjs.com/package/morgan – Текст: электронный.